IGNOU MSC AI & ML · MCS-082 PROGRAMMING USING PYTHON

Block 1: Introduction to Python Programming

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

UNIT 1

Introduction to Python

ഈ unit-ൽ Python എന്ന language എന്താണ്, അത് എങ്ങനെ ഉണ്ടായി, C-യിൽ നിന്ന് എങ്ങനെ വ്യത്യസ്തമാണ്, എവിടെയൊക്കെ ഉപയോഗിക്കുന്നു എന്ന് പഠിക്കുന്നു. പിന്നെ module, package, library, framework എന്നിവയുടെ വ്യത്യാസവും, Python install ചെയ്ത് ആദ്യത്തെ program run ചെയ്യുന്ന വിധവും കാണാം.

- What is Python? Python vs C
- History of Python
- Implementations: CPython, Jython, IronPython, PyPy
- Need for Python and its applications
- Framework vs Library vs Package vs Module
- Popular Python libraries and frameworks
- Getting started: installation and IDEs
- Program structure in Python
- Running the first program

What is Python? Python vs C

Python: a high-level, general-purpose, interpreted programming language that is dynamically typed, garbage-collected and supports procedural, object-oriented and functional programming.

Python: dynamically typed ആയതും garbage-collected ആയതും procedural, object-oriented, functional programming എന്നിവയെ support ചെയ്യുന്നതുമായ ഒരു high-level, general-purpose, interpreted programming language.

Python is easy to learn, needs far fewer lines than C or Java, and is used heavily in AI, Machine Learning and data science. Because it ships with a rich standard library it is called a **"batteries included"** language. It can call operating-system services and libraries and can be extended with C, C++ or Java. NASA, Google, Meta, Netflix, YouTube and ISRO all use it.

മലയാളത്തിൽ

Python-നെ ഒരു "ready-to-cook" kit പോലെ കരുതാം: വേണ്ട സാധനങ്ങളെല്ലാം (standard library) packet-ൽ തന്നെയുണ്ട്, അതുകൊണ്ടാണ് "batteries included" എന്ന് പറയുന്നത്. C-യിൽ ഓരോ variable-ന്റെയും type മുൻകൂട്ടി declare ചെയ്യണം, memory നമ്മൾ തന്നെ നോക്കണം. Python-ൽ ഇതൊന്നും വേണ്ട: type run time-ൽ തനിയെ തീരുമാനിക്കും, ഉപയോഗം കഴിഞ്ഞ memory garbage collector തിരിച്ചെടുക്കും. പക്ഷേ interpreted ആയതുകൊണ്ട് C-യെക്കാൾ വേഗം കുറവാണ്.

Metric	Python	C
Type	High-level, general-purpose, interpreted	General-purpose, procedural, compiled
Speed	Slower (interpreted)	Faster (compiled to machine code)
Code size / syntax	Fewer lines, simple and readable	More lines, harder syntax
Variable declaration	Not needed; dynamically typed, a variable can hold different types at different times	Type must be declared when the variable is created
Error debugging	Easy; execution stops at the faulty statement and the error is shown at once	Whole source is compiled first, then all errors are listed
Function renaming	Supported: one function can be used under two names	Not supported
Memory management	Automatic garbage collection	Programmer manages memory explicitly
Applications	General purpose, AI/ML, web, even microcontrollers	Mostly hardware-related / system software
Built-in functions	Very large library	Limited library
Data structures	Built in (list with `append`, `insert`)	Must be implemented explicitly
Pointers	Not supported	Used frequently

EXAMPLE 1: DYNAMIC TYPING

The same variable holds an int, then a string, then a list. No declaration is needed.

```
x = 10
print(x, type(x))
x = "ten"
print(x, type(x))
x = [1, 2, 3]
print(x, type(x))
```

```
10 <class 'int'>
ten <class 'str'>
[1, 2, 3] <class 'list'>
```

ഇവിടെ x എന്ന ഒരേ variable-ൽ ആദ്യം integer, പിന്നെ string, പിന്നെ list വെക്കുന്നു. `type()` ഓരോ തവണയും ഇപ്പോഴത്തെ value-ന്റെ type കാണിക്കുന്നു. C-യിൽ `int x;` എന്ന് declare ചെയ്ത variable-ൽ string വെക്കാൻ പറ്റില്ല; Python-ൽ പറ്റും, അതാണ് dynamic typing.

EXAMPLE 2: "FUNCTION RENAMING" (ONE FUNCTION, TWO NAMES)

```
def add(a, b):
    return a + b

total = add          # same function, second name
print(add(2, 3))
print(total(2, 3))
print(total is add)
print(total.__name__)
```

```
5
5
True
add
```

Python-ൽ function-ഉം ഒരു object ആണ്. `total = add` എന്ന് എഴുതുമ്പോൾ പുതിയ function ഉണ്ടാകുന്നില്ല, അതേ function-ന് രണ്ടാമതൊരു പേര് കിട്ടുന്നു എന്ന് മാത്രം. `total is add` True ആണ്, ഒറിജിനൽ പേര് (`__name__`) ഇപ്പോഴും add തന്നെ. ഒരാളെ വീട്ടിൽ "കുട്ടൻ" എന്നും സ്കൂളിൽ "രാജേഷ്" എന്നും വിളിക്കുന്ന പോലെ.

EXAMPLE 3: BUILT-IN DATA STRUCTURES

```
nums = [10, 20]
nums.append(30)
nums.insert(0, 5)
print(nums)
print(len(nums), max(nums), sum(nums))
```

```
[5, 10, 20, 30]
4 30 65
```

C-യിൽ ഒരു dynamic list ഉണ്ടാക്കാൻ pointer-ഉം malloc-ഉം ഒക്കെ വേണം. Python-ൽ list built-in ആണ്: append അവസാനത്തിൽ ചേർക്കുന്നു, `insert(0, 5)` 0-ാം സ്ഥാനത്ത് 5 ചേർക്കുന്നു. `len`, `max`, `sum` എന്നിവ ready-made built-in functions ആണ്.

Exam tip: The unit says Python "executes only one instruction at a time". That is true for run-time errors, but CPython first compiles the whole file to bytecode, so a syntax error anywhere stops the program before even line 1 runs (see the two programs in topic 9). Write both points if asked about error debugging.

History of Python

Python: created by Guido van Rossum in 1989 as a successor to the ABC language; first released in 1991.

CPython: the default (reference) implementation of Python, written in C, which compiles source code to bytecode run by its virtual machine.

Python: ABC language-ന്റെ successor ആയി 1989-ൽ Guido van Rossum ഉണ്ടാക്കിയത്; ആദ്യ release 1991-ൽ.

CPython: C-യിൽ എഴുതിയ Python-ന്റെ default (reference) implementation; ഇത് source code-നെ bytecode ആക്കി സ്വന്തം virtual machine-ൽ run ചെയ്യുന്നു.

Python 2.0 (2000) added list comprehensions and a garbage collector that can collect reference cycles. Python 3.0 (2008) was a major revision that fixed design problems of Python 2 but is **not fully backward-compatible**, so much Python 2 code does not run unchanged on Python 3. Python 2.7 (2010) was the last 2.x release and support ended in 2020. Python is developed by a global open-source community and managed by the non-profit **Python Software Foundation (PSF)**. The unit notes Python 3.14.x as the current series (June 2026); download from python.org.

മലയാളത്തിൽ

ഒരു സിനിമയുടെ "part 2" പോലെയാണ് Python 3: കഥ അതേ, പക്ഷേ പല നിയമങ്ങളും മാറി. അതുകൊണ്ട് Python 2-ൽ എഴുതിയ പഴയ code Python 3-ൽ മാറ്റമില്ലാതെ ഓടില്ല. ഇന്ന് എല്ലാവരും Python 3 ആണ് ഉപയോഗിക്കുന്നത്; Python 2-ന്റെ support 2020-ൽ അവസാനിച്ചു. Python-നെ നോക്കി നടത്തുന്നത് Python Software Foundation എന്ന non-profit സംഘടനയാണ്.

1989: Guido van Rossum starts Python (successor to ABC)
1991: first release
2000: Python 2.0 (list comprehensions, cycle-collecting GC)
2008: Python 3.0 (major revision, not backward-compatible)
2010: Python 2.7, last 2.x release; support ended 2020

EXAMPLE: PYTHON 2 CODE ON PYTHON 3

Python 2 wrote `print` as a statement. On Python 3 it is a syntax error:

```
print "Hello from Python 2"
```

```
  File "py2.py", line 1
    print "Hello from Python 2"
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?
```

The Python 3 version, plus the change in division:

```
print("Hello from Python 3")
print(7 / 2)
print(7 // 2)
```

```
Hello from Python 3
3.5
3
```

Python 3-ൽ `print` ഒരു function ആണ്, അതുകൊണ്ട് brackets നിർബന്ധം. Python 2-ൽ `7 / 2` 3 തരുമായിരുന്നു; Python 3-ൽ `/` എപ്പോഴും decimal result (3.5) തരും, പൂർണ്ണസംഖ്യ വേണമെങ്കിൽ `//` (floor division) ഉപയോഗിക്കണം. ഇതുപോലുള്ള മാറ്റങ്ങൾ കാരണമാണ് Python 3 "backward-compatible അല്ല" എന്ന് പറയുന്നത്.

Exam tip: The unit says "migrating a project from Python 3 to Python 2.x requires many changes". The direction is reversed: the real problem was migrating **from Python 2 to Python 3**, which is why some Python 3 features were back-ported to 2.x.

Implementations: CPython, Jython, IronPython, PyPy

Implementation of Python: a program (interpreter/compiler plus runtime) that actually executes Python code; the language is the same, the engine differs.

Implementation of Python: Python code യഥാർത്ഥത്തിൽ execute ചെയ്യുന്ന program (interpreter/compiler-ഉം runtime-ഉം); language ഒന്നുതന്നെ, engine മാത്രം വ്യത്യസ്തം.

CPython compiles source to bytecode and interprets it, which is slower than native machine code; this is why Python is "said to be slow". Other implementations target other platforms or speed. Stackless Python (concurrency with tasklets and channels) is a fork of CPython. MicroPython (microcontrollers) is a separate, lean implementation of Python 3 written in C; the unit groups it with Stackless as "based on the CPython core", but it does not reuse CPython's runtime.

മലയാളത്തിൽ

ഒരേ പാചകക്കുറിപ്പ് (Python code) gas stove-ലും induction-ലും microwave-ലും പാകം ചെയ്യാം; അടുപ്പാണ് implementation. CPython ആണ് സാധാരണ അടുപ്പ്. Jython Java-യുടെ JVM-ൽ, IronPython .NET-ന്റെ CLR-ൽ ഓടുന്നു. PyPy-ക്ക് JIT compiler ഉള്ളതുകൊണ്ട് ഏറ്റവും വേഗം; Guido തന്നെ പറഞ്ഞു "code വേഗം ഓടണമെങ്കിൽ PyPy ഉപയോഗിക്കൂ".

Implementation	Written in / runs on	How it runs code	Key point
CPython	C	Bytecode on the CPython virtual machine	Reference implementation; best compatibility with packages and C extensions
Jython	Java / JVM	Compiles to Java bytecode run by the JVM	Can use Java class libraries; slower and less compatible with CPython libraries
IronPython	C# / .NET CLR	JIT-compiled on the Common Language Runtime	No Global Interpreter Lock, so better for multi-threaded / multi-core programs
PyPy	RPython (subset of Python)	JIT compiles to native machine code	Very fast; stackless micro-threads for massive concurrency

EXAMPLE: WHICH IMPLEMENTATION AM I RUNNING?

```
import sys, platform
print(platform.python_implementation())
print(sys.version_info.major, sys.version_info.minor)
```

```
CPython
3 14
```

`platform.python_implementation()` ഇപ്പോൾ ഓടുന്ന engine-ന്റെ പേര് തരുന്നു; python.org-ൽ നിന്ന് install ചെയ്താൽ അത് CPython ആയിരിക്കും. `sys.version_info` version number തരുന്നു (ഇവിടെ 3.14). PyPy-യിൽ ഇതേ code ഓടിച്ചാൽ "PyPy" എന്ന് കാണിക്കും.

Exam tip: Remember "Jython → Java, IronPython → .NET (C#), PyPy → Python (RPython)". The unit's line "programs written in Jython use Java classes, not Python modules" is too strong: Jython can import pure-Python modules; what it cannot use are CPython's C-extension modules.

Need for Python and its applications

Cross-platform application: an application that runs on any operating system (Windows, Linux, macOS, etc.) without being rewritten.

Cross-platform application: വീണ്ടും എഴുതാതെ തന്നെ Windows, Linux, macOS തുടങ്ങിയ ഏത് operating system-ലും ഓടുന്ന application.

Python is high-level, interpreted and dynamically typed, needs fewer lines than Java or C++, and lets you test changes instantly without a separate compile step. This makes rapid prototyping and debugging easy. It is general-purpose and cross-platform, which is why YouTube, BitTorrent and Dropbox use it.

> മലയാളത്തിൽ
>
> ഒരു Swiss knife പോലെയാണ് Python: web, AI, game, network security എന്നിങ്ങനെ എല്ലാ പണിക്കും ഒരു tool. ഒരേ program Windows-ലും Linux-ലും Mac-ലും ഓടും (cross-platform). code എഴുതിയ ഉടനെ run ചെയ്ത് നോക്കാം, compile ചെയ്യാൻ കാത്തിരിക്കേണ്ട.

Application area	Python tools named in the unit
AI and Machine Learning	NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch
Data Science and Visualization	Pandas, NumPy; Matplotlib, Seaborn for plots
Mobile applications	Kivy, BeeWare
Embedded and IoT	MicroPython; Raspberry Pi boards
Web development	Django, Flask, Pyramid (handle HTTPS, FTP, JSON, XML, Email)
Game development	Pygame, Pyglet (2D), PySoy (3D); scripting in Civilization IV, Toontown Online, Vega Strike
Enterprise and business	ERP, CRM, billing systems
Network security	Scapy (craft, sniff and dissect packets)

EXAMPLE: FEWER LINES THAN C

Reading a list of marks and printing the average needs loops, an array size and `printf` formats in C. In Python:

```
marks = [78, 92, 85, 60]
print("Average:", sum(marks) / len(marks))
```

```
Average: 78.75
```

രണ്ട് വരി മാത്രം: list ഉണ്ടാക്കി, `sum` കൊണ്ട് കൂട്ടി `len` കൊണ്ട് ഹരിക്കുന്നു. C-യിൽ ഇതിന് array declare ചെയ്യണം, for loop വേണം, type ശ്രദ്ധിക്കണം. ഇതാണ് "rapid prototyping" എന്ന് പറയുന്നത്.

Exam tip: For "list application areas" write at least five areas with one library/framework each. Small correction: a Raspberry Pi board runs full Linux with normal CPython; MicroPython is for microcontroller boards such as the Raspberry Pi Pico.

Framework vs Library vs Package vs Module

Module: a single .py file containing Python functions, classes and global variables.

Package: a directory of modules and sub-packages, marked by an `__init__.py` file. **Library:** a collection of packages. **Framework:** a collection of libraries that defines the program's architecture.

Module: Python functions, classes, global variables എന്നിവ അടങ്ങിയ ഒരു .py file.

Package: `__init__.py` file കൊണ്ട് അടയാളപ്പെടുത്തിയ, modules-ഉം sub-packages-ഉം ഉള്ള ഒരു directory. **Library:** packages-ന്റെ ഒരു collection. **Framework:** program-ന്റെ architecture നിശ്ചയിക്കുന്ന libraries-ന്റെ ഒരു collection.

So the size order is module ⊂ package ⊂ library ⊂ framework. Modules give reusability: write code once and import it in any project. Packages can be your own or downloaded from the Python Package Index (PyPI). The key difference between a library and a framework: **your code calls a library**, but **a framework calls your code** (Inversion of Control).

മലയാളത്തിൽ

ഒരു വീട് പണിയുന്നത് ആലോചിക്കൂ. Module ഒരു ഇഷ്ടിക, package ഇഷ്ടികകൾ അടുക്കിയ ഒരു ചുമര്, library പല ചുമരുകളും ജനലുകളും ഉള്ള ഒരു godown, framework ആണ് വീടിന്റെ plan തന്നെ. Library-യിൽ നിന്ന് നമുക്ക് വേണ്ടത് എടുത്ത് ഉപയോഗിക്കാം; framework-ൽ plan അവർ തീരുമാനിക്കും, നമ്മൾ നിർദ്ദേശിച്ച സ്ഥലത്ത് നമ്മുടെ code വെക്കണം.

Module = one .py file (e.g. math, os, datetime)
Package = folder of modules + __init__.py (e.g. urllib)
Library = collection of packages; your code calls it (e.g. NumPy, Pandas)
Framework = collection of libraries; it calls your code (e.g. Django, Flask)

EXAMPLE 1: CREATING AND IMPORTING YOUR OWN MODULE

File `mymath.py`:

```
PI = 3.14159

def area(r):
    return PI * r * r
```

File `main.py` in the same folder:

```
import mymath

print(mymath.PI)
print(mymath.area(2))
```

```
3.14159
12.56636
```

`mymath.py` ഒരു module ആണ്: അതിൽ ഒരു global variable (`PI`)-ഉം ഒരു function (`area`)-ഉം. `main.py` അതിനെ `import` ചെയ്ത് `mymath.` എന്ന prefix വെച്ച് ഉപയോഗിക്കുന്നു. ഇനി ഏത് project-ലും ഈ file copy ചെയ്ത് import ചെയ്താൽ മതി, വീണ്ടും എഴുതേണ്ട: അതാണ് reusability.

EXAMPLE 2: A PACKAGE (THE UNIT'S GAME.SOUND.LOAD)

Folder layout:

```
Game/
    __init__.py
    Sound/
        __init__.py
        load.py        (contains def play(name): print("Playing", name))
game_main.py
```

```
import Game.Sound.load as load_game

load_game.play("intro.wav")
```

```
Playing intro.wav
```

The unit writes the alias as `load game` (with a space). That is invalid:

```
import Game.Sound.load as load game
```

```
  File "bad.py", line 1
    import Game.Sound.load as load game
                              ^^^^
SyntaxError: cannot use name as import target
```

Game ഒരു package, അതിനകത്ത് Sound എന്ന sub-package, അതിനകത്ത് `load` എന്ന module. dot (`.`) കൊണ്ട് folder-ങ്ങളിലേക്ക് ഇറങ്ങുന്നു. as `load_game` എന്നത് ഒരു ചെറിയ വിളിപ്പേര് (alias) ആണ്. alias ഒരു identifier ആയതുകൊണ്ട് അതിൽ space പാടില്ല; underscore ഉപയോഗിക്കണം.

EXAMPLE 3: WILDCARD IMPORT, MODULE VS PACKAGE

```
from mymath import *
print(area(1))
```

```
3.14159
```

```
from Game.Sound import *
print("load" in dir())
```

```
False
```

After adding `__all__ = ["load"]` to `Game/Sound/__init__.py`, the same program prints:

```
True
```

Module-ൽ നിന്ന് * ഉപയോഗിച്ചാൽ അതിലെ എല്ലാ public names-ഉം കിട്ടും. Package-ൽ നിന്ന് * ഉപയോഗിച്ചാൽ submodules തനിയെ load ആകില്ല; `__init__.py`-ൽ ഉള്ള names മാത്രം കിട്ടും. `__all__` list-ൽ submodule-ന്റെ പേര് കൊടുത്താൽ അതും import ആകും.

Exam tip: Write IGNOU's three module-vs-package points (file vs directory; package needs `__init__.py`; * works for modules, not packages). But know the precise truth: `from package import *` is legal; it just does not pull in submodules unless `__init__.py` lists them in `__all__`. Also, since Python 3.3 a folder without `__init__.py` can still be imported as a "namespace package", though a regular package has one.

Popular Python libraries and frameworks

Python library / framework: pre-written, ready-to-use code for common tasks, installed from PyPI and brought into a program with `import`.

Python library / framework: സാധാരണ ജോലികൾക്കായി മുൻകൂട്ടി എഴുതിവെച്ച, ഉടനെ ഉപയോഗിക്കാവുന്ന code; PyPI-യിൽ നിന്ന് install ചെയ്ത് `import` വഴി program-ൽ കൊണ്ടുവരുന്നു.

These are the tools the unit lists. You will study NumPy, Pandas, Matplotlib and Kivy in detail later in the course.

മലയാളത്തിൽ

ഓരോ library-യും ഒരു specialist doctor പോലെയാണ്: കണക്കിന് NumPy, table data-യ്ക്ക് Pandas, graph വരയ്ക്കാൻ Matplotlib, mobile app-ന് Kivy. പ്രശ്നം അനുസരിച്ച് ശരിയായ specialist-നെ import ചെയ്താൽ മതി.

Name	Type	Used for
NumPy	Library	Numerical Python: n-dimensional arrays, matrices, linear algebra, Fourier transform, random numbers; base for SciPy, Scikit-learn, Matplotlib
Pandas	Library	Data analysis and manipulation; reads CSV, SQL, JSON, Excel
Matplotlib	Library	2D plots: histograms, scatter plots, non-Cartesian graphs; object-oriented API (like MATLAB)
Seaborn	Library	High-level statistical graphics on top of Matplotlib: heat maps, violin plots, time series
SciPy	Library	Scientific computing built on NumPy: linear algebra, integration, optimization, statistics
Scikit-learn	Library	Classical ML: classification, regression, clustering (SVM, random forest, k-means, DBSCAN)
TensorFlow	Library	End-to-end ML, deep neural networks, image recognition, NLP
Keras	Library (interface)	User-friendly API to build neural networks, CNNs and RNNs
Theano	Library	Define, optimize and evaluate expressions on multi-dimensional arrays (uses GPU)
Scrapy	Framework	Web crawling / spider bots to extract structured data from websites
Pygame	Library	Games and multimedia (graphics and sound)
Django	Framework	Rapid development of database-driven websites; reusable, pluggable components
Kivy	Framework	Cross-platform GUI / mobile apps (Windows, Linux, macOS, Raspberry Pi, Android, iOS)
BeeWare	Framework	Native-looking mobile and desktop apps in Python
MicroPython	Implementation	Python 3 for microcontrollers, with access to low-level hardware

EXAMPLE 1: NUMPY ARRAY ARITHMETIC

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b)
print(a * 2)
print(np.mean(b))
```

```
[11 22 33]
[2 4 6]
20.0
```

`np.array` ഒരു NumPy array ഉണ്ടാക്കുന്നു. `a + b` element-by-element കൂട്ടുന്നു (1+10, 2+20, 3+30); loop എഴുതേണ്ട. `a * 2` ഓരോ element-നെയും 2 കൊണ്ട് ഗുണിക്കുന്നു. `np.mean` ശരാശരി (average) തരുന്നു. സാധാരണ list-ൽ `[1,2,3] * 2` list-നെ ഇരട്ടിപ്പിക്കുകയാണ് ചെയ്യുക, ഗുണിക്കുകയല്ല.

EXAMPLE 2: A PANDAS TABLE

```
import pandas as pd

marks = pd.DataFrame({"name": ["Anu", "Binu", "Cini"],
                      "score": [78, 92, 85]})
print(marks)
print("Average:", marks["score"].mean())
```

```
   name  score
0   Anu     78
1  Binu     92
2  Cini     85
Average: 85.0
```

DataFrame ഒരു Excel sheet പോലെയുള്ള table ആണ്: name, `score` എന്ന രണ്ട് column, ഇടതുവശത്ത് 0, 1, 2 എന്ന row index. `marks["score"]` ഒരു column എടുക്കുന്നു, `.mean()` അതിന്റെ ശരാശരി കണ്ടുപിടിക്കുന്നു. NumPy-യും Pandas-ഉം built-in അല്ല; `pip install numpy pandas` ചെയ്യണം (Anaconda-യിലും Colab-ലും നേരത്തേ ഉണ്ട്).

Exam tip: CYP-2 Q2 asks the utility of NumPy, Pandas, Matplotlib and Kivy; one line each from the table is enough. The unit's Theano sentence "runs almost 100 times faster than on a CPU alone" is garbled: the claim is that Theano on a **GPU** can run up to about 100× faster than on a CPU. Theano itself is no longer developed (since 2017).

Getting started: installation and IDEs

IDE (Integrated Development Environment): software that combines an editor, interpreter/run tools and a debugger in one place for writing programs (e.g. Jupyter Notebook, Spyder, VS Code, PyCharm).

Anaconda: a free, open-source distribution of Python and R for scientific computing that bundles IDEs and data-science libraries.

IDE (Integrated Development Environment): program എഴുതാൻ editor, interpreter/run tools, debugger എന്നിവ ഒരിടത്ത് ഒരുമിച്ച് തരുന്ന software (ഉദാ: Jupyter Notebook, Spyder, VS Code, PyCharm).

Anaconda: IDEs-ഉം data-science libraries-ഉം ഒരുമിച്ച് തരുന്ന, scientific computing-നുള്ള Python-ന്റെയും R-ന്റെയും free, open-source distribution.

Four ways to start: (1) install Python from python.org (download the installer for your OS and click Next until finished); (2) install Anaconda and open Jupyter, Spyder or VS Code from the **Anaconda Navigator**; (3) use **Google Colab**, a cloud Jupyter notebook that needs only a browser and a Google account; (4) use **PyCharm** by JetBrains. You can work in the interactive shell (type a line, see the result immediately) or write a `.py` script file and run it.

മലയാളത്തിൽ

Python പഠിക്കാൻ വലിയ computer വേണ്ട. System-ൽ install ചെയ്യാൻ പറ്റുന്നില്ലെങ്കിൽ Google Colab തുറന്ന് Gmail കൊണ്ട് login ചെയ്താൽ മതി, എല്ലാം cloud-ൽ ഓടും. Interactive shell ഒരു calculator പോലെയാണ്: ഒരു വരി type ചെയ്യുക, ഉടനെ answer. Script file ഒരു recipe book പോലെ: എല്ലാ വരികളും save ചെയ്ത് ഒന്നിച്ച് run ചെയ്യുന്നു.

EXAMPLE: THE INTERACTIVE SHELL

Type `python3` (or `python` on Windows) in a terminal to get the >>> prompt:

```
>>> 2 + 3
5
>>> "Py" * 3
'PyPyPy'
>>> type(3.5)
<class 'float'>
>>> 10 / 4
2.5
>>> import math
>>> math.sqrt(16)
4.0
```

>>> എന്നത് Python "നിങ്ങളുടെ command കാത്തിരിക്കുന്നു" എന്ന് പറയുന്ന prompt ആണ്. ഓരോ expression-ന്റെയും value ഉടനെ താഴെ കാണിക്കുന്നു; ഇവിടെ `print` വേണ്ട. `"Py" * 3` string മൂന്ന് തവണ ആവർത്തിക്കുന്നു. `import math` ഒന്നും print ചെയ്യുന്നില്ല, പിന്നെ `math.sqrt` square root തരുന്നു. Shell-ൽ നിന്ന് പുറത്തുകടക്കാൻ `exit()`.

Exam tip: The unit lists "R Programming" among Python IDEs; R is a separate language (Anaconda also ships R), not an IDE. The installer versions in the unit (Python 3.7.7, Anaconda 2020.02, Python 2.7 option) are old; today download the latest Python 3 and never Python 2. Jupyter runs code cell by cell.

Program structure in Python

Top-level (high-level) file: the file you run directly; it controls the program and imports modules. **Module file:** a file of statements that is normally not run directly but imported by another file.

Top-level (high-level) file: നമ്മൾ നേരിട്ട് run ചെയ്യുന്ന, program-നെ നിയന്ത്രിക്കുന്ന, modules import ചെയ്യുന്ന file. **Module file:** സാധാരണ നേരിട്ട് run ചെയ്യാതെ മറ്റൊരു file import ചെയ്യുന്ന statements-ന്റെ file.

A Python program is usually split across several files. In the unit's Figure 5, `a.py` is the top-level file where the application starts; `b.py` and `c.py` are modules that `a.py` imports to use the tools (functions, variables) they define. Modules may in turn import other modules. Inside the files you find the usual building blocks: data types, conditionals, loops, functions, file handling, classes, exception handling, libraries, modules and packages (covered in later units).

മലയാളത്തിൽ

ഒരു ഹോട്ടൽ അടുക്കള ആലോചിക്കൂ: head chef (`a.py`) ആണ് പണി തുടങ്ങുന്നതും നിയന്ത്രിക്കുന്നതും. Assistant cooks (`b.py`, `c.py`) സ്വയം ഒന്നും തുടങ്ങില്ല, chef വിളിക്കുമ്പോൾ അവരുടെ special dish ഉണ്ടാക്കിക്കൊടുക്കും. Python ഒരു file-നെ നേരിട്ട് run ചെയ്താൽ അതിന്റെ `__name__` `"__main__"` ആയിരിക്കും; import ചെയ്താൽ file-ന്റെ പേരായിരിക്കും.

```
        a.py  (top-level file: run this)
        /    \
   import b   import c
      |          |
    b.py        c.py   (modules)
```

EXAMPLE: A.PY USES MODULES B.PY AND C.PY

`b.py`:

```
def greet(name):
    return "Hello, " + name
```

`c.py`:

```
def square(n):
    return n * n

print("c.py: __name__ is", __name__)
```

`a.py`:

```
import b
import c

print("a.py: __name__ is", __name__)
print(b.greet("Anoop"))
print(c.square(12))
```

Running `python3 a.py`:

```
c.py: __name__ is c
a.py: __name__ is __main__
Hello, Anoop
144
```

Running `python3 c.py` directly:

```
c.py: __name__ is __main__
```

`a.py` run ചെയ്യുമ്പോൾ ആദ്യം `import c` നടക്കുന്നു, അപ്പോൾ `c.py`-യിലെ print ഓടുന്നു, അതിന്റെ `__name__` `c` ആണ്. പിന്നെ `a.py`-യുടെ സ്വന്തം വരികൾ; അതിന്റെ `__name__` `__main__`, കാരണം അതാണ് top-level file. `b.greet`, `c.square` എന്നിങ്ങനെ module-ന്റെ പേര് prefix ആക്കി tools ഉപയോഗിക്കുന്നു. `c.py` നേരിട്ട് ഓടിച്ചാൽ അത് തന്നെ top-level ആകുന്നു.

Running the first program

print(): built-in function that displays values on the screen. **input():** built-in function that reads a line typed by the user and returns it as a string.

print(): values screen-ൽ കാണിക്കുന്ന built-in function. **input():** user type ചെയ്യുന്ന ഒരു വരി വായിച്ച് അത് string ആയി തിരികെ തരുന്ന built-in function.

In Google Colab choose File → New notebook, type code in a cell and press the play button (or Shift+Enter); the output appears just below the cell. In a terminal, save the code as `hello.py` and run `python3 hello.py`.

മലയാളത്തിൽ

ആദ്യത്തെ program ഒരു സ്വാഗതം print ചെയ്യലാണ്. Colab-ൽ cell-ൽ എഴുതി play button അമർത്തിയാൽ output ഉടനെ താഴെ വരും. ശ്രദ്ധിക്കേണ്ട ഒരു കാര്യം: `input()` എപ്പോഴും string ആണ് തരുന്നത്, സംഖ്യ അല്ല. കൂട്ടണമെങ്കിൽ `int()` അല്ലെങ്കിൽ `float()` കൊണ്ട് number ആക്കണം.

EXAMPLE 1: FIRST PYTHON PROGRAM

```
print("Welcome to Python Programming!")
```

```
Welcome to Python Programming!
```

`print` brackets-നുള്ളിലെ string (double quotes-ൽ) അതേപടി screen-ൽ കാണിക്കുന്നു. Quotes output-ൽ വരുന്നില്ല. Word-ൽ നിന്ന് copy ചെയ്ത "curly quotes" ഉപയോഗിച്ചാൽ SyntaxError വരും; keyboard-ലെ സാധാരണ " തന്നെ വേണം.

EXAMPLE 2: ADD TWO NUMBERS (FIXED VALUES)

```
a = 5
b = 7
total = a + b
print("Sum =", total)
```

```
Sum = 12
```

a-യിൽ 5, b-യിൽ 7 വെക്കുന്നു, അവ കൂട്ടി `total`-ൽ വെക്കുന്നു. `print`-ൽ comma കൊണ്ട് വേർതിരിച്ച values ഇടയ്ക്ക് ഒരു space വെച്ച് കാണിക്കും.

EXAMPLE 3: CYP-2 Q3, TWO NUMBERS FROM THE USER (CORRECT VERSION)

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("Sum of", a, "and", b, "is", a + b)
```

```
Enter first number: 12
Enter second number: 30
Sum of 12 and 30 is 42
```

The common wrong version forgets the conversion:

```
a = input("Enter first number: ")
b = input("Enter second number: ")
print("Sum =", a + b)
```

```
Enter first number: 12
Enter second number: 30
Sum = 1230
```

For decimal numbers use `float`:

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
print(f"{a} + {b} = {a + b}")
```

```
Enter first number: 2.5
Enter second number: 4
2.5 + 4.0 = 6.5
```

`input()` prompt കാണിച്ച് user type ചെയ്യുന്നത് string ആയി തരുന്നു. `int()` ആ string-നെ integer ആക്കുന്നു, അതുകൊണ്ട് 12 + 30 = 42. Conversion മറന്നാൽ "12" + "30" എന്ന രണ്ട് string ഒട്ടിച്ച് "1230" കിട്ടും, കൂട്ടലല്ല. മൂന്നാമത്തേതിൽ `float()` decimal സംഖ്യ സ്വീകരിക്കുന്നു; `f"..."` (f-string) curly brackets-നുള്ളിലെ values നേരിട്ട് text-ൽ ചേർക്കുന്നു.

EXAMPLE 4: RUN-TIME ERROR VS SYNTAX ERROR

```
print("Line 1 runs")
print("Line 2 runs")
print(10 / 0)
print("Line 4 never runs")
```

```
Line 1 runs
Line 2 runs
Traceback (most recent call last):
  File "runtime_err.py", line 3, in <module>
    print(10 / 0)
          ~~~^~~
ZeroDivisionError: division by zero
```

```
print("Line 1")
print("Line 2"
print("Line 3")
```

```
  File "syntax_err.py", line 2
    print("Line 2"
         ^
SyntaxError: '(' was never closed
```

ആദ്യ program-ൽ 1, 2 വരികൾ ഓടി, 3-ാം വരിയിൽ പൂജ്യം കൊണ്ട് ഹരിച്ചപ്പോൾ run-time error, അവിടെ നിന്നു; 4-ാം വരി ഓടിയില്ല. രണ്ടാമത്തേതിൽ 2-ാം വരിയിൽ bracket അടച്ചിട്ടില്ല; Python ആദ്യം മുഴുവൻ file-ഉം bytecode ആക്കാൻ നോക്കുന്നതുകൊണ്ട് "Line 1" പോലും print ആകുന്നില്ല. Syntax error ആദ്യം പിടിക്കും, run-time error ഓടുമ്പോൾ.

Exam tip: The answer to CYP-2 Q3 is blank in the unit. Write Example 3 and mention that `input()` returns a string, so `int()`/`float()` is needed; without it + joins the strings. The unit's printed `print(“Welcome …”)` uses curly quotes from the word processor; type straight quotes.

Quick recap

- Python: high-level, general-purpose, interpreted, dynamically typed, garbage-collected, multi-paradigm; "batteries included".
- Python vs C: interpreted vs compiled, slower vs faster, no declarations vs declared types, automatic GC vs manual memory, no pointers vs pointers.

- Created by Guido van Rossum (1989) as successor to ABC; first release 1991; managed by the Python Software Foundation.
- Python 2.0 (2000), Python 3.0 (2008, not backward-compatible), Python 2.7 last 2.x, ended 2020.
- CPython (C, bytecode VM, reference), Jython (JVM), IronPython (.NET CLR, no GIL), PyPy (RPython, JIT, fastest).
- Stackless Python: concurrency with tasklets; MicroPython: Python 3 for microcontrollers.
- Cross-platform: runs on any OS without rewriting; used by YouTube, Dropbox, BitTorrent.
- Applications: AI/ML, data science, mobile (Kivy, BeeWare), IoT, web (Django, Flask), games (Pygame), business, network security (Scapy).
- Module = .py file; package = directory + `__init__.py`; library = packages; framework = libraries defining the architecture.
- Your code calls a library; a framework calls your code (Inversion of Control).
- NumPy arrays and maths; Pandas tables; Matplotlib and Seaborn plots; Scikit-learn, TensorFlow, Keras for ML.
- Ways to start: python.org installer, Anaconda Navigator (Jupyter, Spyder, VS Code), Google Colab, PyCharm.
- Program structure: one top-level file (`__name__ == "__main__"`) imports module files.
- `input()` returns a string: convert with `int()` or `float()` before adding.

Check yourself

Q. 1. Compare Python and C programming languages.

(i) Python is a high-level interpreted language focused on readability and productivity; C is a lower-level compiled language built for speed and hardware control. (ii) Python is dynamically typed with automatic garbage collection and a huge standard library; C needs static type declarations and manual memory management. (iii) Python stops at the faulty statement at run time; C compiles the whole source first and lists all errors. (iv) Python supports using one function under two names and has built-in data structures; C uses pointers and needs explicit data-structure code. (v) Python suits AI, data science and rapid prototyping; C suits operating systems, firmware and performance-critical software.

Q. 2. Discuss the variants CPython, Jython, IronPython and PyPy.

CPython: reference implementation written in C; compiles to bytecode run on its VM; best compatibility; general use, data science, ML. **Jython:** Java implementation; compiles Python to Java bytecode for the JVM; can use Java classes; good for Java enterprise integration. **IronPython:** C#/.NET implementation on the CLR; JIT and no GIL, so good for multi-threaded programs and .NET interop. **PyPy:** written in RPython; JIT compiles to native code, so much faster for CPU-bound tasks.

Q. 3. List some application areas of Python with one tool each.

AI/ML (TensorFlow, Scikit-learn); data science and visualization (Pandas, Matplotlib); web development (Django, Flask); mobile apps (Kivy, BeeWare); embedded/IoT (MicroPython); games (Pygame); network security (Scapy); automation and web scraping (Scrapy, BeautifulSoup, Selenium); business apps (ERP, CRM, billing).

Q. 4. Compare framework, library, package and module in Python.

Module: a single .py file with functions, classes and variables (e.g. `math`, `os`). **Package:** a directory of related modules with an `__init__.py` file (e.g. `urllib`). **Library:** a collection of packages and modules that your code calls (e.g. NumPy, Pandas). **Framework:** a collection of libraries giving a complete application skeleton that calls your code, Inversion of Control (e.g. Django, Flask).

Q. 5. Discuss the utility of NumPy, Pandas, Matplotlib and Kivy.

NumPy: fast arrays and matrices, linear algebra, Fourier transform, random numbers; the maths engine under most data-science tools. **Pandas:** tabular data (DataFrame) analysis and cleaning; reads CSV, Excel, SQL, JSON. **Matplotlib:** 2D charts such as line, bar, histogram and scatter plots, with full control of every detail. **Kivy:** open-source framework for touch-friendly GUI apps that run on Windows, Linux, macOS, Android and iOS.

Q. 6. Write a Python program that takes two numbers as input and displays their sum.

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("Sum of", a, "and", b, "is", a + b)
```

```
Enter first number: 12
Enter second number: 30
Sum of 12 and 30 is 42
```

`int()` is essential because `input()` returns a string; without it the output would be `1230`. Use `float()` to allow decimals.

Q. 7. What is the output of: `x = "5"; y = 3; print(x * y)`? And of `print(int(x) * y)`?

`x` is a string, so `x * y` repeats it 3 times: 555. `int(x) * y` is 5 × 3 = 15.

```
555
15
```

Q. 8. Why does Python 2 code often fail on Python 3? Give one example.

Python 3.0 (2008) was a major revision that is not backward-compatible. Example: Python 2's `print "hi"` is a statement; in Python 3 `print` is a function, so it must be `print("hi")`, otherwise `SyntaxError: Missing parentheses in call to 'print'`. Also `7 / 2` gives 3 in Python 2 but 3.5 in Python 3.

Q. 9. Explain the structure of a Python program with top-level and module files.

A program is split into files. The top-level file (e.g. `a.py`) is run directly and controls the program; its `__name__` is `"__main__"`. It imports module files (`b.py`, `c.py`), which are not normally run directly, and uses their tools as `b.greet()`, `c.square()`. Modules may import other modules. Inside, programs use data types, conditionals, loops, functions, classes, exception handling and so on.

Q. 10. Name four ways to start writing Python programs.

(1) Install Python from python.org and use the interactive shell (>>>) or run `.py` scripts; (2) install Anaconda and launch Jupyter Notebook, Spyder or VS Code from Anaconda Navigator; (3) use Google Colab in a browser with a Google account, no installation needed; (4) use the PyCharm IDE from JetBrains.

UNIT 2

Data Structures in Python

ഈ unit–ൽ Python–ന്റെ അടിസ്ഥാന ഘടകങ്ങളായ identifiers, keywords, literals, variables, operators, data types എന്നിവ പഠിക്കുന്നു. തുടർന്ന് Python–ലെ നാല് built–in data structures ആയ List, Tuple, Dictionary, Set (ഒപ്പം frozenset) എന്നിവ ഓരോ operation–ഉം method–ഉം run ചെയ്ത output സഹിതം വിശദമായി കാണുന്നു.

Identifiers and keywords · Literals · Statements and expressions · Variables and comments · Arithmetic and assignment operators · Relational and logical operators · Bitwise operators and operator precedence · Data types: numeric, Boolean and None · Strings · Data structures at a glance · List: creation, indexing and slicing · List operations and the list() function · Modifying and traversing lists · List methods · Built-in functions used on lists · Nested lists · Tuple: creation and basic operations · The tuple() function, built-in functions and tuple methods · Tuple assignment and tuples vs lists · Dictionary: creation and access · Built-in functions used on dictionaries · Dictionary methods · Populating, traversing and deleting in dictionaries · Relation between tuples and dictionaries · Set: creation and set operations · Set methods and traversing sets · Frozenset

---

Identifiers and keywords

> **Identifier:** a name given to a variable, function, class or module. **Keyword:** a reserved word with a predefined meaning to the Python interpreter; it cannot be used as an identifier.
>
> **Identifier:** ഒരു variable, function, class അല്ലെങ്കിൽ module–ന് നൽകുന്ന പേര്. **Keyword:** Python interpreter–ന് മുൻകൂട്ടി നിശ്ചയിച്ച അർത്ഥമുള്ള reserved word; അത് identifier ആയി ഉപയോഗിക്കാൻ പാടില്ല.

An identifier may contain letters (a–z, A–Z), digits (0–9) and the underscore (_). It must begin with a letter or an underscore, never with a digit. Spaces and special symbols such as !, @, #, $, % are not allowed, keywords cannot be used, and there is no length limit (but short, meaningful names are preferred). Python is case-sensitive, so `Marks` and `marks` are different names, and keywords must be written exactly as listed.

മലയാളത്തിൽ

Identifier എന്നാൽ നമ്മൾ ഒരു variable-നോ function-നോ ഇടുന്ന പേര്, വീട്ടിലെ ഓരോ കുട്ടിക്കും പേരിടുന്നതു പോലെ. പേര് ഒരു letter കൊണ്ടോ underscore കൊണ്ടോ തുടങ്ങണം, digit കൊണ്ട് തുടങ്ങാൻ പാടില്ല; space-ഉം @, $ പോലുള്ള symbols-ഉം പാടില്ല. `for`, `if`, `True` പോലുള്ള keywords Python സ്വന്തം ആവശ്യത്തിന് "ബുക്ക് ചെയ്ത" വാക്കുകളാണ്, അവ നമുക്ക് പേരായി എടുക്കാൻ പറ്റില്ല. Python case-sensitive ആയതിനാൽ Age-ഉം age-ഉം രണ്ട് വ്യത്യസ്ത പേരുകളാണ്.

Valid: myIndia, Marks_1, mca_course, _temp, plus1, Subject1_Marks
Invalid: 1plus (starts with digit), my name (space), rate$ (symbol), for (keyword)

EXAMPLE 1: TEST NAMES WITH ISIDENTIFIER()

```
import keyword
names = ["myIndia", "Marks_1", "_temp", "plus1", "1plus", "my name", "rate$", "for"]
for n in names:
    ok = n.isidentifier() and not keyword.iskeyword(n)
    print(n, "->", "valid" if ok else "invalid")
```

```
myIndia -> valid
Marks_1 -> valid
_temp -> valid
plus1 -> valid
1plus -> invalid
my name -> invalid
rate$ -> invalid
for -> invalid
```

`isidentifier()` checks the letter/digit/underscore rules; `keyword.iskeyword()` rejects reserved words like `for`.

EXAMPLE 2: USING AN INVALID IDENTIFIER

```
1plus = 5
```

```
SyntaxError: invalid decimal literal
```

EXAMPLE 3: LIST ALL KEYWORDS OF YOUR PYTHON VERSION

```
import keyword
print(len(keyword.kwlist))
print(keyword.kwlist)
```

```
35
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',
'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or',
'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

Exam tip: IGNOU's Table 1 lists only 33 keywords; Python 3 has 35 because `async` and `await` are also keywords (since Python 3.7). Only `True`, `False` and `None` start with a capital letter. "Maximum length of an identifier" → any length.

Literals

Literal: a raw, fixed data value written directly in the code, such as 10, 3.14, 'Hello', True or None.
Docstring: a triple-quoted string written as the first statement of a function, class or module, stored in its `__doc__` attribute.

Literal: 10, 3.14, 'Hello', True, None പോലെ code-ൽ നേരിട്ട് എഴുതുന്ന സ്ഥിരമായ (fixed) data value. **Docstring:** ഒരു function, class അല്ലെങ്കിൽ module-ന്റെ ആദ്യ statement ആയി എഴുതുന്ന triple-quoted string; അത് `__doc__` attribute-ൽ സൂക്ഷിക്കപ്പെടുന്നു.

Python has four kinds of literals. **String literals** use single quotes, double quotes or triple quotes (triple quotes allow multi-line text and keep newlines and indentation as typed). **Numeric literals** are immutable and come in three types: int (including hexadecimal 0x, octal 0o and binary 0b forms), float (decimal point or scientific notation like 1.2e5) and complex (a + bj). **Boolean literals** are True and False. The **special literal** None means "no value".

മലയാളത്തിൽ

Literal എന്നാൽ program-ൽ നേരിട്ട് എഴുതി വെക്കുന്ന value, കണക്കുകൂട്ടി കിട്ടുന്നതല്ല. കടയിലെ വിലവിവരപ്പട്ടികയിൽ എഴുതിയ "₹40" പോലെ, അത് എപ്പോഴും അതേ value തന്നെ. String-ന് ' ', " ", """ """ മൂന്ന് തരം quotes ഉപയോഗിക്കാം; triple quotes കൊണ്ട് പല വരികളുള്ള text എഴുതാം. Number-കൾ int, float, complex എന്നിങ്ങനെ മൂന്ന് തരം. None എന്നാൽ "ഒന്നുമില്ല" എന്നാണ്, അത് 0-ഓ empty string-ഓ അല്ല.

String: 'Hello', "Python", """multi-line"""
int: 10, -5, 0xAF (hex), 0o17 (octal), 0b10101 (binary)
float: 10.5, -3.14, 1.2e5 (= 1.2×10^5)
complex: 3 + 4j, complex(3, 4)
Boolean: True, False   Special: None

EXAMPLE 1: STRING AND NUMERIC LITERALS

```
print('Hello', "Python")
s = """Line one
Line two"""
print(s)
print(0xAF, 0o17, 0b10101, 1.2e5)
Z = 5 + 4j
print("printing Z:")
print(Z)
Y = complex(3, 4)
print("printing Y:")
print(Y, Y.real, Y.imag)
```

```
Hello Python
Line one
Line two
175 15 21 120000.0
printing Z:
(5+4j)
printing Y:
(3+4j) 3.0 4.0
```

Hex, octal and binary literals are all stored as ordinary int values: 0xAF = 175, 0o17 = 15, 0b10101 = 21.

EXAMPLE 2: DOCSTRING (IGNOU'S EXAMPLE)

```
def calculate_area(width, height):
    """ Calculate the area of a rectangle.
    Arguments:
    width -- the width of the rectangle
    height -- the height of the rectangle
    """
    return width * height

print(calculate_area.__doc__)
```

```
Calculate the area of a rectangle.
Arguments:
width -- the width of the rectangle
height -- the height of the rectangle
```

EXAMPLE 3: PROPERTIES OF NONE

```
x = None
print(x, type(x))
print(x == 0, x == False, x == "")
print(bool(x))
a = None
print(a is x)          # singleton: only one None object exists

def greet():
    pass               # no return statement
print(greet())
```

```
None <class 'NoneType'>
False False False
False
True
None
```

None is falsy (`bool(None)` is False) but it is not equal to 0, False or "". A function without `return` returns None.

Exam tip: IGNOU's printed output for the docstring example drops the first line "Calculate the area of a rectangle." — `__doc__` contains the whole string, so the first line is printed too (as shown above). Python 3.13+ also strips the common indentation from docstrings, so older versions print the lines indented.

Statements and expressions

Statement: a unit of code that the Python interpreter can execute. **Expression:** a combination of values, variables and operators that evaluates to a single value.

Statement: Python interpreter-ന് execute ചെയ്യാൻ കഴിയുന്ന code-ന്റെ ഒരു unit.

Expression: values, variables, operators എന്നിവ ചേർന്ന് ഒരു single value ആയി evaluate ചെയ്യുന്ന combination.

Statements are mainly of two kinds: expression statements and assignment statements. An assignment statement such as `x = 2` produces no output. A single value (25), a single variable (x) or a combination (`x + 25`) are all expressions. In interactive mode (the >>> prompt) an expression's value is displayed at once; in a script nothing is shown unless you `print()` it.

മലയാളത്തിൽ

Statement എന്നത് Python-ന് കൊടുക്കുന്ന ഒരു "ആജ്ഞ" ആണ്; expression എന്നത് ഒരു value ആയി മാറുന്ന കണക്ക്. `total = number_of_item * rate` എന്നതിൽ വലതുവശത്തെ `number_of_item * rate` ആണ് expression, മുഴുവൻ വരി ഒരു assignment statement. അതായത് expression പലപ്പോഴും ഒരു statement-നുള്ളിൽ ഇരിക്കും. Script-ൽ ഒരു expression തനിയെ എഴുതിയാൽ ഒന്നും കാണില്ല, print() ചെയ്യണം.

EXAMPLE 1: A SCRIPT WITH SEVERAL STATEMENTS

```
print(1)
x = 2
print(x)
```

```
1
2
```

EXAMPLE 2: AN EXPRESSION INSIDE A STATEMENT

```
X = 7 + 3         # assignment statement: no output
7 + 3             # bare expression in a script: no output either
print(X)          # print() shows the value
number_of_item, rate = 4, 25
total = number_of_item * rate
print(total)
```

```
10
100
```

Exam tip: the unit shows X=7+3 followed by "Output: 10". That is not correct: an assignment statement produces no output, even at the >>> prompt. You see 10 only after typing `X` (interactive mode) or `print(X)`.

Variables and comments

Variable: a named placeholder (a reference to an object in memory) that holds data the program can use, assign and modify during execution. **Comment:** text after # that the interpreter ignores.

Variable: program execution സമയത്ത് ഉപയോഗിക്കാനും assign ചെയ്യാനും മാറ്റാനും കഴിയുന്ന data സൂക്ഷിക്കുന്ന പേരുള്ള placeholder (memory-യിലെ ഒരു object-ലേക്കുള്ള reference). **Comment:** # കഴിഞ്ഞ് എഴുതുന്ന, interpreter ignore ചെയ്യുന്ന text.

Python is **dynamically typed**: you do not declare a type, you simply assign with `variable_name = expression`. The type belongs to the object, not to the name, so the same name can later refer to an object of a different type. Names are case-sensitive (`IGNOU` and `Ignou` differ). One value can be assigned to several names at once (`x = y = z = 5`). A variable must be assigned before it is used, otherwise Python raises `NameError`.

മലയാളത്തിൽ

Python-ൽ variable ഒരു "പെട്ടി" അല്ല, ഒരു object-ൽ ഒട്ടിക്കുന്ന "പേര് ടാഗ്" ആണ്. കല്യാണത്തിന് കസേരയിൽ ഒട്ടിക്കുന്ന name slip പോലെ, അതേ slip നാളെ മറ്റൊരു കസേരയിൽ ഒട്ടിക്കാം. അതുകൊണ്ടാണ് `year`-ന് ആദ്യം int-ഉം പിന്നെ string-ഉം കൊടുക്കാൻ പറ്റുന്നത്; type object-ന്റേതാണ്, പേരിന്റേതല്ല. Variable declare ചെയ്യേണ്ട ആവശ്യമില്ല, പക്ഷേ ഉപയോഗിക്കുന്നതിന് മുമ്പ് value assign ചെയ്തിരിക്കണം. # കഴിഞ്ഞുള്ളതെല്ലാം comment ആണ്, Python അത് വായിക്കില്ല.

variable_name = expression

x = y = z = 5   (same value to many names)

single-line comment   """ unassigned triple-quoted string used as a block comment """

EXAMPLE 1: DYNAMIC TYPING AND MULTIPLE ASSIGNMENT

```
number = 100            # int assigned
name = "Python"         # str assigned
year = 2026
print(year, type(year))
year = "twenty twenty six"   # same name, new type
print(year, type(year))
x = y = z = 5
print(x, y, z)
IGNOU, Ignou = 1, 2     # case-sensitive: two different variables
print(IGNOU, Ignou)
```

```
2026 <class 'int'>
twenty twenty six <class 'str'>
5 5 5
1 2
```

EXAMPLE 2: USING A VARIABLE BEFORE ASSIGNING IT

```
a = 25
b = 35
a = b
print(a)
apple = mango           # mango was never assigned
```

```
35
NameError: name 'mango' is not defined. Did you mean: 'range'?
```

`a = b` makes `a` refer to 35. The last line fails with NameError because `mango` does not exist.

Exam tip: "Do we need to declare variables in Python?" → No; Python is dynamically typed, but a variable must be assigned a value before it is read. Python has no true multi-line comment; use # on every line (PEP 8 style) or an unassigned triple-quoted string.

Arithmetic and assignment operators

Operator: a special symbol that represents a computation; the values it acts on are called **operands**. **Assignment operator:** = (and its compound forms like +=) stores the value of the right-hand expression in the left-hand variable.

Operator: ഒരു computation സൂചിപ്പിക്കുന്ന പ്രത്യേക symbol; അത് പ്രവർത്തിക്കുന്ന values-നെ **operands** എന്ന് പറയുന്നു. **Assignment operator:** = (ഒപ്പം += പോലുള്ള compound രൂപങ്ങൾ) വലതുവശത്തെ expression-ന്റെ value ഇടതുവശത്തെ variable-ൽ സൂക്ഷിക്കുന്നു.

Python supports arithmetic, assignment, relational, logical and bitwise operators. The arithmetic operators are +, -, *, / (true division, always gives a float), % (remainder), ** (power) and // (floor division, rounds down towards negative infinity). Each compound assignment operator `m op= n` is shorthand for `m = m op n`.

> **മലയാളത്തിൽ**
> / എപ്പോഴും float ഉത്തരം തരും: 16 / 2 = 8.0. // "floor division" ആണ്, ദശാംശം വെട്ടിക്കളഞ്ഞ് താഴോട്ടുള്ള whole number തരും: 10 // 3 = 3. % ബാക്കി (remainder) തരും. 100 രൂപയ്ക്ക് 30 രൂപയുടെ സാധനം എത്ര വാങ്ങാം എന്നത് 100 // 30 = 3, ബാക്കി പൈസ 100 % 30 = 10. `m += 5` എന്നത് `m = m + 5` എന്നതിന്റെ ചുരുക്കെഴുത്ത് മാത്രം.

Operator	Name	n1 = 5 , n2 = 6	Result
+	Addition	n1 + n2	11
-	Subtraction	n1 - n2	-1
*	Multiplication	n1 * n2	30
/	Division	n1 / n2	0.8333333333333334
%	Modulus (remainder)	n1 % n2	5
**	Exponent	n1 ** n2	15625
//	Floor division	n1 // n2	0

EXAMPLE 1: ARITHMETIC OPERATORS (TABLE 3)

```
n1, n2 = 5, 6
print(n1 + n2, n1 - n2, n1 * n2)
print(n1 / n2)
print(n1 % n2, n1 ** n2, n1 // n2)
```

```
11 -1 30
0.8333333333333334
5 15625 0
```

EXAMPLE 2: / VERSUS //

```
print(10 / 3, 10 // 3)
print(16 / 2, 10 // 2)
print(10.0 // 3)      # a float operand gives a float result
print(-7 // 2)        # rounds DOWN, towards negative infinity
```

```
3.3333333333333335 3
8.0 5
3.0
-4
```

EXAMPLE 3: ASSIGNMENT OPERATORS APPLIED ONE AFTER ANOTHER (TABLE 4)

```
n1, n2 = 5, 6
m = n1 + n2;  print("m = n1 + n2 :", m)
m += n1;      print("m += n1     :", m)
m -= n1;      print("m -= n1     :", m)
m *= n1;      print("m *= n1     :", m)
m /= n1;      print("m /= n1     :", m)
m **= n1;     print("m **= n1    :", m)
m //= n1;     print("m //= n1    :", m)
m %= n1;      print("m %= n1     :", m)
```

```
m = n1 + n2 : 11
m += n1     : 16
m -= n1     : 11
m *= n1     : 55
m /= n1     : 11.0
m **= n1    : 161051.0
m //= n1    : 32210.0
m %= n1     : 0.0
```

Once /= is used, m becomes a float (11.0), so every later result is a float too.

Exam tip: IGNOU's answer to "difference between / and //" says `print(16 / 2) # Output: 8.0 (Notice it is 8.0, not 5)`; it should read "8.0, not 8" — / always returns a float. Also Table 4 writes `m *= n1 ~ m = m* n`; it means `m = m * n1`.

Relational and logical operators

Relational (comparison) operators (==, !=, >, <, >=, <=) compare two operands and return True or False. **Logical operators** (and, or, not) combine or reverse Boolean conditions.

Relational (comparison) operators (==, !=, >, <, >=, <=) രണ്ട് operands താരതമ്യം ചെയ്ത് True അല്ലെങ്കിൽ False തിരികെ നൽകുന്നു. **Logical operators** (and, or, not) Boolean conditions കൂട്ടിച്ചേർക്കുകയോ തിരിച്ചിടുകയോ ചെയ്യുന്നു.

Strings are compared character by character using their Unicode values; the first differing character decides. `and` is True only when both operands are True, `or` is True when at least one is True, and `not` reverses the value. With non-Boolean operands, `and`/`or` return one of the operands: `a and b` returns `a` if it is falsy, else `b`; `a or b` returns `a` if it is truthy, else `b`.

മലയാളത്തിൽ

Comparison operators ഒരു ചോദ്യം ചോദിക്കുന്നു, ഉത്തരം True അല്ലെങ്കിൽ False. String-കൾ dictionary-യിലെ പോലെ ഓരോ character-ന്റെയും Unicode value നോക്കി താരതമ്യം ചെയ്യും: 'H' (72) 'W' (87)-നേക്കാൾ ചെറുതായതിനാൽ "Hello" < "World". and എന്നാൽ "രണ്ടും ശരിയാകണം", `or` എന്നാൽ "ഒന്നെങ്കിലും ശരിയാകണം", `not` ഉത്തരം തിരിച്ചിടും. `100 and 200` എന്നതിന് True അല്ല, 200 ആണ് ഉത്തരം എന്ന് ഓർക്കുക.

n1	n2	n1 and n2	n1 or n2	not n1
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

EXAMPLE 1: RELATIONAL OPERATORS (TABLE 5)

```
n1, n2 = 10, 0
s1, s2 = "Hello", "World"
print(n1 == n2, s1 == s2)
print(n1 != n2, s1 != s2)
print(n1 > n2, s1 > s2)
print(n1 < n2, n1 >= n2, n1 <= n2)
print(ord('H'), ord('W'))
```

```
False False
True True
True False
False True False
72 87
```

EXAMPLE 2: LOGICAL OPERATORS (TABLE 6)

```
n1, n2 = 10, -20
print(n1 == 10 and n2 == -20)
print(n1 >= 10 or n2 < -20)
print(not(n1 == 20))
a, b = 100, 200
print(a and b, a or b, 0 and b, 0 or b)
```

```
True
True
True
200 100 0 200
```

> **Exam tip:** IGNOU's Table 5 is wrong for two string rows. With s1 = "Hello", s2 = "World": `s1 != s2` is **True** (the table says False) and `s1 > s2` is **False** (the table says True), because 'H' (72) < 'W' (87). Also, `print(100 and 200)` prints 200, not True.

Bitwise operators and operator precedence

Bitwise operators (&, |, ^, ~, <<, >>) work on the binary bits of integers. **Operator precedence** is the order in which Python evaluates operators in an expression that has more than one operator.

Bitwise operators (&, |, ^, ~, <<, >>) integers–ന്റെ binary bits–ൽ പ്രവർത്തിക്കുന്നു.

Operator precedence എന്നത് ഒന്നിലധികം operators ഉള്ള expression–ൽ Python അവ evaluate ചെയ്യുന്ന ക്രമമാണ്.

& gives 1 where both bits are 1; | gives 1 where either bit is 1; ^ gives 1 where exactly one bit is 1; ~n equals −(n + 1) in two's complement; `n << k` multiplies by 2^k and `n >> k` floor-divides by 2^k. When operators are mixed, higher-precedence ones are evaluated first; parentheses override everything. ** is right-associative, so `2 ** 3 ** 2` = 2 ** 9.

മലയാളത്തിൽ

Bitwise operators number–നെ 0–ഉം 1–ഉം ആയി പൊളിച്ച് ഓരോ bit–ഉം നോക്കുന്നു. 60 = 0011 1100, 13 = 0000 1101; ഇവയുടെ & എടുത്താൽ രണ്ടിലും 1 ഉള്ളിടത്ത് മാത്രം 1, അതായത് 0000 1100 = 12. Left shift ഒന്ന് ചെയ്താൽ value ഇരട്ടിക്കും, right shift ഒന്ന് ചെയ്താൽ പകുതിയാകും. Precedence ഗണിതത്തിലെ BODMAS പോലെയാണ്: ആദ്യം brackets, പിന്നെ **, പിന്നെ * / // %, പിന്നെ + −, അവസാനം comparisons, not, and, or.

Operator	Name	n1 = 60 (0011 1100) , n2 = 13 (0000 1101)	Result
&	Binary AND	n1 & n2	12 (0000 1100)
|	Binary OR	n1 | n2	61 (0011 1101)
^	Binary XOR	n1 ^ n2	49 (0011 0001)
~	Ones complement	~n1	-61
<<	Left shift	n1 << 2	240 (1111 0000)
>>	Right shift	n1 >> 2	15 (0000 1111)

Precedence (highest → lowest):

() → ** → +x, -x, ~x → *, /, //, % → +, - → <<, >> → & → ^ → | → ==, !=, >, >=, <, <= → not → and → or

EXAMPLE 1: BITWISE OPERATORS (TABLE 8)

```
n1, n2 = 60, 13
print(n1 & n2, n1 | n2, n1 ^ n2, ~n1)
print(n1 << 2, n1 >> 2)
print(format(n1, '08b'), format(n2, '08b'), format(n1 & n2, '08b'))
a, b = 4, 11           # 0100 and 1011
print(a | b)
print(a >> 2)
```

```
12 61 49 -61
240 15
00111100 00001101 00001100
15
1
```

EXAMPLE 2: PRECEDENCE IN ACTION

```
print(2 ** 3 ** 2)       # right to left: 2 ** 9
print((2 ** 3) ** 2)
print(-2 ** 2)           # ** binds tighter than unary minus
print(2 + 3 * 4, (2 + 3) * 4)
print(1 + 2 == 3 and not 5 < 2)
```

```
512
64
-4
14 20
True
```

Exam tip: Table 9's "~n1" column shows ~0 = 1 and ~1 = 0 at the single-bit level. On a Python int, `~0` is -1 and `~1` is -2, because ~n = −(n + 1).

Data types: numeric, Boolean and None

Data type: specifies the kind of data (such as numbers or characters) a program stores and manipulates. In Python every value is an **object** having an identity, a type and a value.

Data type: ഒരു program സൂക്ഷിക്കുകയും കൈകാര്യം ചെയ്യുകയും ചെയ്യുന്ന data-യുടെ തരം (numbers, characters മുതലായവ) വ്യക്തമാക്കുന്നു. Python-ൽ എല്ലാ value-ഉം identity, type, value എന്നിവയുള്ള ഒരു **object** ആണ്.

Python has no "primitive" types as in C or Java; everything is an object. `id()` gives an object's identity, `type()` its class. The numeric types are `int`, `float` and `complex`. `bool` is a subclass of `int` with only two instances, True (= 1) and False (= 0). `None` is the only value of type `NoneType` and represents the absence of a value. Converting one type to another (e.g. `int("45")`) is called type conversion or type casting.

> **മലയാളത്തിൽ**
>
> Python-ൽ 5 പോലുള്ള ഒരു ചെറിയ number പോലും ഒരു object ആണ്; ഓരോ object-നും ഒരു identity (memory address, `id()`), ഒരു type (`type()`), ഒരു value ഉണ്ട്. Aadhaar number (identity), ജോലി (type), പേര് (value) ഉള്ള ഒരാളെ പോലെ. Numbers മൂന്ന് തരം: int, float, complex. True യഥാർത്ഥത്തിൽ 1-ഉം False 0-ഉം ആണ്, അതുകൊണ്ട് True + True = 2. None എന്നാൽ "value ഇല്ല".

Type	Class	Examples
Integer	int	-12, 0, 123
Float	float	-2.04, 4.0, 14.23
Complex	complex	3+4j, 2-2j
Boolean	bool	True, False
String	str	'hello', "python course"
None	NoneType	None

EXAMPLE 1: TYPE() AND BOOL AS A SUBCLASS OF INT

```
print(type(-12), type(4.0), type(3+4j))
print(type(True), type("hi"), type(None))
print(isinstance(True, int), True + True, False * 10)
money = None
print(money is None)
```

```
<class 'int'> <class 'float'> <class 'complex'>
<class 'bool'> <class 'str'> <class 'NoneType'>
True 2 0
True
```

EXAMPLE 2: TYPE CONVERSION (TYPE CASTING)

```
print(int("45") + 5)
print(float(3), str(10) + "0", bool(0), bool(7))
print(int(9.99))       # int() truncates, it does not round
```

```
50
3.0 100 False True
9
```

Exam tip: section 2.6.1 says numbers "come in four types: Integer, Float, and Complex" — there are **three** numeric types (int, float, complex). Converting one data type to another is called both type conversion and type casting (CYP-1 Q8 answer: c).

Strings

String: an immutable, ordered sequence of characters enclosed in single, double or triple quotes; it belongs to the built-in class `str`.

String: single, double അല്ലെങ്കിൽ triple quotes-ങ്ങളിൽ എഴുതുന്ന characters-ന്റെ immutable, ordered sequence; ഇത് built-in class `str`-ൽ പെടുന്നു.

Python has no separate character type; a character is a string of length 1. A string has a length (`len()`), can be indexed from 0 (negative indices count from the end, -1 is the last character) and sliced with `s[start:end]`, which includes `start` but excludes `end`. + concatenates and * repeats. Strings are immutable, so assigning to `s[i]` raises TypeError. Arithmetic is not possible on a string even if it looks numeric, like "452".

മലയാളത്തിൽ

String ഒരു മാലയിലെ മുത്തുകൾ പോലെ ക്രമത്തിൽ കോർത്ത characters ആണ്; ഓരോ മുത്തിനും 0 മുതൽ തുടങ്ങുന്ന ഒരു index ഉണ്ട്. പിന്നിൽ നിന്ന് എണ്ണാൻ −1, −2 ഉപയോഗിക്കാം. Slice `[4:8]` എന്നാൽ index 4 മുതൽ 7 വരെ, 8 ഉൾപ്പെടില്ല. String immutable ആണ്, അതായത് ഒരിക്കൽ ഉണ്ടാക്കിയാൽ അതിലെ ഒരു character മാറ്റാൻ പറ്റില്ല; പുതിയൊരു string ഉണ്ടാക്കണം.

len(s) → number of characters

s[i] → character at index i (0-based); s[-1] → last character

s[a:b] → characters from index a up to b-1; s[:b] from start; s[a:] to end

s1 + s2 → concatenation; s * n → n copies

EXAMPLE 1: IGNOU'S STRING EXAMPLE, RUN FOR REAL

```
str1 = 'Hello Friend'
str2 = "452"
print(len(str1))
print(str1[4], str1[-1])
print(str1[4:8])
print(str1[:3])
print(str1[3:])
str2 = ' Narender'
print(str1 + str2)
print(str1 * 3)
```

```
12
o d
o Fr
Hel
lo Friend
Hello Friend Narender
Hello FriendHello FriendHello Friend
```

Index map: H0 e1 l2 l3 o4 (space)5 F6 r7 i8 e9 n10 d11, so `str1[4:8]` is 'o Fr'.

EXAMPLE 2: STRINGS ARE IMMUTABLE

```
str1 = 'Hello Friend'
new = str1[:2] + 'a' + str1[3:]    # build a NEW string instead
print(new)
str1[2] = 'a'
```

```
Healo Friend
TypeError: 'str' object does not support item assignment
```

EXAMPLE 3: A NUMERIC-LOOKING STRING IS STILL A STRING

```
str2 = "452"
print(int(str2) + 1)
print(str2 + 1)
```

```
453
TypeError: can only concatenate str (not "int") to str
```

Exam tip: the unit's printed outputs use a lowercase 'h' although `str1 = 'Hello Friend'`. Correct outputs: `str1[:3]` → 'Hel' (not 'hel'), `str1 + str2` → 'Hello Friend Narender', and `str1 * 3` → 'Hello FriendHello FriendHello Friend' (no spaces are added between copies).

Data structures at a glance

Data structure: a way of organising, storing and managing data for efficient access and modification. List, tuple, dictionary and set are Python's built-in (non-primitive) data structures.

Data structure: data കാര്യക്ഷമമായി access ചെയ്യാനും മാറ്റാനും വേണ്ടി അതിനെ ക്രമീകരിക്കുകയും സൂക്ഷിക്കുകയും കൈകാര്യം ചെയ്യുകയും ചെയ്യുന്ന രീതി. List, tuple, dictionary, set എന്നിവ Python-ന്റെ built-in (non-primitive) data structures ആണ്.

Officially these four are built-in compound data types, but because each has its own rules for ordering, indexing and storing items, they are studied as data structures. A **list** is an ordered, mutable sequence in [ ]. A **tuple** is an ordered, immutable sequence in ( ). A **dictionary** stores key:value pairs in { } with unique, immutable keys. A **set** is an unordered, mutable collection of unique items in { }.

മലയാളത്തിൽ

ഒരു അടുക്കളയിൽ സാധനങ്ങൾ സൂക്ഷിക്കാൻ പല പാത്രങ്ങൾ ഉള്ളതുപോലെ Python-ൽ data സൂക്ഷിക്കാൻ നാല് "പാത്രങ്ങൾ" ഉണ്ട്. List ഒരു shopping list പോലെ, ക്രമമുണ്ട്, വെട്ടുകയും ചേർക്കുകയും ചെയ്യാം. Tuple ഒരു printed certificate പോലെ, മാറ്റാൻ പറ്റില്ല. Dictionary ഫോണിലെ contact list പോലെ, പേര് (key) കൊടുത്താൽ നമ്പർ (value) കിട്ടും. Set ഒരു attendance register പോലെ, ഒരാളുടെ പേര് രണ്ട് തവണ വരില്ല.

Feature	List	Tuple	Dictionary	Set
Syntax	[1, 2, 3]	(1, 2, 3)	{'a': 1, 'b': 2}	{1, 2, 3}
Class	list	tuple	dict	set
Empty	[] or list()	() or tuple()	{} or dict()	set() only
Mutable?	Yes	No	Yes (keys immutable)	Yes (items immutable)
Ordered?	Yes	Yes	Insertion order (3.7+)	No
Indexing / slicing?	Yes / Yes	Yes / Yes	By key / No	No / No
Duplicates?	Allowed	Allowed	Keys unique, values may repeat	Not allowed
Heterogeneous?	Yes	Yes	Yes	Yes (hashable items)
Typical use	Collection that changes	Fixed record, multiple return values	Lookup by key	Membership test, removing duplicates

EXAMPLE: THE FOUR STRUCTURES (IGNOU SECTIONS 2.6.4-2.6.7)

```
list1 = [5, 3.4, 'IGNOU', 'Part 3', 45]
print(list1, list1[0], list1[3], list1[-1], list1[-3])
t = ('hello', 'world', 2)
print(t, t[1])
dict1 = {'Fruit': 'Apple', 'Climate': 'Cold', 'Price(Kg)': 120}
print(dict1)
print(dict1['Climate'])
print(dict1.keys())
print(dict1.values())
set2 = set([2, 3, 2, 'MCA', 'IGNOU', 1, 'IGNOU'])
print(len(set2))
print(sorted(set("IGNOU MCA BATCH")))
```

```
[5, 3.4, 'IGNOU', 'Part 3', 45] 5 Part 3 45 IGNOU
('hello', 'world', 2) world
{'Fruit': 'Apple', 'Climate': 'Cold', 'Price(Kg)': 120}
Cold
dict_keys(['Fruit', 'Climate', 'Price(Kg)'])
dict_values(['Apple', 'Cold', 120])
5
[' ', 'A', 'B', 'C', 'G', 'H', 'I', 'M', 'N', 'O', 'T', 'U']
```

set2 keeps only 5 unique items: 1, 2, 3, 'MCA', 'IGNOU'. We print `sorted(...)` for the letter set because printing a set of strings shows them in an order that can change from run to run.

Exam tip: the unit calls these both "primitive data structures" and "non-primitive data structures" in the same paragraph. The correct term is **non-primitive** (built-in compound) data structures. Also, `{}` creates an empty dictionary, not an empty set.

List: creation, indexing and slicing

List: an ordered, mutable, heterogeneous sequence of items separated by commas and enclosed in square brackets [ ]; it acts as a dynamic array. **Index:** the integer position of an item, starting at 0.

List: square brackets [ ]-നുള്ളിൽ comma കൊണ്ട് വേർതിരിച്ച items-ന്റെ ordered, mutable, heterogeneous sequence; ഇത് ഒരു dynamic array പോലെ പ്രവർത്തിക്കുന്നു. **Index:** 0 മുതൽ തുടങ്ങുന്ന, ഒരു item-ന്റെ integer സ്ഥാനം.

A list has a current length (`len()`), an order from left to right, can mix types (numbers, strings, even other lists) and can grow and shrink. Items are accessed with `list_name[index]`; valid indices for n items are 0 to n−1, and -1 to −n from the end. An index outside this range raises `IndexError`. Slicing `list[a:b]` returns a new list of items a to b−1; omitting both indices (`list[:]`) copies the whole list.

മലയാളത്തിൽ

List ഒരു train-ലെ compartments പോലെയാണ്: ഓരോ compartment-നും ഒരു number (index) ഉണ്ട്, ആദ്യത്തേത് 0. ഒരു compartment-ൽ ആളുകൾ, മറ്റൊന്നിൽ ചരക്ക്, അതായത് പല type items ഒരുമിച്ച് വെക്കാം. 6 items ഉള്ള list-ൽ index 0 മുതൽ 5 വരെ മാത്രം; numlist[6] ചോദിച്ചാൽ IndexError. -1 എന്നാൽ അവസാനത്തെ item, -2 അതിന് മുമ്പുള്ളത്.

list_name = [item_1, item_2, ..., item_n]
list_name[index]   list_name[start:end]   list_name[:] (full copy)

EXAMPLE 1: CREATING LISTS AND INDEXING

```
empty = []
stringlist = ["mahesh", "ramesh", "suresh", "vishnu", "ganesh"]
numlist = [4, 6, 7, 10, 15, 13]
mixed_list = ['cat', 87.23, 65, [9, 8, 1]]
print(empty, len(numlist))
print(stringlist[0], numlist[4], stringlist[-2])
print(mixed_list[3], mixed_list[3][0])
```

```
[] 6
mahesh 15 vishnu
[9, 8, 1] 9
```

EXAMPLE 2: SLICING

```
numlist = [4, 6, 7, 10, 15, 13]
mixed_list = ['cat', 87.23, 65, [9, 8, 1]]
print(numlist[1:3])
print(mixed_list[:3])
print(mixed_list[2:])
print(numlist[:])
print(numlist[-3:])
print(numlist[::2])      # every 2nd item (step)
```

```
[6, 7]
['cat', 87.23, 65]
[65, [9, 8, 1]]
[4, 6, 7, 10, 15, 13]
[10, 15, 13]
[4, 7, 15]
```

EXAMPLE 3: INDEX OUT OF RANGE

```
numlist = [4, 6, 7, 10, 15, 13]
print(numlist[10:20])    # slicing never raises an error
print(numlist[6])
```

```
[]
IndexError: list index out of range
```

Exam tip: CYP-2 Q2 answer is (b) `IndexError: list index out of range`. An out-of-range index raises an error, but an out-of-range slice just returns what exists (possibly `[]`).

List operations and the list() function

Concatenation (+) joins two lists into a new list; **repetition (*)** repeats a list a given number of times; **membership (in, not in)** tests whether an item is present. **list([sequence])** creates a list from a sequence, or an empty list if none is given.

Concatenation (+) രണ്ട് lists ചേർത്ത് ഒരു പുതിയ list ഉണ്ടാക്കുന്നു; **repetition (*)** ഒരു list നിശ്ചിത തവണ ആവർത്തിക്കുന്നു; **membership (in, not in)** ഒരു item ഉണ്ടോ എന്ന് പരിശോധിക്കുന്നു. **list([sequence])** ഒരു sequence-ൽ നിന്ന് list ഉണ്ടാക്കുന്നു; ഒന്നും കൊടുത്തില്ലെങ്കിൽ empty list.

== compares two lists item by item. + works only between two lists; adding a string to a list raises TypeError, so convert the string with `list()` first. `list("abc")` splits a string into its characters; `list()` with no argument gives `[]`.

മലയാളത്തിൽ

+ കൊണ്ട് രണ്ട് list-കൾ ഒട്ടിച്ച് ഒന്നാക്കാം, * കൊണ്ട് ഒരു list പല തവണ ആവർത്തിക്കാം. in ചോദിക്കുന്നത് "ഈ item list-ൽ ഉണ്ടോ?" എന്നാണ്. List-ലേക്ക് നേരിട്ട് ഒരു string + ചെയ്യാൻ പറ്റില്ല; ആദ്യം `list()` കൊണ്ട് string-നെ characters-ന്റെ list ആക്കണം, പിന്നെ ചേർക്കാം.

EXAMPLE 1: +, *, == AND MEMBERSHIP

```
a = [1, 2, 3]
b = [4, 5, 6]
print(a + b)
c = ['pass']
print(c * 3)
print(a == b, a == [1, 2, 3])
print(2 in a, 4 not in a)
```

```
[1, 2, 3, 4, 5, 6]
['pass', 'pass', 'pass']
False True
True True
```

EXAMPLE 2: THE LIST() FUNCTION

```
print(list())
greet = "How are you doing?"
str_to_list = list(greet)
print(str_to_list)
print(list((1, 2, 3)))
friend = "nidhi"
print(str_to_list + list(friend))
print(str_to_list + friend)
```

```
[]
['H', 'o', 'w', ' ', 'a', 'r', 'e', ' ', 'y', 'o', 'u', ' ', 'd', 'o', 'i', 'n',
'g', '?']
[1, 2, 3]
['H', 'o', 'w', ' ', 'a', 'r', 'e', ' ', 'y', 'o', 'u', ' ', 'd', 'o', 'i', 'n',
'g', '?', 'n', 'i', 'd', 'h', 'i']
TypeError: can only concatenate list (not "str") to list
```

Exam tip: the unit writes "`4 not in [a]` will also give True". `[a]` is a list whose only item is the list a, so it is True for a different reason; the intended test is `4 not in a`, which is also True. CYP-2 Q1: both `[]` and `list()` create an empty list (answer c).

Modifying and traversing lists

Mutable: an object whose items can be changed, added or removed after creation without changing its identity (`id()`); lists are mutable. **Aliasing:** two variable names referring to the same list object.

Mutable: ഉണ്ടാക്കിയ ശേഷവും identity (`id()`) മാറാതെ തന്നെ items മാറ്റാനും ചേർക്കാനും നീക്കാനും കഴിയുന്ന object; lists mutable ആണ്. **Aliasing:** രണ്ട് variable പേരുകൾ ഒരേ list object-നെ refer ചെയ്യുന്ന അവസ്ഥ.

Assigning to `list[i]` replaces an item in place. Writing `new = old` does not copy a list; both names point to the same list, so a change through one name is seen through the other. Use `old[:]` or `old.copy()` for a real copy. To read every item use `for item in list`; to update items you need indices, so combine `range()` and `len()`.

മലയാളത്തിൽ

List mutable ആയതിനാൽ `stringlist[1] = "umesh"` എന്ന് എഴുതിയാൽ അതേ list-ൽ തന്നെ മാറ്റം വരും. `listofstrings = stringlist` എന്ന് എഴുതുമ്പോൾ പുതിയ list ഉണ്ടാകുന്നില്ല; ഒരേ വീടിന് രണ്ട് വിലാസ-ബോർഡ് വെച്ചതു പോലെ, ഒരു പേരിലൂടെ മാറ്റിയാൽ മറ്റേ പേരിലും അത് കാണും. Items വായിക്കാൻ മാത്രമാണെങ്കിൽ സാധാരണ for loop മതി; items മാറ്റണമെങ്കിൽ `range(len(list))` ഉപയോഗിച്ച് index വഴി പോകണം.

EXAMPLE 1: MODIFYING AN ITEM AND ALIASING

```
stringlist = ["mahesh", "ramesh", "suresh", "vishnu", "ganesh"]
before = id(stringlist)
stringlist[1] = "umesh"
print(stringlist, id(stringlist) == before)
listofstrings = stringlist          # alias, not a copy
listofstrings[1] = "ramesh"
print(listofstrings)
print(stringlist)
copy_list = stringlist[:]           # a real copy
copy_list[0] = "X"
print(stringlist[0], copy_list[0])
```

```
['mahesh', 'umesh', 'suresh', 'vishnu', 'ganesh'] True
['mahesh', 'ramesh', 'suresh', 'vishnu', 'ganesh']
['mahesh', 'ramesh', 'suresh', 'vishnu', 'ganesh']
mahesh X
```

EXAMPLE 2: TRAVERSING TO READ, AND TO UPDATE

```
stringlist = ["mahesh", "ramesh", "suresh", "vishnu", "ganesh"]
for names in stringlist:
    print(names)
numlist = [4, 6, 7, 10, 15, 13]
for i in range(len(numlist)):
    numlist[i] = numlist[i] * 2
print(numlist)
print(range(len(numlist)), list(range(len(numlist))))
```

```
mahesh
ramesh
suresh
vishnu
ganesh
[8, 12, 14, 20, 30, 26]
range(0, 6) [0, 1, 2, 3, 4, 5]
```

Exam tip: "Why is a list mutable?" → its items can be changed, added or removed after creation without creating a new list (same `id()`). The unit says `range()` "returns a list of indices"; in Python 3 it returns a `range` object that produces 0 to n−1 lazily (wrap it in `list()` to see them).

List methods

Method: a function that belongs to an object and is called with dot notation, `object.method(arguments)`. Most list methods modify the list in place and return None.

Method: ഒരു object-ന് സ്വന്തമായ, dot notation (`object.method(arguments)`) ഉപയോഗിച്ച് വിളിക്കുന്ന function. മിക്ക list methods-ഉം list-നെ in place ആയി മാറ്റുകയും None തിരികെ നൽകുകയും ചെയ്യുന്നു.

`append` adds one item at the end; `extend` adds all items of another list; `insert(i, x)` puts x at index i; `remove(x)` deletes the first x (ValueError if absent); `pop([i])` removes and returns the item at i (last item if i is omitted); `index(x)` gives the first position of x; `count(x)` counts x; `sort()` and `reverse()` reorder in place. The `del` statement removes by index or slice without returning anything. `dir(list)` lists all methods.

മലയാളത്തിൽ

Methods list-ന്റെ "സ്വന്തം ഉപകരണങ്ങൾ" ആണ്. append ഒരു item അവസാനം ചേർക്കും, extend മറ്റൊരു list-ലെ എല്ലാ items-ഉം ചേർക്കും. Index അറിയാമെങ്കിൽ pop (നീക്കിയ value തിരികെ തരും) അല്ലെങ്കിൽ del (ഒന്നും തരില്ല); value മാത്രം അറിയാമെങ്കിൽ remove. sort(), reverse() എന്നിവ അതേ list മാറ്റുന്നു, പുതിയ list തരുന്നില്ല; അതുകൊണ്ട് `print(numlist.sort())` None കാണിക്കും.

Method	Syntax	What it does	Returns
append()	list.append(item)	Adds one item at the end	None
extend()	list.extend(list2)	Adds every item of list2 at the end	None
insert()	list.insert(index, item)	Inserts item at index, shifting others right	None
remove()	list.remove(item)	Removes first occurrence; ValueError if absent	None
pop()	list.pop([index])	Removes item at index (default: last)	The removed item
index()	list.index(item)	Position of first occurrence; ValueError if absent	int
count()	list.count(item)	Number of occurrences	int
sort()	list.sort()	Sorts in place (low to high; reverse=True for high to low)	None
reverse()	list.reverse()	Reverses in place	None

EXAMPLE 1: IGNOU'S NUMLIST SEQUENCE

```
numlist = [8, 12, 14, 20, 30, 26]
numlist.append(50)
print(numlist)
nlist = [25, 35, 45]
numlist.extend(nlist)
print(numlist)
numlist.sort()
print(numlist)
print(numlist.pop(5))
print(numlist)
del numlist[0]
print(numlist)
del numlist[0:2]
print(numlist)
numlist.remove(30)
print(numlist)
```

```
[8, 12, 14, 20, 30, 26, 50]
[8, 12, 14, 20, 30, 26, 50, 25, 35, 45]
[8, 12, 14, 20, 25, 26, 30, 35, 45, 50]
26
[8, 12, 14, 20, 25, 30, 35, 45, 50]
[12, 14, 20, 25, 30, 35, 45, 50]
[20, 25, 30, 35, 45, 50]
[20, 25, 35, 45, 50]
```

EXAMPLE 2: INSERT, COUNT, INDEX, REVERSE, POP()

```
marks = [45, 67, 45, 90]
marks.insert(1, 55)
print(marks)
print(marks.count(45), marks.index(45))
marks.reverse()
print(marks)
print(marks.pop(), marks)
marks.sort(reverse=True)
print(marks)
print(marks.sort())          # sort() returns None
marks.remove(100)
```

```
[45, 55, 67, 45, 90]
2 0
[90, 45, 67, 55, 45]
45 [90, 45, 67, 55]
[90, 67, 55, 45]
None
ValueError: list.remove(x): x not in list
```

EXAMPLE 3: APPEND VS EXTEND

```
x = [1, 2]
y = [1, 2]
x.append([3, 4])
y.extend([3, 4])
print(x, len(x))
print(y, len(y))
```

```
[1, 2, [3, 4]] 3
[1, 2, 3, 4] 4
```

append adds the whole list as one item; extend adds its items one by one.

Exam tip: pop vs remove vs del is a favourite question: pop works by index and returns the item, remove works by value and returns None, del works by index or slice and is a statement (not a method).

Built-in functions used on lists

Built-in functions such as len(), sum(), max(), min(), any(), all() and sorted() take a list as an argument; **sorted()** returns a new sorted list and leaves the original unchanged.

len(), sum(), max(), min(), any(), all(), sorted() പോലുള്ള **built-in functions** ഒരു list argument ആയി സ്വീകരിക്കുന്നു; **sorted()** ഒരു പുതിയ sorted list തിരികെ നൽകുന്നു, original list മാറ്റുന്നില്ല.

`len()` counts items, `sum()` adds numbers, `max()`/`min()` give the largest/smallest, `any()` is True if at least one item is truthy, and `all()` is True only if every item is truthy (0 is falsy, non-zero is truthy). Unlike the method `list.sort()`, the function `sorted()` makes a copy.

മലയാളത്തിൽ

ഇവ list-ന്റെ methods അല്ല, എല്ലാ sequence-ലും ഉപയോഗിക്കാവുന്ന പൊതു functions ആണ്. any() "ഒരാളെങ്കിലും ഹാജരുണ്ടോ?", all() "എല്ലാവരും ഹാജരുണ്ടോ?" എന്ന് ചോദിക്കുന്നു; 0 എന്നത് False ആയി കണക്കാക്കും. `sorted(list)` ഒരു photocopy എടുത്ത് അതിനെ sort ചെയ്യും, original അതേപടി ഇരിക്കും; `list.sort()` original തന്നെ മാറ്റും.

	list.sort()	sorted(list)
Kind	Method (lists only)	Built-in function (any iterable)
Original list	Changed	Unchanged
Returns	None	New sorted list

EXAMPLE

```
stringlist = ["mahesh", "ramesh", "suresh", "vishnu", "ganesh"]
numlist = [20, 25, 35, 45, 50]
print(len(stringlist))
print(sum(numlist), max(numlist), min(numlist))
print(any([1, 1, 0, 0, 1, 0]), all([1, 1, 1, 1]), all([1, 0, 1]))
sorted_stringlist = sorted(stringlist)
print(sorted_stringlist)
print(stringlist)
```

```
5
175 50 20
True True False
['ganesh', 'mahesh', 'ramesh', 'suresh', 'vishnu']
['mahesh', 'ramesh', 'suresh', 'vishnu', 'ganesh']
```

Exam tip: the unit uses `max()` and `min()` in its examples but omits them from Table 13; they are equally important built-ins. `all([])` is True and `any([])` is False.

Nested lists

Nested list: a list that contains other lists as its items; an item is accessed with two indices, `list[i][j]` (row i, column j).

Nested list: മറ്റ് lists-നെ items ആയി ഉൾക്കൊള്ളുന്ന list; ഒരു item `list[i][j]` (row i, column j) എന്ന രണ്ട് indices ഉപയോഗിച്ച് access ചെയ്യുന്നു.

Nested lists represent tables and matrices. The first index selects the inner list (row) and the second selects the item inside it (column). Nested `for` loops traverse them row by row. A classic program is the transpose of a matrix, where element [r][c] moves to [c][r].

മലയാളത്തിൽ

Nested list ഒരു theatre-ലെ seating പോലെയാണ്: ആദ്യ index row, രണ്ടാമത്തേത് ആ row-ലെ seat. `asia[1][2]` എന്നാൽ രണ്ടാം row-ലെ മൂന്നാം item. Matrix transpose-ൽ rows columns ആകുന്നു: [r][c]-ലെ value [c][r]-ലേക്ക് മാറ്റുന്നു, അതിന് രണ്ട് for loops ഒന്നിനുള്ളിൽ മറ്റൊന്നായി വേണം.

EXAMPLE 1: ACCESSING A NESTED LIST

```
asia = [["India", "Japan", "Korea"],
        ["Srilanka", "Myanmar", "Thailand"],
        ["Cambodia", "Vietnam", "Israel"]]
print(asia[0])
print(asia[0][1])
print(asia[1][2])
for row in asia:
    for country in row:
        print(country, end=" ")
    print()
```

```
['India', 'Japan', 'Korea']
Japan
Thailand
India Japan Korea
Srilanka Myanmar Thailand
Cambodia Vietnam Israel
```

EXAMPLE 2: TRANSPOSE OF A MATRIX (IGNOU PROGRAM)

```
matrix = [[10, 20],
          [30, 40],
          [50, 60]]
matrix_transpose = [[0, 0, 0],
                    [0, 0, 0]]
for rows in range(len(matrix)):
    for columns in range(len(matrix[0])):
        matrix_transpose[columns][rows] = matrix[rows][columns]
print("Transposed Matrix is")
for items in matrix_transpose:
    print(items)
```

```
Transposed Matrix is
[10, 30, 50]
[20, 40, 60]
```

The 3×2 matrix becomes 2×3. The assignment line must be indented inside the inner loop (the extracted IGNOU text loses this indentation; without it you get an IndentationError).

Tuple: creation and basic operations

Tuple: an ordered, finite, immutable sequence of possibly heterogeneous items, written as comma-separated values usually enclosed in parentheses ( ); it belongs to the built-in class `tuple`.

Tuple: സാധാരണയായി parentheses ( )-നുള്ളിൽ comma കൊണ്ട് വേർതിരിച്ച് എഴുതുന്ന, വ്യത്യസ്ത type ആകാവുന്ന items-ന്റെ ordered, finite, immutable sequence; ഇത് built-in class `tuple`-ൽ പെടുന്നു.

It is the **comma** that makes a tuple, not the parentheses: `t = 'a', 'b'` is a tuple, and a single-item tuple needs a trailing comma, `('a',)`; `('a')` is just a string. Indexing, slicing, +, *, `in`/`not in` and comparisons all work, but assigning to an item raises TypeError. You can, however, build a new tuple and rebind the name. Tuples are compared element by element from the left; the first unequal pair decides.

മലയാളത്തിൽ

Tuple ഒരു "sealed" list ആണ്: ഉണ്ടാക്കിയ ശേഷം items മാറ്റാനോ ചേർക്കാനോ നീക്കാനോ പറ്റില്ല, ഒരു birth certificate പോലെ. Tuple ഉണ്ടാക്കുന്നത് comma ആണ്, bracket അല്ല; അതുകൊണ്ട് ഒരു item മാത്രമുള്ള tuple-ന് `('a',)` എന്ന് comma നിർബന്ധം. Item മാറ്റണമെങ്കിൽ പഴയതിൽ നിന്ന് ഒരു പുതിയ tuple ഉണ്ടാക്കി അതേ പേരിൽ വെക്കാം. Compare ചെയ്യുമ്പോൾ ആദ്യം വ്യത്യാസം കാണുന്ന item മാത്രം നോക്കും, ബാക്കി എത്ര വലുതാണെങ്കിലും നോക്കില്ല.

```
tuple_name = (item_1, item_2, ..., item_n)
t = 'a', 'b', 'c'    (parentheses optional)
single item: ('a',)    empty: () or tuple()
```

EXAMPLE 1: CREATING TUPLES

```
t = 'a', 'b', 'c', 'd', 'e'
print(t, type(t))
t1 = ('a',)
t2 = ('a')
print(type(t1), type(t2))
empty_tuple = ()
print(empty_tuple)
socis = ('mca', '6 sem', '3-6 yrs', 54000)
ignou = ('soe', 'soh', socis, 'soms')
print(ignou)
```

```
('a', 'b', 'c', 'd', 'e') <class 'tuple'>
<class 'tuple'> <class 'str'>
()
('soe', 'soh', ('mca', '6 sem', '3-6 yrs', 54000), 'soms')
```

EXAMPLE 2: INDEXING, SLICING, +, *, IN, COMPARISON

```
t = ('a', 'b', 'c', 'd', 'e')
print(t[0], t[1:3])
t = ('A',) + t[1:]           # new tuple, same name
print(t)
print(t * 2)
print('b' in t, 'z' not in t)
print((0, 1, 2) < (0, 3, 4))
print((0, 1, 500000) < (0, 3, 4))
t[0] = 'a'
```

```
a ('b', 'c')
('A', 'b', 'c', 'd', 'e')
('A', 'b', 'c', 'd', 'e', 'A', 'b', 'c', 'd', 'e')
True True
True
True
TypeError: 'tuple' object does not support item assignment
```

Exam tip: CYP-2 Q3: with `tupl = (5, 2, 7, 0, 3)`, only `tupl1 = tupl + tupl` runs without error (answer c); item assignment, `append()` and `sort()` all fail because tuples are immutable.

The tuple() function, built-in functions and tuple methods

tuple([iterable]) creates a tuple from the items of an iterable (string, list, tuple, etc.) or an empty tuple if no argument is given. A tuple has only two methods: **count()** and **index()**.

tuple([iterable]) ഒരു iterable-ലെ (string, list, tuple മുതലായവ) items-ൽ നിന്ന് tuple ഉണ്ടാക്കുന്നു; argument ഇല്ലെങ്കിൽ empty tuple. Tuple-ന് രണ്ട് methods മാത്രം: **count()**, **index()**.

`tuple('IGNOU')` gives one item per character. `len()`, `sum()`, `max()`, `min()` and `sorted()` accept tuples; `sorted()` returns a list, leaving the tuple untouched. Because a tuple cannot change, it has no append, remove or sort; only `count(item)` and `index(item)` (ValueError if the item is absent).

മലയാളത്തിൽ

tuple() function ഒരു string-ഓ list-ഓ എടുത്ത് tuple ആക്കി മാറ്റും. Tuple മാറ്റാൻ പറ്റാത്തതിനാൽ മാറ്റം വരുത്തുന്ന methods ഒന്നുമില്ല; എണ്ണാൻ count(), സ്ഥാനം കണ്ടുപിടിക്കാൻ index() മാത്രം. sorted() ഒരു tuple-നെ sort ചെയ്ത് തരുന്നത് list ആയിട്ടാണ്, tuple ആയിട്ടല്ല എന്ന് ഓർക്കുക.

EXAMPLE 1: TUPLE() AND BUILT-IN FUNCTIONS

```
t3 = tuple()
print(t3)
t3 = tuple('IGNOU')
print(t3)
t4 = (1, 2, 3, 4)
nested_t = (t3, t4)
print(nested_t)
print(tuple([10, 20]))
print(len(t3), sum(t4), max(t4), min(t4))
t5 = sorted(t3)
print(t5, type(t5))
print(t3)
```

```
()
('I', 'G', 'N', 'O', 'U')
(('I', 'G', 'N', 'O', 'U'), (1, 2, 3, 4))
(10, 20)
5 10 4 1
['G', 'I', 'N', 'O', 'U'] <class 'list'>
('I', 'G', 'N', 'O', 'U')
```

EXAMPLE 2: COUNT() AND INDEX()

```
channels = ("dd", "star", "sony", "zee", "dd", "sab")
print(channels.count("dd"), channels.index("dd"))
print(channels.index("zee"), channels.count("sab"))
print(channels.index("hbo"))
```

```
2 0
3 1
ValueError: tuple.index(x): x not in tuple
```

EXAMPLE 3: TUPLE() NEEDS AN ITERABLE, NOT A SINGLE NUMBER

```
tuple(5)
```

```
TypeError: 'int' object is not iterable
```

Exam tip: the unit says the sequence given to `tuple()` "can be a number, string or tuple". A number is not allowed: `tuple(5)` raises TypeError. The argument must be an iterable such as a string, list, tuple, set or dictionary.

Tuple assignment and tuples vs lists

Tuple assignment (unpacking): assigning the items of a sequence on the right to a tuple of variables on the left in one statement; the number of variables must equal the number of values.

Tuple assignment (unpacking): വലതുവശത്തെ sequence-ലെ items ഇടതുവശത്തെ variables-ന്റെ tuple-ലേക്ക് ഒറ്റ statement-ൽ assign ചെയ്യുന്നത്; variables-ന്റെ എണ്ണവും values-ന്റെ എണ്ണവും തുല്യമായിരിക്കണം.

`x, y = m` is roughly `x = m[0]; y = m[1]`. It gives the famous swap `a, b = b, a` without a temporary variable. A tuple itself cannot change, but if one of its items is mutable (e.g. a list), that item can be changed; immutability applies to the container, not to the objects it refers to. Since you cannot add to a tuple, you "grow" one by creating a new tuple with +=, or by collecting items in a list and converting it with `tuple()`.

മലയാളത്തിൽ

Tuple assignment-ൽ വലതുവശത്തെ values ക്രമത്തിൽ ഇടതുവശത്തെ variables-ലേക്ക് "പൊട്ടിച്ച്" ഇടുന്നു. അതുകൊണ്ട് `a, b = b, a` എന്ന് എഴുതിയാൽ മൂന്നാമതൊരു glass ഇല്ലാതെ തന്നെ രണ്ട് glass-ലെ ചായയും കാപ്പിയും മാറ്റിവെക്കാം. Tuple-നുള്ളിൽ ഒരു list ഉണ്ടെങ്കിൽ ആ list-ൽ append ചെയ്യാം; tuple-ന്റെ "slot" അതേ list-നെ തന്നെ ചൂണ്ടുന്നതിനാൽ tuple മാറിയിട്ടില്ല, list-ന്റെ ഉള്ളടക്കം മാത്രമാണ് മാറിയത്.

Feature	List	Tuple
Brackets	[ ]	( ) (comma is what matters)
Mutability	Mutable	Immutable
Methods	append, extend, insert, remove, pop, sort, reverse, count, index	count, index only
Usual contents	Items of a similar kind that change	Heterogeneous fixed record
Access	Indexing	Indexing or unpacking
Dictionary key?	No (unhashable)	Yes, if all its items are immutable

EXAMPLE 1: UNPACKING AND A TUPLE HOLDING A LIST

```
m = ['good', 'luck']
x, y = m
print(x, y)
(p, q) = m                  # same thing with parentheses
print(p, q)
ignou = ['soe', 'soms', 'socis']
univ = ('du', 'ggsipu', 'jnu', ignou)
univ[3].append('soss')
print(univ)
list_of_tuples = [(1, 2), (3, 4), (5, 6)]
print(list_of_tuples)
a, b = 1, 2, 3
```

```
good luck
good luck
('du', 'ggsipu', 'jnu', ['soe', 'soms', 'socis', 'soss'])
[(1, 2), (3, 4), (5, 6)]
ValueError: too many values to unpack (expected 2, got 3)
```

EXAMPLE 2: POPULATING A TUPLE AND A LIST FROM USER INPUT (IGNOU PROGRAM)

```
tuple_items = ()
total_items = int(input("Enter the total number of items for tuple: "))
for i in range(total_items):
    user_input = int(input("Enter a number: "))
    tuple_items += (user_input,)
print(f"Items added to tuple are : {tuple_items}")
list_items = []
total_items = int(input("Enter the total number of items for List: "))
for i in range(total_items):
    item = input("Enter an item to add: ")
    list_items.append(item)
items_of_tuple = tuple(list_items)
print(f"Tuple items are: {items_of_tuple}")
```

```
Enter the total number of items for tuple: 3
Enter a number: 10
Enter a number: 20
Enter a number: 30
Items added to tuple are : (10, 20, 30)
Enter the total number of items for List: 2
Enter an item to add: pen
Enter an item to add: book
Tuple items are: ('pen', 'book')
```

Sample run with inputs 3, 10, 20, 30, 2, pen, book. Method 1 creates a brand-new tuple on every += (note the comma in `(user_input,)`); method 2 fills a list and converts it once.

EXAMPLE 3: SWAP TWO NUMBERS WITHOUT A TEMPORARY VARIABLE (IGNOU PROGRAM)

```
a = int(input("Enter a value for the first number:"))
b = int(input("Enter a value for the second number:"))
b, a = a, b
print("After Swapping")
print(f"Value for first number: {a}")
print(f"Value for second number: {b}")
```

```
Enter a value for the first number:15
Enter a value for the second number:40
After Swapping
Value for first number: 40
Value for second number: 15
```

Exam tip: "Why can tuples contain mutable items?" → immutability applies to the tuple container (its size and which object sits in each position), not to the objects it references; so a list inside a tuple can still be modified. Unpacking with a wrong count raises ValueError.

Dictionary: creation and access

Dictionary: a mutable collection of key:value pairs enclosed in curly braces { }, in which keys are unique and immutable and each key maps to a value; it belongs to the built-in class `dict`.

Dictionary: curly braces { }–നുള്ളിൽ എഴുതുന്ന key:value pairs–ന്റെ mutable collection; ഇതിൽ keys unique–ഉം immutable–ഉം ആണ്; ഓരോ key–ഉം ഒരു value–ലേക്ക് map ചെയ്യുന്നു; ഇത് built-in class `dict`–ൽ പെടുന്നു.

A dictionary is a mapping from keys to values; one key with its value is called an item. Keys must be immutable (strings, numbers, tuples of immutables) and are case-sensitive; lists cannot be keys. Values may be of any type and may repeat. Values are looked up by key with `d[key]`, never by an integer position, and a missing key raises `KeyError`. Since Python 3.7 dictionaries preserve insertion order, but slicing is still not supported because a dictionary is accessed by key, not by position.

മലയാളത്തിൽ

Dictionary നമ്മുടെ ഫോണിലെ contact list പോലെയാണ്: എല്ലാ നമ്പറും ഓർത്തുവെക്കേണ്ട, പേര് (key) തിരഞ്ഞാൽ നമ്പർ (value) കിട്ടും. ഒരേ പേര് രണ്ട് തവണ key ആയി വരില്ല, പക്ഷേ രണ്ടുപേർക്ക് ഒരേ value ഉണ്ടാകാം. Key മാറ്റാൻ പറ്റാത്ത type ആയിരിക്കണം (string, number, tuple); list key ആക്കാൻ പറ്റില്ല. d[0] പോലെ position വെച്ച് എടുക്കാൻ പറ്റില്ല, key തന്നെ കൊടുക്കണം; ഇല്ലാത്ത key ചോദിച്ചാൽ KeyError.

dictionary_name = {key_1: value_1, key_2: value_2, ..., key_n: value_n}

d[key] → value   d[key] = value → add or change   empty: {} or dict()

EXAMPLE 1: CREATING AND ACCESSING

```
eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
print(eng2sp)
print(eng2sp['two'])
eng2sp['four'] = 'cuatro'        # add a new pair
eng2sp['one'] = 'UNO'            # change an existing value
print(eng2sp)
print({'A': 1, 'a': 2})          # keys are case-sensitive
print(eng2sp['five'])
```

```
{'one': 'uno', 'two': 'dos', 'three': 'tres'}
dos
{'one': 'UNO', 'two': 'dos', 'three': 'tres', 'four': 'cuatro'}
{'A': 1, 'a': 2}
KeyError: 'five'
```

EXAMPLE 2: INVALID KEY TYPE AND NO INTEGER INDEXING

```
d1 = {"Ravi": 5, "Mohan": 6, "Sohan": 7}
print({(1, 2): "tuple key ok"})
print(d1[0])
```

```
{(1, 2): 'tuple key ok'}
KeyError: 0
```

EXAMPLE 3: A LIST CANNOT BE A KEY

```
bad = {[1, 2]: "list key"}
```

```
TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
```

Lists are mutable, hence unhashable. (Older Python versions word the message as `TypeError: unhashable type: 'list'`.)

Exam tip: CYP-3 Q1: `print(d1[0])` → Error (answer d), specifically `KeyError: 0`, because 0 is not a key. The unit says ordering holds "with Python 3.6 onwards"; in 3.6 it was only a CPython implementation detail, and it is a guaranteed language feature from Python 3.7.

Built-in functions used on dictionaries

len(d) returns the number of key:value pairs; **all(d)**/**any(d)** test whether all/any keys are true; **sorted(d)** returns a list of the keys in sorted order; **key in d** tests whether a key is present.

len(d) key:value pairs–ന്റെ എണ്ണം നൽകുന്നു; **all(d)/any(d)** എല്ലാ/ഏതെങ്കിലും keys true ആണോ എന്ന് പരിശോധിക്കുന്നു; **sorted(d)** keys–ന്റെ sorted list നൽകുന്നു; **key in d** ഒരു key ഉണ്ടോ എന്ന് പരിശോധിക്കുന്നു.

When a dictionary is passed to a function or used in a loop, Python uses its **keys**. So `all()`/`any()` look at the keys (0 is false, non-zero is true), `sorted()` sorts the keys (`reverse=True` for descending), and `in` checks keys only, not values. `in` on a dictionary uses a hash table, so it takes about the same time however large the dictionary is, whereas `in` on a list is a linear search.

മലയാളത്തിൽ

Dictionary–യെ ഒരു function–ന് കൊടുക്കുമ്പോൾ അത് keys മാത്രമേ നോക്കൂ, values അല്ല. അതുകൊണ്ട് `'uno' in eng2sp` False ആണ്, കാരണം 'uno' ഒരു value ആണ്, key അല്ല. List–ൽ തിരയുന്നത് ഒരു ക്യൂവിൽ ഓരോരുത്തരെയും നോക്കി പോകുന്നതു പോലെയാണ്; dictionary–ൽ hash table ഉള്ളതിനാൽ locker number അറിയാവുന്നതു പോലെ നേരെ ചെന്ന് എടുക്കാം.

EXAMPLE

```
eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
print(len(eng2sp))
dict_func = {0: True, 1: False, 4: True}
print(all(dict_func), any(dict_func))
print(sorted(eng2sp))
print(sorted(eng2sp, reverse=True))
print('one' in eng2sp, 'uno' in eng2sp)
print('uno' in eng2sp.values())
```

```
3
False True
['one', 'three', 'two']
['two', 'three', 'one']
True False
True
```

`all(dict_func)` is False because the key 0 is false; the values True/False are ignored.

Exam tip: the unit says descending order needs `reverse = True` but then shows `sorted(eng2sp, reverse= False)`, which is ascending. The descending result is `['two', 'three', 'one']`.

Dictionary methods

Dictionary methods such as get(), keys(), values(), items(), update(), pop(), popitem(), setdefault(), fromkeys() and clear() read, add, change or remove key:value pairs. **View object:** the live object returned by keys(), values() and items().

get(), keys(), values(), items(), update(), pop(), popitem(), setdefault(), fromkeys(), clear() പോലുള്ള **dictionary methods** key:value pairs വായിക്കുകയോ ചേർക്കുകയോ മാറ്റുകയോ നീക്കുകയോ ചെയ്യുന്നു. **View object:** keys(), values(), items() തിരികെ നൽകുന്ന live object.

`get(key[, default])` is the safe way to read: it returns None (or your default) instead of raising KeyError. `pop(key[, default])` removes a key and returns its value. `popitem()` removes and returns the last inserted pair. `setdefault(key[, default])` returns the value if the key exists, otherwise inserts the key with the default (None if not given). `update(other)` adds or overwrites pairs and returns None. `fromkeys(seq[, value])` builds a new dictionary with the given keys all set to one value.

മലയാളത്തിൽ

d[key] ഇല്ലാത്ത key-ക്ക് KeyError തരും, എന്നാൽ get() None അല്ലെങ്കിൽ നമ്മൾ പറയുന്ന default തരും, error ഇല്ല. keys(), values(), items() എന്നിവ dictionary-യിലേക്കുള്ള ഒരു "ജനാല" (view) ആണ്; dictionary മാറിയാൽ ജനാലയിലൂടെ കാണുന്നതും മാറും. update() പുതിയ pairs ചേർക്കും, ഉള്ള key ആണെങ്കിൽ value മാറ്റും. setdefault() "ഉണ്ടെങ്കിൽ തരൂ, ഇല്ലെങ്കിൽ ഇതുവെച്ച് ചേർക്കൂ" എന്ന് പറയുന്നതു പോലെ.

Method	Syntax	What it does
clear()	d.clear()	Removes all pairs
fromkeys()	dict.fromkeys(seq[, value])	New dict with keys from seq, each set to value (default None)
get()	d.get(key[, default])	Value for key, else default (None); never KeyError
items()	d.items()	View of (key, value) tuples
keys()	d.keys()	View of keys
values()	d.values()	View of values
pop()	d.pop(key[, default])	Removes key, returns its value; KeyError if absent and no default
popitem()	d.popitem()	Removes and returns the last inserted (key, value); KeyError if empty
setdefault()	d.setdefault(key[, default])	Returns value if key exists; else inserts key with default and returns it
update()	d.update(other)	Adds/overwrites pairs from other; returns None

EXAMPLE 1: IGNOU'S MILLION_DOLLAR EXAMPLE

```
million_dollar = {"sanju": 2018, "tiger zindahai": 2017, "baahubali 2": 2017,
                  "dangal": 2016, "bajrangibhaijaan": 2015}
bolwd_million_dollar = million_dollar.fromkeys(million_dollar, "1,00,00,000")
print(bolwd_million_dollar)
print(million_dollar.get("baahubali 1"))
print(million_dollar.get("baahubali 2"))
print(million_dollar.keys())
print(million_dollar.values())
print(million_dollar.items())
million_dollar.update({"bahubali 1": 2015})
print(million_dollar)
```

```
{'sanju': '1,00,00,000', 'tiger zindahai': '1,00,00,000', 'baahubali 2':
'1,00,00,000', 'dangal': '1,00,00,000', 'bajrangibhaijaan': '1,00,00,000'}
None
2017
dict_keys(['sanju', 'tiger zindahai', 'baahubali 2', 'dangal', 'bajrangibhaijaan'])
dict_values([2018, 2017, 2017, 2016, 2015])
dict_items([('sanju', 2018), ('tiger zindahai', 2017), ('baahubali 2', 2017),
('dangal', 2016), ('bajrangibhaijaan', 2015)])
{'sanju': 2018, 'tiger zindahai': 2017, 'baahubali 2': 2017, 'dangal': 2016,
'bajrangibhaijaan': 2015, 'bahubali 1': 2015}
```

EXAMPLE 2: GET WITH DEFAULT, POP, POPITEM, SETDEFAULT, CLEAR

```
marks = {'maths': 80, 'python': 92}
print(marks.get('ai', 0))
print(marks.setdefault('python', 50), marks.setdefault('ai', 70))
print(marks)
print(marks.pop('maths'), marks)
print(marks.pop('hindi', 'not found'))
print(marks.popitem(), marks)
print(marks.update({'dbms': 65}))       # update returns None
marks.clear()
print(marks)
marks.popitem()
```

```
0
92 70
{'maths': 80, 'python': 92, 'ai': 70}
80 {'python': 92, 'ai': 70}
not found
('ai', 70) {'python': 92}
None
{}
KeyError: 'popitem(): dictionary is empty'
```

Exam tip: the unit says popitem() removes an "arbitrary" pair; since Python 3.7 it always removes the **last inserted** pair (LIFO). Its setdefault() row ("If the key is present, the default is None") is garbled: the default argument defaults to None, and if the key is present its existing value is returned unchanged. The unit also misprints `print(illion_dollar.items())` for `million_dollar.items()`.

Populating, traversing and deleting in dictionaries

Traversing a dictionary: a for loop over a dictionary visits its keys in insertion order; `d[key]` gives each value. **del d[key]** deletes the pair with that key.

Traversing a dictionary: ഒരു dictionary-യിൽ for loop ഓടിച്ചാൽ insertion order-ൽ അതിന്റെ keys ഓരോന്നായി കിട്ടും; `d[key]` ഓരോ value-ഉം തരും. **del d[key]** ആ key-യുള്ള pair നീക്കം ചെയ്യുന്നു.

A common pattern is to start with an empty dictionary `{}` and add pairs one by one, with `d.update({k: v})` or simply `d[k] = v`; a missing key is created automatically. `for key in d` gives keys; `for k, v in d.items()` gives both. Dictionaries are also handy as counters, as in the program below that counts digits, uppercase and lowercase letters in a sentence. Indentation defines the blocks in such a program.

മലയാളത്തിൽ

ഒരു കാലി dictionary എടുത്ത് ഓരോ state-ഉം തലസ്ഥാനവും ചേർക്കുന്നത് ഒരു പുതിയ ഡയറിയിൽ ഓരോ പേജ് എഴുതുന്നതു പോലെയാണ്. For loop dictionary-യിലൂടെ ഓടുമ്പോൾ keys ആണ് കിട്ടുന്നത്, value വേണമെങ്കിൽ `states[key]` എടുക്കണം. Dictionary-യെ counter ആയും ഉപയോഗിക്കാം: ഓരോ character നോക്കി digit ആണെങ്കിൽ "digits" count ഒന്ന് കൂട്ടും. Python-ൽ indentation ആണ് block നിശ്ചയിക്കുന്നത്, അത് തെറ്റിയാൽ program തെറ്റും.

EXAMPLE 1: POPULATE, TRAVERSE AND DELETE

```
states = {}
states.update({"Haryana": "Chandigarh"})
states.update({"Bihar": "Patna"})
states["west bengal"] = "kolkata"          # same effect as update
print(states)
for key in states:
    print(key, states[key])
del states["west bengal"]
print(states)
for k, v in states.items():
    print(k, "->", v)
```

```
{'Haryana': 'Chandigarh', 'Bihar': 'Patna', 'west bengal': 'kolkata'}
Haryana Chandigarh
Bihar Patna
west bengal kolkata
{'Haryana': 'Chandigarh', 'Bihar': 'Patna'}
Haryana -> Chandigarh
Bihar -> Patna
```

EXAMPLE 2: COUNT DIGITS, UPPERCASE AND LOWERCASE LETTERS (IGNOU PROGRAM)

```
sentence = input("Enter a sentence : ")
construct_dictionary = {"digits": 0, "lowercase": 0, "uppercase": 0}
for each_character in sentence:
    if each_character.isdigit():
        construct_dictionary["digits"] += 1
    elif each_character.isupper():
        construct_dictionary["uppercase"] += 1
    elif each_character.islower():
        construct_dictionary["lowercase"] += 1
print("The number of digits, lowercase and uppercase letters are:")
print(construct_dictionary)
```

```
Enter a sentence : IGNOU MCA Batch 2026
The number of digits, lowercase and uppercase letters are:
{'digits': 4, 'lowercase': 4, 'uppercase': 9}
```

Input "IGNOU MCA Batch 2026": uppercase I, G, N, O, U, M, C, A, B = 9; lowercase a, t, c, h = 4; digits 2, 0, 2, 6 = 4. Spaces match none of the conditions.

Relation between tuples and dictionaries

dict(sequence_of_pairs) builds a dictionary from a sequence of two-item tuples, taking the first item of each tuple as the key and the second as the value; `d.items()` does the reverse.

dict(sequence_of_pairs) രണ്ട് items വീതമുള്ള tuples-ന്റെ sequence-ൽ നിന്ന് dictionary ഉണ്ടാക്കുന്നു; ഓരോ tuple-ന്റെയും ആദ്യ item key-ഉം രണ്ടാമത്തേത് value-ഉം ആകുന്നു; `d.items()` ഇതിന്റെ വിപരീതം ചെയ്യുന്നു.

Tuples and dictionaries work closely together: tuples of pairs convert to dictionaries with `dict()`, `items()` returns (key, value) tuples that can be unpacked in a loop, and a tuple (of immutable items) can itself be a dictionary key.

> **മലയാളത്തിൽ**
>
> (പേര്, വർഷം) പോലുള്ള ജോഡി tuples ഒരുമിച്ച് dict()-ന് കൊടുത്താൽ ഓരോ ജോഡിയിലെയും ആദ്യത്തേത് key-ഉം രണ്ടാമത്തേത് value-ഉം ആയി dictionary കിട്ടും. തിരിച്ച്, items() dictionary-യെ വീണ്ടും tuples ആക്കി തരും. ഒരു ഡയറിയിലെ പട്ടികയെ contact list ആക്കി മാറ്റുന്നതു പോലെ.

EXAMPLE

```
pm_year = (('jln', 1947), ('lbs', 1964), ('ig', 1966), ('rg', 1984),
           ('pvn', 1991), ('abv', 1998), ('nm', 2014))
pm_year_dict = dict(pm_year)
print(pm_year_dict)
print(pm_year_dict['pvn'])
print(tuple(pm_year_dict.items())[:2])
for name, year in pm_year_dict.items():
    if year > 1990:
        print(name, year)
```

```
{'jln': 1947, 'lbs': 1964, 'ig': 1966, 'rg': 1984, 'pvn': 1991, 'abv': 1998, 'nm':
2014}
1991
(('jln', 1947), ('lbs', 1964))
pvn 1991
abv 1998
nm 2014
```

Set: creation and set operations

Set: an unordered, mutable collection of unique (immutable) items, written in curly braces { } or created with `set()`; it belongs to the built-in class `set`.

Set: curly braces { }-ൽ എഴുതുകയോ `set()` കൊണ്ട് ഉണ്ടാക്കുകയോ ചെയ്യുന്ന, unique (immutable) items-ന്റെ unordered, mutable collection; ഇത് built-in class `set`-ൽ പെടുന്നു.

Duplicates passed at creation are silently removed. Because a set is unordered, it has no indexing, slicing or `.sort()` method; use `sorted(set)` to get a sorted list. Its main uses are fast membership testing (`in`) and removing duplicates, and it supports the mathematical operations union (|), intersection (&), difference (-) and symmetric difference (^). An empty set must be written `set()`, because `{}` is an empty dictionary.

മലയാളത്തിൽ

Set ഒരു കല്യാണ guest list പോലെയാണ്: ഒരാളുടെ പേര് രണ്ട് തവണ എഴുതിയാലും ഒരിക്കലേ എണ്ണൂ. ക്രമം ഇല്ലാത്തതിനാൽ index ഇല്ല, s[0] പറ്റില്ല. Maths-ൽ പഠിച്ച union (∪), intersection (∩), difference, symmetric difference എല്ലാം |, &, −, ^ കൊണ്ട് ചെയ്യാം. Empty set വേണമെങ്കിൽ set() എന്ന് തന്നെ എഴുതണം; {} എഴുതിയാൽ കിട്ടുന്നത് empty dictionary.

a | b (union, a ∪ b): items in a, b or both

a & b (intersection, a ∩ b): items in both

a - b (difference): items in a but not in b

a ^ b (symmetric difference): items in a or b but not both

EXAMPLE 1: CREATING A SET, MEMBERSHIP, LEN AND SORTED

```
basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
print(len(basket))
print(sorted(basket))
print('orange' in basket, 'grapes' in basket)
print(type({}), type(set()))
nums = {3, 1, 2, 3, 1}
print(nums)
print(nums[0])
```

```
4
['apple', 'banana', 'orange', 'pear']
True False
<class 'dict'> <class 'set'>
{1, 2, 3}
TypeError: 'set' object is not subscriptable
```

Printing a set of strings directly shows the items in an unpredictable order that can differ between runs, so `sorted()` is used for a stable display.

EXAMPLE 2: SET OPERATIONS (IGNOU'S ABRAKADABRA EXAMPLE)

```
a = set('abrakadabra')
b = set('alexander')
print(sorted(a))
print(sorted(b))
print(sorted(a - b))
print(sorted(a | b))
print(sorted(a & b))
print(sorted(a ^ b))
```

```
['a', 'b', 'd', 'k', 'r']
['a', 'd', 'e', 'l', 'n', 'r', 'x']
['b', 'k']
['a', 'b', 'd', 'e', 'k', 'l', 'n', 'r', 'x']
['a', 'd', 'r']
['b', 'e', 'k', 'l', 'n', 'x']
```

EXAMPLE 3: UNION OF INT AND STR ITEMS (CYP-3 Q2)

```
set1 = {10, 20, 30, 40}
set2 = {50, 20, "10", 60}
set3 = set1.union(set2)
print(len(set3))
print(10 in set3, "10" in set3)
```

```
7
True True
```

10 (int) and "10" (str) are different items, so the union has 7 items: 10, 20, 30, 40, 50, 60 and '10' (answer c). Mixed types in a set are fine.

Exam tip: the unit prints sets such as `{'apple', 'banana', 'orange', 'pear'}` in alphabetical order; real output order for a set of strings is arbitrary and may change between runs. In an exam, any order is correct as long as the items are right.

Set methods and traversing sets

Set methods add or remove items (add, update, remove, discard, pop, clear), compute new sets (union, intersection, difference, symmetric_difference) or test relations (issubset, issuperset, isdisjoint). **remove()** raises KeyError for a missing item; **discard()** does not.

Set methods items ചേർക്കുകയോ നീക്കുകയോ ചെയ്യുന്നു (add, update, remove, discard, pop, clear), പുതിയ sets കണക്കാക്കുന്നു (union, intersection, difference, symmetric_difference), അല്ലെങ്കിൽ ബന്ധങ്ങൾ പരിശോധിക്കുന്നു (issubset, issuperset, isdisjoint). ഇല്ലാത്ത item–ന് **remove()** KeyError തരും; **discard()** തരില്ല.

`add(x)` inserts one item and `update(*others)` adds all items of other sets (both change the set and return None). `pop()` removes and returns an arbitrary item (KeyError if empty). `issubset(other)` is True if every item of this set is in other; `issuperset(other)` is True if every item of other is in this set; `isdisjoint(other)` is True if they share nothing. A for loop visits every item, in no guaranteed order.

മലയാളത്തിൽ

remove()-ഉം discard()-ഉം item നീക്കും; വ്യത്യാസം, ഇല്ലാത്ത item remove ചെയ്യാൻ ശ്രമിച്ചാൽ KeyError വരും, discard ഒന്നും മിണ്ടാതെ വിട്ടുകളയും. union(), intersection() പോലുള്ളവ പുതിയ set തരും, പഴയത് മാറില്ല; add(), update(), discard() പഴയ set തന്നെ മാറ്റി None തരും. pop() ഏത് item ആണ് എടുക്കുക എന്ന് മുൻകൂട്ടി പറയാൻ പറ്റില്ല, കാരണം set-ന് ക്രമമില്ല.

<table>
<tr><th>Method</th><th>Operator form</th><th>Returns / effect</th></tr>
<tr><td>add(item)</td><td>–</td><td>Adds item; None</td></tr>
<tr><td>update(*others)</td><td>a |= b</td><td>Adds all items of others; None</td></tr>
<tr><td>remove(item)</td><td>–</td><td>Removes item; KeyError if absent</td></tr>
<tr><td>discard(item)</td><td>–</td><td>Removes item if present; no error</td></tr>
<tr><td>pop()</td><td>–</td><td>Removes and returns an arbitrary item; KeyError if empty</td></tr>
<tr><td>clear()</td><td>–</td><td>Removes all items; None</td></tr>
<tr><td>union(*others)</td><td>a | b</td><td>New set: items in any of the sets</td></tr>
<tr><td>intersection(*others)</td><td>a & b</td><td>New set: items common to all sets</td></tr>
<tr><td>difference(*others)</td><td>a - b</td><td>New set: items in a but not in the others</td></tr>
<tr><td>symmetric_difference(other)</td><td>a ^ b</td><td>New set: items in exactly one of the two sets</td></tr>
<tr><td>issubset(other)</td><td>a <= b</td><td>True if every item of a is in b</td></tr>
<tr><td>issuperset(other)</td><td>a >= b</td><td>True if every item of b is in a</td></tr>
<tr><td>isdisjoint(other)</td><td>–</td><td>True if a ∩ b = ∅</td></tr>
</table>

EXAMPLE 1: IGNOU'S FLOWER PROGRAM (LABELS CORRECTED, SORTED FOR STABLE OUTPUT)

```
f1 = {'sunflower', 'roses', 'lavender', 'tulips', 'goldcrest'}
f2 = {'roses', 'tulips', 'lilies', 'daisies'}
f2.add('orchids')
print("Set f1:", sorted(f1))
print("Set f2:", sorted(f2))
print("Difference f2 - f1:", sorted(f2.difference(f1)))
print("Intersection f2 and f1:", sorted(f2.intersection(f1)))
print("Is disjoint f2, f1:", f2.isdisjoint(f1))
print("Is superset f2 of f1:", f2.issuperset(f1))
print("Is subset f2 of f1:", f2.issubset(f1))
print("Symmetric Difference f2 and f1:", sorted(f2.symmetric_difference(f1)))
print("Update f2:", f2.update(f1))
print("Discard f2 - roses:", f2.discard("roses"))
print("f2 now:", sorted(f2))
print("Clear f2:", f2.clear(), f2)
```

```
Set f1: ['goldcrest', 'lavender', 'roses', 'sunflower', 'tulips']
Set f2: ['daisies', 'lilies', 'orchids', 'roses', 'tulips']
Difference f2 - f1: ['daisies', 'lilies', 'orchids']
Intersection f2 and f1: ['roses', 'tulips']
Is disjoint f2, f1: False
Is superset f2 of f1: False
Is subset f2 of f1: False
Symmetric Difference f2 and f1: ['daisies', 'goldcrest', 'lavender', 'lilies',
'orchids', 'sunflower']
Update f2: None
Discard f2 - roses: None
f2 now: ['daisies', 'goldcrest', 'lavender', 'lilies', 'orchids', 'sunflower',
'tulips']
Clear f2: None set()
```

EXAMPLE 2: REMOVE VS DISCARD, SUBSET, POP AND TRAVERSAL

```
my_set = {1, 2, 3}
my_set.discard(5)            # no error
print(my_set)
print({1, 2}.issubset(my_set), my_set.issuperset({1, 2}))
x = my_set.pop()
print(len(my_set))
for n in {10, 20, 30}:
    print(n, "is in the set")
my_set.remove(5)
```

```
{1, 2, 3}
True True
2
10 is in the set
20 is in the set
30 is in the set
KeyError: 5
```

For small integers CPython happens to print them in ascending order, but that is not guaranteed; never rely on set order.

Exam tip: Table 18 describes symmetric_difference() as returning "a new set with items from the set set_name and all other sets" – that is the definition of union. Correct: it returns items that are in **exactly one** of the two sets (and it takes a single other set). In the unit's program, the line labelled "Intersection f2 and f1" actually prints `f2.issuperset(f1)`; the label should be "Is superset". CYP-3 Q3: `discard("Blue")` on a set without "Blue" does nothing, so the set prints unchanged (answer a).

Frozenset

Frozenset: an immutable version of a set, created with `frozenset(iterable)`; because it is hashable, it can be used as a set member or as a dictionary key.

Frozenset: `frozenset(iterable)` കൊണ്ട് ഉണ്ടാക്കുന്ന set-ന്റെ immutable രൂപം; hashable ആയതിനാൽ ഇത് മറ്റൊരു set-ലെ member ആയോ dictionary key ആയോ ഉപയോഗിക്കാം.

A frozenset supports all non-modifying set operations (union, intersection, difference, issubset, in, len, etc.) but has no add, remove, update, discard, pop or clear; calling one raises AttributeError. Passing a dictionary to `frozenset()` uses its keys. A normal set cannot be a dictionary key or a set member because it is mutable (unhashable).

മലയാളത്തിൽ

Frozenset ഒരു "ഫ്രീസറിൽ വെച്ച" set ആണ്: ഉണ്ടാക്കിയ ശേഷം ഒന്നും ചേർക്കാനോ മാറ്റാനോ പറ്റില്ല. മാറ്റം വരാത്തതിനാൽ Python-ന് അതിനെ വിശ്വസിച്ച് dictionary key ആക്കാം. ഉദാഹരണത്തിന്, ഒരു user-ന്റെ permissions ({"read", "write"}) ഒരു frozenset ആക്കി, അതിനെ key ആക്കി role കണ്ടുപിടിക്കാം.

	set	frozenset
Created with	{1, 2} or set()	frozenset(iterable) only
Mutable?	Yes	No
add/remove/update	Available	Not available (AttributeError)
Dictionary key / set member?	No (unhashable)	Yes

EXAMPLE 1: CREATING AND USING FROZENSETS

```
fs = frozenset(["g", "o", "o", "d"])
print(sorted(fs), len(fs))
pets = set([fs, "horse", "cow"])          # frozenset inside a set
print(fs in pets)
lang_used = {"english": 59, "french": 29, "spanish": 21}
print(sorted(frozenset(lang_used)))       # keys only
frs = frozenset(["german"])
lang_used[frs] = 6                        # frozenset as a dictionary key
print(lang_used)
print({set([1, 2]): "x"})
```

```
['d', 'g', 'o'] 3
True
['english', 'french', 'spanish']
{'english': 59, 'french': 29, 'spanish': 21, frozenset({'german'}): 6}
TypeError: cannot use 'set' as a dict key (unhashable type: 'set')
```

EXAMPLE 2: ROLE LOOKUP WITH FROZENSET KEYS (IGNOU PROGRAM)

```
admin_perms = frozenset(["read", "write", "delete", "manage_users"])
editor_perms = frozenset(["read", "write"])
viewer_perms = frozenset(["read"])
role_lookup = {admin_perms: "Administrator", editor_perms: "Editor",
               viewer_perms: "Viewer"}
current_user_perms = frozenset(["write", "read"])    # order does not matter
role = role_lookup.get(current_user_perms, "Unknown")
print(f"User Permissions: {sorted(current_user_perms)}")
print(f"Assigned Role: {role}")
try:
    editor_perms.add("delete")
except AttributeError as e:
    print(f"Error Attempted: {e}")
```

```
User Permissions: ['read', 'write']
Assigned Role: Editor
Error Attempted: 'frozenset' object has no attribute 'add'
```

The lookup succeeds because frozenset(["write", "read"]) equals frozenset(["read", "write"]); sets compare by content, not order.

Exam tip: IGNOU's version uses `role_lookup.get(current_user_perms, "Editor")`. With "Editor" as the default, the program prints "Editor" even when the lookup fails, so it cannot prove anything; a neutral default like "Unknown" (used above) shows that the frozenset key really matched.

Quick recap

- Identifiers: letters, digits, _; cannot start with a digit; no keywords; any length; case-sensitive. Python 3 has 35 keywords.
- Literals: string (' ', " ", """ """), numeric (int, float, complex), Boolean (True, False) and the special literal None.
- A statement is executed; an expression evaluates to a value. Assignment produces no output.
- Python is dynamically typed: variables are labels on objects; the type belongs to the object.
- / always gives a float; // floors towards −∞; ** is right-associative (2 ** 3 ** 2 = 512).
- `a and b` / `a or b` return an operand, not necessarily True/False (100 and 200 → 200).
- Every value is an object with identity (id), type (type) and value; bool is a subclass of int.
- Strings are immutable ordered sequences; index from 0, slice [a:b] excludes b.
- List [ ]: ordered, mutable, duplicates allowed; methods append, extend, insert, remove, pop, index, count, sort, reverse.
- `b = a` aliases a list; use `a[:]` or `a.copy()` to copy. `sort()` changes the list and returns None; `sorted()` returns a new list.

- Tuple ( ): ordered, immutable; the comma makes the tuple, ('a',) for one item; only count() and index().
- Tuple assignment unpacks sequences; `a, b = b, a` swaps. A list inside a tuple can still change.
- Dictionary {k: v}: keys unique and immutable, insertion-ordered (3.7+), lookup by key; get() avoids KeyError; functions and loops use keys.
- Set { }: unordered, unique items, no indexing; |, &, -, ^; remove() raises KeyError, discard() does not; empty set is set(). Frozenset is an immutable, hashable set.

Check yourself

Q. 1. What is the output of `print(2 ** 3 ** 2)`? And of `a = 100; b = 200; print(a and b)`?

** is right-associative: 3 ** 2 = 9 first, then 2 ** 9 = **512** (option d).

`and` returns the first falsy operand, or the last operand if all are truthy. 100 is truthy, so the result is **200** (option b), not True.

Q. 2. With a = 4 (0100) and b = 11 (1011), what do `print(a | b)` and `print(a >> 2)` show?

0100 | 1011 = 1111 = **15**. 4 >> 2 shifts 100 right by two places giving 1, i.e. 4 // 2^2 = **1**. Answer: (a) 15, 1.

Q. 3. State the rules for naming identifiers in Python. Is `2nd_year` valid? Is `_year2`?

An identifier may contain letters, digits and underscores; it must start with a letter or underscore, not a digit; it cannot be a keyword; no spaces or special symbols; any length; case-sensitive.

`2nd_year` is invalid (starts with a digit). `_year2` is valid.

Q. 4. Explain the difference between / and // with examples, including a negative number.

/ is true division and always returns a float: 10 / 3 = 3.3333333333333335, 16 / 2 = 8.0. // is floor division: it rounds the quotient down towards −∞: 10 // 3 = 3, 10 // 2 = 5, −7 // 2 = −4 (not −3). If either operand is a float, // returns a float: 10.0 // 3 = 3.0.

Q. 5. Given `numlist = [4, 6, 7, 10, 15, 13]`, what are `numlist[-2]`, `numlist[1:4]` and `numlist[6]`?

`numlist[-2]` = 15 (second from the end). `numlist[1:4]` = [6, 7, 10] (indices 1, 2, 3). `numlist[6]` raises `IndexError: list index out of range` because valid indices are 0–5.

Q. 6. Write the output: `x = [3, 1, 2]; y = x; y.append(0); print(x, sorted(x), x.sort(), x)`

y is an alias of x, so appending to y changes x to [3, 1, 2, 0]. The arguments are evaluated left to right: `x` → the list object, `sorted(x)` → [0, 1, 2, 3], `x.sort()` → None and sorts x in place, then `x` → the same, now sorted list. Because the first and last arguments are the same object, printed after sorting, the output is:

```
[0, 1, 2, 3] [0, 1, 2, 3] None [0, 1, 2, 3]
```

Q. 7. Which of these runs without error for `tupl = (5, 2, 7, 0, 3)`: tupl[1] = 2, tupl.append(2), tupl1 = tupl + tupl, tupl.sort()? Why can a tuple still contain a list that changes?

Only `tupl1 = tupl + tupl` (it creates a new tuple). The others try to modify an immutable tuple (TypeError / AttributeError).

Immutability applies to the tuple container: which objects sit in which positions cannot change. If one of those objects is a list, the list itself is mutable, so `t[3].append(x)` works while `t[3] = [...]` does not.

Q. 8. For `d1 = {"Ravi": 5, "Mohan": 6, "Sohan": 7}`, what do `d1[0]`, `d1.get("Ram")`, `"Ravi" in d1` and `5 in d1` give?

`d1[0]` → KeyError: 0 (dictionaries are accessed by key, not position). `d1.get("Ram")` → None. `"Ravi" in d1` → True. `5 in d1` → False, because `in` checks keys, not values.

Q. 9. Give the output of `set1 = {10, 20, 30, 40}; set2 = {50, 20, "10", 60}; print(set1.union(set2))` and of `s = {"Yellow", "Orange", "Black"}; s.discard("Blue"); print(s)`.

The union contains 10, 20, 30, 40, 50, 60 and the string '10' (int 10 and str "10" are different) – option (c); the printed order may vary.

discard() on a missing item does nothing, so the set is unchanged: {'Yellow', 'Orange', 'Black'} in some order – option (a). `remove("Blue")` would have raised KeyError: 'Blue'.

Q. 10. Differentiate between a set and a dictionary, and between a set and a frozenset.

Set vs dictionary: a set stores only unique values, a dictionary stores key:value pairs with unique keys (values may repeat). Set items cannot be accessed by index or key; dictionary values are accessed by key. Both use { } and both are mutable, but `{}` means an empty dictionary; an empty set is `set()`. Sets are used for membership tests and removing duplicates; dictionaries for storing and retrieving data by meaningful keys.

Set vs frozenset: a set is mutable (add, remove, update) and unhashable; a frozenset is immutable and hashable, so it can be a dictionary key or a member of another set.

UNIT 3

Python's Control Flow Structures

ഈ unit-ൽ program-ലെ statements ഏത് ക്രമത്തിൽ run ആകണം എന്ന് തീരുമാനിക്കുന്ന control flow statements പഠിക്കുന്നു: sequential flow, if / if...else / if...elif...else / nested if എന്നീ decision statements, while, for എന്നീ loops, പിന്നെ break, continue. Prime, factorial, Fibonacci, pattern printing പോലുള്ള exam programs-ഉം ഇവിടെ ചെയ്തു നോക്കുന്നു.

Control flow and sequential statements | The if statement | The if...else statement | The if...elif...else statement | Nested if statement | The while loop | The for loop and range() | Nested loops and pattern printing | The break and continue statements | pass and the else clause of loops | Comprehensions | More classic exam programs

---

Control flow and sequential statements

Control flow statement: a statement that decides the order in which the statements of a program are executed.
Sequential control flow: statements are executed one after another, top to bottom, in the order they appear in the program.

Control flow statement: ഒരു program-ലെ statements ഏത് ക്രമത്തിൽ execute ആകണം എന്ന് തീരുമാനിക്കുന്ന statement.
Sequential control flow: statements program-ൽ എഴുതിയ അതേ ക്രമത്തിൽ, മുകളിൽ നിന്ന് താഴേക്ക് ഒന്നിനു പിറകെ ഒന്നായി execute ആകുന്നത്.

Normally Python runs a program line by line from top to bottom. Real problems also need to decide (run a block only if a condition holds) and to repeat (run a block many times). Python therefore has three kinds of control flow: **sequential**, **decision** (if, if...else, if...elif...else) and **loop** (while, for, with break and continue).

മലയാളത്തിൽ

സാധാരണ Python ഒരു program മുകളിൽ നിന്ന് താഴേക്ക് ഓരോ വരിയായി വായിച്ച് run ചെയ്യുന്നു; ഇതാണ് sequential flow. പക്ഷേ ജീവിതത്തിലെ പോലെ program-നും ചിലപ്പോൾ തീരുമാനം എടുക്കണം ("മഴയുണ്ടെങ്കിൽ കുട എടുക്കുക"), ചിലപ്പോൾ ഒരേ കാര്യം പലതവണ ചെയ്യണം ("എല്ലാ കുട്ടികളുടെയും മാർക്ക് കൂട്ടുക"). അതിനാണ് decision statements-ഉം loops-ഉം.

Sequential: statements run in written order

Decision: if, if...else, if...elif...else, nested if

Loop (iteration / repetition): while, for; with break and continue

EXAMPLE: SEQUENTIAL PROGRAM (IGNOU)

Read three numbers and display their sum.

```
# Prompt the user to enter three numbers
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
num3 = float(input("Enter the third number: "))
# Calculate the sum
total_sum = num1 + num2 + num3
# Display the result
print(f"The sum of {num1}, {num2}, and {num3} is: {total_sum}")
```

```
Enter the first number: 10
Enter the second number: 20
Enter the third number: 30.5
The sum of 10.0, 20.0, and 30.5 is: 60.5
```

Every line runs exactly once, in order. The `f` before the string makes it an **f-string** (formatted string literal): Python replaces each `{name}` with the value of that variable or expression.

ഇവിടെ ഓരോ വരിയും ഒരു തവണ മാത്രം, എഴുതിയ ക്രമത്തിൽ run ആകുന്നു: മൂന്ന് number വായിക്കുന്നു, കൂട്ടുന്നു, print ചെയ്യുന്നു. `float()` ആയതുകൊണ്ട് 10 എന്ന് type ചെയ്താലും 10.0 എന്നാണ് കാണിക്കുന്നത്. String-ന് മുന്നിലെ `f` കാരണം `{num1}` പോലുള്ള ഭാഗങ്ങൾ variable-ന്റെ value കൊണ്ട് replace ആകുന്നു.

EXAMPLE: F-STRING WITH AN EXPRESSION AND FORMATTING

```
name = "Anu"
marks = 87.456
print(f"{name} scored {marks}")
print(f"Rounded: {marks:.1f}, next year age: {20 + 1}")
```

```
Anu scored 87.456
Rounded: 87.5, next year age: 21
```

Curly braces-ങ്ങളിൽ variable മാത്രമല്ല, `20 + 1` പോലുള്ള expression-ഉം എഴുതാം. `:.1f` എന്നാൽ ഒരു decimal place വരെ round ചെയ്ത് കാണിക്കുക എന്നാണ്.

The if statement

if statement: a decision statement that executes its indented block only when its condition (Boolean expression) evaluates to True; otherwise the block is skipped.

if statement: condition (Boolean expression) True ആകുമ്പോൾ മാത്രം അതിന്റെ indented block execute ചെയ്യുന്ന decision statement; False ആണെങ്കിൽ ആ block skip ചെയ്യപ്പെടുന്നു.

The expression after `if` is called the **condition**. The line ends with a colon, and the body is indented. The body must have at least one statement; the first unindented line marks the end of the block. Any value can act as a condition: `0`, `0.0`, `""`, empty list and `None` count as False, everything else as True, so you do not need `== True`.

മലയാളത്തിൽ

"ബസ് വന്നാൽ കയറുക" എന്നത് പോലെ: condition ശരിയാണെങ്കിൽ മാത്രം block run ആകും, അല്ലെങ്കിൽ നേരെ അടുത്ത unindented വരിയിലേക്ക് പോകും. Python-ൽ block തിരിച്ചറിയുന്നത് brackets കൊണ്ടല്ല, indentation (സാധാരണ 4 spaces) കൊണ്ടാണ്. `if` വരിയുടെ അവസാനം colon (:) നിർബന്ധം.

```
if Boolean_Expression :
    statement(s)
```

EXAMPLE 1

```
x = 7
if x > 0:
    print('x is positive')
print('done')
```

```
x is positive
done
```

x = 7 ആയതിനാൽ `x > 0` True; അതുകൊണ്ട് 'x is positive' print ആകുന്നു. 'done' if block-ന് പുറത്തായതിനാൽ എപ്പോഴും print ആകും.

EXAMPLE 2: A VARIABLE ITSELF AS THE CONDITION

```
name = "Meera"
if name:
    print("name is not empty")
count = 0
if count:
    print("this line never prints")
print("end")
```

```
name is not empty
end
```

Empty അല്ലാത്ത string True ആയി കണക്കാക്കും, അതുകൊണ്ട് ആദ്യ message വരുന്നു. count 0 ആണ്, 0 False ആയതിനാൽ രണ്ടാമത്തെ print run ആകുന്നില്ല.

EXAMPLE 3: VOTING ELIGIBILITY (IGNOU), TWO RUNS

```
age = int(input("Enter your age : "))
if age >= 18:
    print("Congratulations!!! you can vote")
    print("!!! Thanks !!!")
print("Voting is your birth right. Use judiciously")
```

```
Enter your age : 20
Congratulations!!! you can vote
!!! Thanks !!!
Voting is your birth right. Use judiciously
```

```
Enter your age : 15
Voting is your birth right. Use judiciously
```

Age 20 ആകുമ്പോൾ condition True, indented രണ്ട് വരികളും print ആകുന്നു. Age 15 ആകുമ്പോൾ False, അതിനാൽ if block മുഴുവൻ skip ചെയ്ത് അവസാനത്തെ unindented വരി മാത്രം print ആകുന്നു.

Exam tip: IGNOU's printed code uses curly quotes (“ ” ‘ ’); typed into Python these give a SyntaxError. Always use straight quotes " or '. (The unit also prints "Use judicially"; the intended word is "judiciously".)

The if...else statement

if...else statement: a two-way decision statement that executes the if block when the condition is True and the else block when it is False.

if...else statement: condition True ആണെങ്കിൽ if block-ഉം False ആണെങ്കിൽ else block-ഉം execute ചെയ്യുന്ന two-way decision statement.

The `else` part is optional and has no condition of its own. Exactly one of the two blocks runs, and then control passes to the statement after the whole if...else. `if` and `else` must be at the same indentation, and both lines end with a colon.

മലയാളത്തിൽ

ഒരു വഴി അടഞ്ഞാൽ മറ്റേ വഴി: "പാസായാൽ treat, ഇല്ലെങ്കിൽ supplementary-ക്ക് പഠിക്കൽ". രണ്ടിൽ ഒന്ന് മാത്രം നടക്കും, രണ്ടും ഒരുമിച്ച് ഒരിക്കലും നടക്കില്ല. `else`-ന് condition എഴുതാൻ പാടില്ല.

```
if Boolean_Expression :
    statement_blk_1
else :
    statement_blk_2
```

One-line form (conditional expression): value_1 if condition else value_2

EXAMPLE 1: ODD OR EVEN (IGNOU), TWO RUNS

```
num = int(input("Enter a number : "))
if num % 2 == 0:
    print(num, "is even")
else:
    print(num, "is odd")
```

```
Enter a number : 15
15 is odd
```

```
Enter a number : 42
42 is even
```

`num % 2` എന്നത് 2 കൊണ്ട് ഹരിക്കുമ്പോൾ കിട്ടുന്ന ശിഷ്ടം. ശിഷ്ടം 0 ആണെങ്കിൽ even, അല്ലെങ്കിൽ else block run ആയി odd എന്ന് പറയുന്നു. 15 odd, 42 even.

EXAMPLE 2: IF/ELSE IN ONE LINE

```
num = 15
result = "even" if num % 2 == 0 else "odd"
print(num, "is", result)
a, b = 8, 13
print("Larger:", a if a > b else b)
```

```
15 is odd
Larger: 13
```

ചെറിയ തീരുമാനങ്ങൾ ഒരു വരിയിൽ എഴുതാം: condition True ആണെങ്കിൽ `if`-ന് മുമ്പുള്ള value, False ആണെങ്കിൽ `else`-ന് ശേഷമുള്ള value. ഇതിനെ conditional expression (ternary operator) എന്ന് പറയും.

Exam tip: IGNOU's odd/even listing has `int(input("Enter a number : ")` with a missing closing bracket, and its syntax box writes `else` without a colon. Both are SyntaxErrors; the correct forms are `int(input("Enter a number : "))` and `else:` as shown above.

The if...elif...else statement

if...elif...else statement: a multi-way decision statement that tests conditions in order and executes only the block of the first True condition; if none is True, the else block runs.

if...elif...else statement: conditions ക്രമത്തിൽ പരിശോധിച്ച്, ആദ്യം True ആകുന്ന condition-ന്റെ block മാത്രം execute ചെയ്യുന്ന multi-way decision statement; ഒന്നും True അല്ലെങ്കിൽ else block run ആകും.

`elif` is short for "else if" and avoids deep indentation. You may have any number of elif parts; the else part is optional but, if present, must come last and acts as the default. As soon as one condition is True, its block runs and all remaining conditions are skipped, so the order of conditions matters.

മലയാളത്തിൽ

KSRTC ടിക്കറ്റ് കൗണ്ടറിലെ പോലെ: ആദ്യ ചോദ്യം "student ആണോ?", അല്ലെങ്കിൽ "senior citizen ആണോ?", അല്ലെങ്കിൽ full ticket. ആദ്യം ശരിയാകുന്ന ഉത്തരത്തിൽ തീരുമാനം കഴിഞ്ഞു, ബാക്കി ചോദ്യങ്ങൾ ചോദിക്കില്ല. അതുകൊണ്ട് grade program-ൽ വലിയ limit (0.9) ആദ്യം check ചെയ്യണം.

```
if Boolean_Expression_1 :
   statement_blk_1
elif Boolean_Expression_2 :
   statement_blk_2
...
else :
   statement_blk_last
```

EXAMPLE 1: GRADE FROM SCORE 0.0 TO 1.0 (IGNOU), THREE RUNS

Score ≥ 0.9 → A, ≥ 0.8 → B, ≥ 0.7 → C, ≥ 0.6 → D, < 0.6 → F; out of range → error.

```
score = float(input("Enter your score :"))
if score < 0 or score > 1:
    print("Wrong Input")
elif score >= 0.9:
    print("Your Grade is A")
elif score >= 0.8:
    print("Your Grade is B")
elif score >= 0.7:
    print("Your Grade is C")
elif score >= 0.6:
    print("Your Grade is D")
else:
    print("Your Grade is F")
```

```
Enter your score :0.85
Your Grade is B
```

```
Enter your score :0.55
Your Grade is F
```

```
Enter your score :1.5
Wrong Input
```

ആദ്യം score 0–നും 1–നും ഇടയിലാണോ എന്ന് നോക്കുന്നു. 0.85–ന് 0.9 condition False, 0.8 condition True, അതുകൊണ്ട് B; ബാക്കി elif–കൾ നോക്കുന്നില്ല. 0.55 ഒരു condition–ഉം പാലിക്കാത്തതിനാൽ else–ലെ F. 1.5 range–ന് പുറത്തായതിനാൽ "Wrong Input".

EXAMPLE 2: GRADE FROM MARKS OUT OF 100

```
marks = int(input("Enter marks (0-100): "))
if marks >= 90:
    grade = "O"
elif marks >= 75:
    grade = "A"
elif marks >= 60:
    grade = "B"
elif marks >= 40:
    grade = "C"
else:
    grade = "Fail"
print("Grade:", grade)
```

```
Enter marks (0-100): 73
Grade: B
```

73 എന്നത് 90-നും 75-നും താഴെ, പക്ഷേ 60-ന് മുകളിൽ; അതിനാൽ ആദ്യം True ആകുന്നത് `marks >= 60`, grade B.

EXAMPLE 3: LARGEST OF THREE NUMBERS

```
a, b, c = 12, 45, 29
if a >= b and a >= c:
    largest = a
elif b >= c:
    largest = b
else:
    largest = c
print("Largest is", largest)
```

```
Largest is 45
```

and ഉപയോഗിച്ച് a മറ്റ് രണ്ടിനേക്കാളും വലുതാണോ എന്ന് നോക്കുന്നു. അല്ലെങ്കിൽ b-യും c-യും മാത്രം compare ചെയ്താൽ മതി. ഇവിടെ 45 ആണ് വലുത്.

Exam tip: IGNOU's grade listing prints `"our Grade is B"` and `"our Grade is C"`; the "Y" is a typo, corrected above. Also remember: writing the conditions in the wrong order (e.g. `score >= 0.6` first) would give D to every passing score.

Nested if statement

Nested if: an if statement placed inside the if block or the else block of another if statement.

Nested if: മറ്റൊരു if statement-ന്റെ if block-ലോ else block-ലോ ഉള്ളിൽ എഴുതുന്ന if statement.

The inner condition is tested only when the outer condition leads into that block. If Boolean_Expression_1 is True, control moves to Boolean_Expression_2: True gives statement_blk_1, False gives statement_blk_2. If Boolean_Expression_1 is False, statement_blk_3 runs. Indentation shows which else belongs to which if.

> **മലയാളത്തിൽ**
>
> വീട്ടിൽ കയറിയിട്ട് മാത്രമേ അകത്തെ മുറിയുടെ വാതിൽ തുറക്കാൻ പറ്റൂ; അതുപോലെ outer condition True ആയാൽ മാത്രമേ inner if പരിശോധിക്കൂ. ഏത് else ഏത് if-ന്റേതാണ് എന്ന് indentation നോക്കിയാണ് Python മനസ്സിലാക്കുന്നത്, അതിനാൽ spaces ശ്രദ്ധിക്കുക.

```
if Boolean_Expression_1 :
  if Boolean_Expression_2 :
    statement_blk_1
  else :
    statement_blk_2
else :
  statement_blk_3
```

EXAMPLE 1: THREE-BRANCH COMPARISON (IGNOU)

```
x, y = 4, 9
if x == y:
    print('x and y are equal')
else:
    if x < y:
        print('x is less than y')
    else:
        print('x is greater than y')
```

```
x is less than y
```

x, y തുല്യമല്ലാത്തതിനാൽ outer else-ലേക്ക് പോകുന്നു. അവിടെ inner if x < y പരിശോധിക്കുന്നു; 4 < 9 True ആയതിനാൽ 'x is less than y'.

EXAMPLE 2: LEAP YEAR (IGNOU), FOUR RUNS

Rule: a year divisible by 4 is a leap year, except century years, which must be divisible by 400.

```
year = int(input("Enter a year: "))
if year % 4 == 0:
    if year % 100 == 0:
        if year % 400 == 0:
            print(year, "is a leap year")
        else:
            print(year, "is not a leap year")
    else:
        print(year, "is a leap year")
else:
    print(year, "is not a leap year")
```

```
Enter a year: 2024
2024 is a leap year
```

```
Enter a year: 1900
1900 is not a leap year
```

```
Enter a year: 2000
2000 is a leap year
```

```
Enter a year: 2023
2023 is not a leap year
```

ആദ്യം 4 കൊണ്ട് ഹരിക്കാമോ എന്ന് നോക്കുന്നു. ഹരിക്കാമെങ്കിൽ century year (100 കൊണ്ട് ഹരിക്കാവുന്നത്) ആണോ എന്ന് അകത്തെ if നോക്കുന്നു. Century year ആണെങ്കിൽ 400 കൊണ്ട് കൂടി ഹരിക്കണം. അതുകൊണ്ട് 1900 leap year അല്ല, 2000 leap year ആണ്.

EXAMPLE 3: ODD/EVEN AND POSITIVE/NEGATIVE (CHECK YOUR PROGRESS 1, Q1)

```
num = int(input("Enter a number: "))
if num == 0:
    print("Zero is even, and it is neither positive nor negative")
else:
    if num % 2 == 0:
        kind = "even"
    else:
        kind = "odd"
    if num > 0:
        sign = "positive"
    else:
        sign = "negative"
    print(num, "is", kind, "and", sign)
```

```
Enter a number: -7
-7 is odd and negative
```

```
Enter a number: 10
10 is even and positive
```

```
Enter a number: 0
Zero is even, and it is neither positive nor negative
```

0 ആണെങ്കിൽ പ്രത്യേകം പറയുന്നു. അല്ലെങ്കിൽ else block-നുള്ളിൽ രണ്ട് if...else ഉണ്ട്: ഒന്ന് odd/even, മറ്റൊന്ന് positive/negative. രണ്ടിന്റെയും ഫലം ചേർത്ത് ഒരു വരിയിൽ print ചെയ്യുന്നു.

EXAMPLE 4: EMPLOYEE BONUS (CHECK YOUR PROGRESS 1, Q2, IGNOU'S SOLUTION)

Assumed rates: grade A: DA 50%, HRA 30%, bonus 20% of gross; B: DA 40%, HRA 20%, bonus 15%; C: DA 30%, HRA 10%, bonus 10%.

```
emp_id = input("Enter Employee ID: ")
name = input("Enter Employee Name: ")
basic_pay = float(input("Enter Basic Pay: "))
grade = input("Enter Grade (A, B, or C): ").upper()
if grade == 'A':
    da = 0.50 * basic_pay
    hra = 0.30 * basic_pay
    bonus_percentage = 0.20
elif grade == 'B':
    da = 0.40 * basic_pay
    hra = 0.20 * basic_pay
    bonus_percentage = 0.15
elif grade == 'C':
    da = 0.30 * basic_pay
    hra = 0.10 * basic_pay
    bonus_percentage = 0.10
else:
    print("Invalid Grade entered! By default Grade is C.")
    grade = 'C'
    da = 0.30 * basic_pay
    hra = 0.10 * basic_pay
    bonus_percentage = 0.10
gross_salary = basic_pay + da + hra
bonus = gross_salary * bonus_percentage
print("=" * 40)
print(f"PAYSLIP FOR EMPLOYEE: {name} (ID: {emp_id})")
print("=" * 40)
print(f"Grade        : {grade}")
print(f"Basic Pay    : ₹{basic_pay:,.1f}")
print(f"DA           : ₹{da:,.1f}")
print(f"HRA          : ₹{hra:,.1f}")
print(f"Gross Salary : ₹{gross_salary:,.1f}")
print(f"Bonus        : ₹{bonus:,.1f}")
print("=" * 40)
```

```
Enter Employee ID: E101
Enter Employee Name: Ravi
Enter Basic Pay: 30000
Enter Grade (A, B, or C): b
========================================
PAYSLIP FOR EMPLOYEE: Ravi (ID: E101)
========================================
Grade        : B
Basic Pay    : ₹30,000.0
DA           : ₹12,000.0
HRA          : ₹6,000.0
Gross Salary : ₹48,000.0
Bonus        : ₹7,200.0
========================================
```

`.upper()` കാരണം 'b' എന്ന് type ചെയ്താലും 'B' ആയി മാറുന്നു. Grade അനുസരിച്ച് elif വഴി DA, HRA, bonus percentage എന്നിവ set ചെയ്യുന്നു. Gross = 30000 + 12000 + 6000 = 48000, bonus = 48000 × 0.15 = 7200. `:,.1f` ആയിരങ്ങൾക്ക് comma ഇട്ട് ഒരു decimal വരെ കാണിക്കുന്നു.

> **Exam tip:** the leap-year condition can also be written in one line: `if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:`. IGNOU gives only a screenshot for this program, so learn the nested version above.

The while loop

Loop (iteration statement): a control structure that executes a block of statements repeatedly until a termination condition is met.
while loop: an indefinite loop that keeps executing its body as long as its condition is True.

Loop (iteration statement): ഒരു termination condition പാലിക്കുന്നതുവരെ ഒരു block statements ആവർത്തിച്ച് execute ചെയ്യുന്ന control structure.
while loop: condition True ആയിരിക്കുന്നിടത്തോളം body ആവർത്തിച്ച് execute ചെയ്യുന്ന indefinite loop.

Flow: (1) evaluate the condition; (2) if False, leave the loop and continue with the next statement; (3) if True, run the body and go back to step 1. One execution of the body is an **iteration**. The variable that changes each time and decides when the loop ends is the **iteration variable**; if nothing makes the condition False, the loop never ends (an **infinite loop**).

മലയാളത്തിൽ

"ചോറ് തീരുന്നതുവരെ വിളമ്പുക" പോലെ: എത്ര തവണ എന്ന് മുൻകൂട്ടി അറിയില്ല, condition നോക്കി തുടരുന്നു. ഓരോ round-ലും ഏതെങ്കിലും variable മാറണം (ഉദാ: n = n - 1); ഇല്ലെങ്കിൽ condition എന്നും True ആയി loop ഒരിക്കലും നിൽക്കില്ല (infinite loop). അത് നിർത്താൻ Ctrl+C അമർത്തണം.

while Boolean_Expression :
statement(s)

Loop = initialise → test condition → body → update → test again ...

EXAMPLE 1: COUNTDOWN (IGNOU)

```
n = 5
while n > 0:
    print(n)
    n = n - 1
print('Job done!')
```

```
5
4
3
2
1
Job done!
```

n = 5 മുതൽ തുടങ്ങി ഓരോ iteration-ലും n print ചെയ്ത് 1 കുറയ്ക്കുന്നു. n = 0 ആകുമ്പോൾ n > 0 False, loop നിന്ന് 'Job done!' print ആകുന്നു. ആകെ 5 iterations.

EXAMPLE 2: SUM OF 1 TO N

```
n = 10
i = 1
total = 0
while i <= n:
    total = total + i
    i = i + 1
print("Sum of 1 to", n, "=", total)
```

```
Sum of 1 to 10 = 55
```

i ഒന്നിൽ തുടങ്ങി 10 വരെ ഓരോന്നായി total-ൽ കൂട്ടുന്നു. Formula $n(n + 1)/2 = 10 \times 11/2 = 55$ ഉപയോഗിച്ചും ഉത്തരം check ചെയ്യാം.

EXAMPLE 3: FIBONACCI SEQUENCE UP TO N TERMS (IGNOU), TWO RUNS

```
nterms = int(input('How many terms? '))
current = 0
previous = 1
next_term = 0
count = 0
if nterms <= 0:
    print('Please enter a positive number')
elif nterms == 1:
    print('Fibonacci Sequence')
    print('0')
else:
    print("Fibonacci Sequence")
    while count < nterms:
        print(next_term)
        current = next_term
        next_term = previous + current
        previous = current
        count += 1
```

```
How many terms? 7
Fibonacci Sequence
0
1
1
2
3
5
8
```

```
How many terms? 1
Fibonacci Sequence
0
```

ഓരോ term-ഉം മുമ്പത്തെ രണ്ട് term-കളുടെ തുകയാണ്: 0, 1, 1, 2, 3, 5, 8 … count എത്ര term print ചെയ്തു എന്ന് എണ്ണുന്നു; count = nterms ആകുമ്പോൾ loop നിൽക്കുന്നു. 0 അല്ലെങ്കിൽ negative number കൊടുത്താൽ error message കാണിക്കും.

EXAMPLE 4: SHORTER FIBONACCI USING TUPLE ASSIGNMENT

```
n = 10
a, b = 0, 1
count = 0
while count < n:
    print(a, end=" ")
    a, b = b, a + b
    count += 1
print()
```

```
0 1 1 2 3 5 8 13 21 34
```

`a, b = b, a + b` രണ്ട് values ഒരുമിച്ച് update ചെയ്യുന്നു, temporary variable വേണ്ട.

`end=" "` കാരണം എല്ലാം ഒരേ വരിയിൽ space ഇട്ട് print ആകുന്നു.

Exam tip: IGNOU's Fibonacci listing writes `elif nterms = = 1 :` (a space inside ==) and its while body is mis-indented; both would fail. The correct listing is Example 3 above. Similarly, n=n–1 in the countdown uses an en dash; type an ordinary minus sign.

The for loop and range()

for loop: a definite loop that assigns each item of a sequence, in turn, to an iteration variable and executes the loop body once for each item.
range(): a built-in function that generates a sequence of integers from start up to, but not including, stop, in steps of step.

for loop: ഒരു sequence-ലെ ഓരോ item-ഉം ക്രമത്തിൽ iteration variable-ലേക്ക് assign ചെയ്ത്, ഓരോ item-നും ഒരിക്കൽ body execute ചെയ്യുന്ന definite loop.
range(): start മുതൽ stop വരെ (stop ഉൾപ്പെടാതെ) step ഇടവിട്ട് integers generate ചെയ്യുന്ന built-in function.

A while loop is **indefinite** (runs until a condition becomes False); a for loop is **definite**: it runs once per item of a list, string, tuple, file or range. In `range([start,] stop [, step])`, start defaults to 0 and step to 1; all arguments must be integers, and step may be negative but never zero.

മലയാളത്തിൽ

ക്ലാസിലെ attendance വിളിക്കുന്നത് പോലെ: list-ലെ ഓരോ പേരും ഒന്നിനു പിറകെ ഒന്നായി എടുക്കുന്നു, list തീർന്നാൽ loop തീരും. എത്ര തവണ എന്ന് മുൻകൂട്ടി അറിയാം, അതിനാൽ definite loop. `range(1, 11)` 1 മുതൽ 10 വരെ മാത്രം തരുന്നു; stop value ഒരിക്കലും ഉൾപ്പെടില്ല എന്നത് ഓർക്കുക.

for iteration_variable in sequence :

  statement(s)

range(stop) → 0, 1, ..., stop − 1

range(start, stop) → start, start + 1, ..., stop − 1

range(start, stop, step) → start, start + step, ... (stops before reaching stop)

EXAMPLE 1: LOOPING OVER A LIST AND A STRING

```
fruits = ["apple", "banana", "cherry"]
for f in fruits:
    print(f)
for ch in "Kochi":
    print(ch, end=" ")
print()
```

```
apple
banana
cherry
K o c h i
```

ആദ്യ loop list-ലെ ഓരോ പഴവും f-ലേക്ക് എടുത്ത് print ചെയ്യുന്നു. രണ്ടാമത്തേത് string "Kochi"-യിലെ ഓരോ അക്ഷരവും എടുക്കുന്നു.

EXAMPLE 2: WHAT RANGE() PRODUCES

```
print(list(range(5)))
print(list(range(2, 10)))
print(list(range(1, 10, 2)))
print(list(range(10, 0, -3)))
print(list(range(5, 1)))
```

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6, 7, 8, 9]
[1, 3, 5, 7, 9]
[10, 7, 4, 1]
[]
```

`list()` ഉപയോഗിച്ച് range-ലെ numbers കാണാം. Negative step (−3) ഉള്ളപ്പോൾ എണ്ണം കുറഞ്ഞു വരുന്നു. `range(5, 1)` empty ആണ്, കാരണം step +1 ഉപയോഗിച്ച് 5-ൽ നിന്ന് 1-ലേക്ക് എത്താൻ പറ്റില്ല.

EXAMPLE 3: MULTIPLICATION TABLE

```
n = 7
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

i 1 മുതൽ 10 വരെ മാറുമ്പോൾ ഓരോ തവണയും 7 × i print ചെയ്യുന്നു. Stop 11 കൊടുത്തത് 10 ഉൾപ്പെടാനാണ്.

EXAMPLE 4: SUM OF ODD AND EVEN NUMBERS (IGNOU)

```
number = int(input("Enter a number: "))
even = 0
odd = 0
for i in range(number):
    if i % 2 == 0:
        even = even + i
    else:
        odd = odd + i
print("Sum of even numbers:", even)
print("Sum of odd numbers:", odd)
```

```
Enter a number: 25
Sum of even numbers: 156
Sum of odd numbers: 144
```

range(25) 0 മുതൽ 24 വരെ തരുന്നു. ഓരോ number-ഉം even ആണോ odd ആണോ എന്ന് നോക്കി അതത് total-ൽ കൂട്ടുന്നു: 2 + 4 + … + 24 = 156, 1 + 3 + … + 23 = 144. 25 എന്ന number ഉൾപ്പെടുന്നില്ല.

EXAMPLE 5: FACTORIAL (IGNOU), THREE RUNS

```
number = int(input("Enter a number: "))
factorial = 1
if number < 0:
    print("Factorial doesn't exist for negative numbers")
elif number == 0:
    print("The factorial of 0 is 1")
else:
    for i in range(1, number + 1):
        factorial = factorial * i
    print("The factorial of", number, "is", factorial)
```

```
Enter a number: 6
The factorial of 6 is 720
```

```
Enter a number: -5
Factorial doesn't exist for negative numbers
```

```
Enter a number: 0
The factorial of 0 is 1
```

factorial = 1 ൽ തുടങ്ങി 1 മുതൽ number വരെയുള്ള ഓരോ i–യും ഗുണിക്കുന്നു: $1 \times 2 \times 3 \times 4 \times 5 \times 6 = 720$. Negative number–ന് factorial ഇല്ല, 0! = 1 എന്നത് നിർവചനം.

Exam tip: to include the user's number itself, write `range(1, n + 1)`, not `range(1, n)`. Forgetting the +1 is the most common off-by-one mistake in IGNOU answers.

Nested loops and pattern printing

Nested loop: a loop inside another loop, in which the inner loop runs completely for every single iteration of the outer loop.

Nested loop: ഒരു loop–നുള്ളിൽ മറ്റൊരു loop; outer loop–ന്റെ ഓരോ iteration–ലും inner loop മുഴുവനായി run ആകുന്നു.

If the outer loop runs m times and the inner loop n times, the inner body runs m × n times. For patterns, the outer loop usually controls the **rows** and the inner loop the **columns** (items in that row). `print(..., end=" ")` stays on the same line; an empty `print()` moves to the next line.

മലയാളത്തിൽ

ക്ലോക്കിലെ മണിക്കൂർ സൂചിയും മിനിറ്റ് സൂചിയും പോലെ: മണിക്കൂർ സൂചി ഒരു പടി നീങ്ങുമ്പോൾ മിനിറ്റ് സൂചി 60 പടി മുഴുവൻ കറങ്ങും. Pattern programs-ൽ outer loop വരി (row), inner loop ആ വരിയിലെ star-കൾ.

EXAMPLE 1: EVERY ADJECTIVE WITH EVERY FRUIT (IGNOU)

```
adj = ["red", "big", "tasty"]
fruits = ["apple", "banana", "cherry"]
for x in adj:
    for y in fruits:
        print(x, y)
```

```
red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry
```

x = "red" ആയിരിക്കുമ്പോൾ inner loop മൂന്ന് പഴങ്ങളും കടന്നുപോകുന്നു; പിന്നെ x = "big", പിന്നെ "tasty". 3 × 3 = 9 വരികൾ.

EXAMPLE 2: RIGHT-ANGLED TRIANGLE OF STARS

```
n = 5
for i in range(1, n + 1):
    for j in range(i):
        print("*", end=" ")
    print()
```

```
*
* *
* * *
* * * *
* * * * *
```

i-ാം വരിയിൽ i star. Inner loop i തവണ star print ചെയ്യുന്നു, പിന്നെ `print()` പുതിയ വരിയിലേക്ക് മാറ്റുന്നു.

EXAMPLE 3: NUMBER TRIANGLE

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Star-ന് പകരം inner loop-ലെ j തന്നെ print ചെയ്യുന്നു, അതിനാൽ ഓരോ വരിയും 1 മുതൽ i വരെ.

EXAMPLE 4: PYRAMID

```
n = 4
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

```
   *
  ***
 *****
*******
```

i-ാം വരിയിൽ ആദ്യം (n − i) spaces, പിന്നെ (2i − 1) stars. String-നെ number കൊണ്ട് ഗുണിച്ചാൽ ("*" * 3 → ***) അത് ആവർത്തിക്കും; അതുകൊണ്ട് ഇവിടെ inner loop വേണ്ട.

EXAMPLE 5: INVERTED TRIANGLE

```
n = 5
for i in range(n, 0, -1):
    print("* " * i)
```

```
* * * * *
* * * *
* * *
* *
*
```

`range(n, 0, -1)` 5, 4, 3, 2, 1 തരുന്നു, അതിനാൽ star-കളുടെ എണ്ണം ഓരോ വരിയിലും കുറയുന്നു.

The break and continue statements

break: immediately terminates the loop and transfers control to the first statement after the loop.
continue: skips the remaining statements of the current iteration and passes control to the next iteration of the loop.

break: loop ഉടനെ അവസാനിപ്പിച്ച് loop-ന് ശേഷമുള്ള ആദ്യ statement-ലേക്ക് control കൈമാറുന്നു.
continue: നിലവിലെ iteration-ലെ ബാക്കി statements skip ചെയ്ത് loop-ന്റെ അടുത്ത iteration-ലേക്ക് control കൈമാറുന്നു.

Both work in while and for loops and are almost always placed inside an if. `break` leaves the loop completely; `continue` stays in the loop but jumps to the next round. In a nested loop, they affect only the innermost loop that contains them.

മലയാളത്തിൽ

ബസിൽ യാത്ര ചെയ്യുമ്പോൾ: break എന്നാൽ യാത്ര നിർത്തി ഇറങ്ങിപ്പോകുന്നത്; continue എന്നാൽ ഈ stop-ൽ ഇറങ്ങാതെ അടുത്ത stop-ലേക്ക് പോകുന്നത്. break-ന് ശേഷം loop തീർന്നു, continue-ന് ശേഷം loop തുടരും.

EXAMPLE 1: PRINT 0 TO 5 USING BREAK (IGNOU)

```
count = 0
while True:
    if count == 6:
        break
    print(count)
    count = count + 1
print("Loop ended at count =", count)
```

```
0
1
2
3
4
5
Loop ended at count = 6
```

`while True` സ്വയം ഒരിക്കലും നിൽക്കാത്ത loop ആണ്. count 6 ആകുമ്പോൾ break loop-ൽ നിന്ന് പുറത്തുകടത്തുന്നു, അതിനാൽ 6 print ആകുന്നില്ല.

EXAMPLE 2: STOP SEARCHING AT THE FIRST NUMBER ABOVE 10

```
for n in [3, 8, 11, 4, 20]:
    if n > 10:
        print("Found", n, "- stopping")
        break
    print("Checked", n)
```

```
Checked 3
Checked 8
Found 11 - stopping
```

11 കണ്ടതോടെ break; അതിനു ശേഷമുള്ള 4, 20 ഒന്നും check ചെയ്യുന്നില്ല.

EXAMPLE 3: ONLY ODD NUMBERS UP TO 10 USING CONTINUE (IGNOU)

```
count = 0
while count < 10:
    count = count + 1
    if count % 2 == 0:
        continue
    print(count)
```

```
1
3
5
7
9
```

count even ആണെങ്കിൽ continue print skip ചെയ്ത് നേരെ അടുത്ത iteration-ലേക്ക് പോകുന്നു. Loop 10 തവണയും run ആകുന്നു, പക്ഷേ odd numbers മാത്രം print ആകുന്നു. ശ്രദ്ധിക്കുക: count കൂട്ടുന്നത് continue-ന് മുമ്പാണ്; പിന്നീടാണെങ്കിൽ infinite loop ആകും.

EXAMPLE 4: PRIME NUMBER CHECK (IGNOU), THREE RUNS

```
num = int(input("Enter a number: "))
if num <= 1:
    print(num, "is not a prime number")
else:
    is_prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        print(num, "is a prime number")
    else:
        print(num, "is not a prime number")
```

```
Enter a number: 29
29 is a prime number
```

```
Enter a number: 21
21 is not a prime number
```

```
Enter a number: 1
1 is not a prime number
```

1–ഉം അതിൽ താഴെയുള്ളതും prime അല്ല. ബാക്കി number–കൾക്ക് 2 മുതൽ √num വരെയുള്ള ഏതെങ്കിലും number കൊണ്ട് ഹരിക്കാമോ എന്ന് നോക്കുന്നു. ഒരു divisor കിട്ടിയാൽ ഉടനെ break, കാരണം ഇനി നോക്കേണ്ട കാര്യമില്ല. 21 = 3 × 7 ആയതിനാൽ prime അല്ല.

Exam tip: checking divisors only up to √n is enough, because if n = a × b then one of a, b is ≤ √n. Writing `range(2, n)` is also accepted, just slower.

pass and the else clause of loops

pass: a null statement that does nothing; it is used where a statement is syntactically required but no action is needed.
Loop else clause: a block attached to a for or while loop that executes when the loop ends normally, but not when it is exited by break.

---

pass: ഒന്നും ചെയ്യാത്ത null statement; syntax അനുസരിച്ച് ഒരു statement നിർബന്ധമുള്ള, എന്നാൽ ഒരു action-ഉം വേണ്ടാത്ത സ്ഥലത്ത് ഉപയോഗിക്കുന്നു.
Loop else clause: for അല്ലെങ്കിൽ while loop-നോട് ചേർത്ത block; loop സാധാരണ രീതിയിൽ അവസാനിക്കുമ്പോൾ run ആകും, break വഴി പുറത്തുകടന്നാൽ run ആകില്ല.

These are not in the IGNOU text, but the unit's Check Your Progress (Q8) uses a for...else, so you need them. A block can never be empty in Python, so `pass` acts as a placeholder. The loop else is handy for searches: "if the loop finished without finding anything (no break), then ...".

മലയാളത്തിൽ

Block ഒഴിച്ചിടാൻ Python സമ്മതിക്കില്ല; പിന്നീട് code എഴുതാം എന്ന് വെക്കുന്നിടത്ത് pass ഇടുക. Loop-ന്റെ else-നെ "ഒന്നും കിട്ടിയില്ലെങ്കിൽ" എന്ന് വായിക്കാം: വീട് മുഴുവൻ താക്കോൽ തിരഞ്ഞിട്ടും കിട്ടിയില്ലെങ്കിൽ (break ഇല്ല) മാത്രം "താക്കോൽ കാണാനില്ല" എന്ന് പറയുന്നു.

EXAMPLE 1: PASS

```
for i in range(5):
    if i == 2:
        pass
    else:
        print(i, end=" ")
print()
```

```
0 1 3 4
```

i = 2 ആകുമ്പോൾ pass ഒന്നും ചെയ്യുന്നില്ല, അതിനാൽ 2 print ആകുന്നില്ല. continue-ൽ നിന്ന് വ്യത്യാസം: pass-ന് ശേഷം അതേ iteration-ലെ ബാക്കി code run ആകും, continue അത് skip ചെയ്യും.

EXAMPLE 2: FOR…ELSE FOR A PRIME TEST

```
for num in [21, 29]:
    for i in range(2, num):
        if num % i == 0:
            print(num, "is not prime, divisible by", i)
            break
    else:
        print(num, "is prime")
```

```
21 is not prime, divisible by 3
29 is prime
```

21–ന് 3 എന്ന divisor കിട്ടി break ആയി, അതിനാൽ else run ആകുന്നില്ല. 29–ന് ഒരു divisor–ഉം കിട്ടാതെ loop മുഴുവൻ പൂർത്തിയായി, അപ്പോൾ else "29 is prime" എന്ന് print ചെയ്യുന്നു. Flag variable വേണ്ട.

EXAMPLE 3: ALL PRIMES BELOW 30

```
for num in range(2, 30):
    for i in range(2, num):
        if num % i == 0:
            break
    else:
        print(num, end=" ")
print()
```

```
2 3 5 7 11 13 17 19 23 29
```

Outer loop 2 മുതൽ 29 വരെ ഓരോ number–ഉം എടുക്കുന്നു; inner loop divisor തിരയുന്നു. Divisor ഇല്ലാതെ inner loop തീർന്നാൽ else ആ number print ചെയ്യുന്നു.

> **Exam tip:** the loop else pairs with the loop, so it is aligned with `for/while`, not with an `if` inside the loop. Misreading this alignment is exactly the trap in Check Your Progress Q8.

Comprehensions

List comprehension: a compact expression that builds a new list by applying an expression to each item of an iterable, optionally filtered by a condition.
Dictionary comprehension: the same idea producing a dictionary of key: value pairs.

List comprehension: ഒരു iterable-ലെ ഓരോ item-നും ഒരു expression apply ചെയ്ത് (ആവശ്യമെങ്കിൽ condition വെച്ച് filter ചെയ്ത്) പുതിയ list ഉണ്ടാക്കുന്ന ചുരുക്കിയ expression.
Dictionary comprehension: അതേ രീതിയിൽ key: value pairs ഉള്ള dictionary ഉണ്ടാക്കുന്നത്.

A comprehension is a for loop written inside brackets. It is not in the unit's text, but Check Your Progress Q9 asks about one. Square brackets give a list, curly braces with `key: value` give a dictionary.

മലയാളത്തിൽ

മൂന്നോ നാലോ വരിയുള്ള for loop ഒരു വരിയിൽ ചുരുക്കി എഴുതുന്ന രീതി. `[x * x for x in range(1, 6)]` വായിക്കുന്നത്: "1 മുതൽ 5 വരെയുള്ള ഓരോ x-നും x × x എടുത്ത് list ഉണ്ടാക്കുക".

[expression for item in iterable]
[expression for item in iterable if condition]
{key_expr: value_expr for item in iterable}

EXAMPLE

```
squares = [x * x for x in range(1, 6)]
print(squares)
evens = [x for x in range(10) if x % 2 == 0]
print(evens)
a = {i: i * i for i in range(6)}
print(a)
```

```
[1, 4, 9, 16, 25]
[0, 2, 4, 6, 8]
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

ആദ്യത്തേത് 1 മുതൽ 5 വരെയുള്ളവയുടെ squares. രണ്ടാമത്തേത് `if` ഉപയോഗിച്ച് even numbers മാത്രം എടുക്കുന്നു. മൂന്നാമത്തേത് dictionary: ഓരോ i-യും key, i × i value. range(6) 0 മുതൽ 5 വരെ മാത്രം, അതിനാൽ 6: 36 ഇല്ല.

More classic exam programs

Digit extraction: for a positive integer n, `n % 10` gives the last digit and `n // 10` removes it; repeating this in a while loop processes every digit.

Digit extraction: ഒരു positive integer n-ന് `n % 10` അവസാനത്തെ digit തരുന്നു, `n // 10` ആ digit നീക്കം ചെയ്യുന്നു; ഇത് while loop-ൽ ആവർത്തിച്ചാൽ എല്ലാ digits-ഉം process ചെയ്യാം.

Sum of digits, reversing a number, palindrome and Armstrong-number checks all use the same while-loop idea: take off the last digit, use it, and stop when the number becomes 0. These appear regularly in IGNOU term-end papers alongside prime, factorial and Fibonacci.

മലയാളത്തിൽ

ഒരു number-ന്റെ അവസാന digit എടുക്കാൻ % 10, അത് വെട്ടിമാറ്റാൻ // 10. പലഹാരപ്പാത്രത്തിൽ നിന്ന് ഓരോന്നായി എടുക്കുന്നത് പോലെ, പാത്രം (number) കാലിയാകുന്നതുവരെ (0) loop തുടരുന്നു.

EXAMPLE 1: SUM OF DIGITS AND REVERSE

```
num = int(input("Enter a number: "))
temp = num
digit_sum = 0
reverse = 0
while temp > 0:
    digit = temp % 10
    digit_sum = digit_sum + digit
    reverse = reverse * 10 + digit
    temp = temp // 10
print("Sum of digits:", digit_sum)
print("Reverse:", reverse)
```

```
Enter a number: 12345
Sum of digits: 15
Reverse: 54321
```

ഓരോ round-ലും അവസാന digit എടുക്കുന്നു: 5, 4, 3, 2, 1. അത് digit_sum-ൽ കൂട്ടുന്നു, reverse-നെ 10 കൊണ്ട് ഗുണിച്ച് digit ചേർക്കുന്നു. 1 + 2 + 3 + 4 + 5 = 15, reverse 54321.

EXAMPLE 2: PALINDROME NUMBER, TWO RUNS

```
num = int(input("Enter a number: "))
temp, rev = num, 0
while temp > 0:
    rev = rev * 10 + temp % 10
    temp //= 10
if rev == num:
    print(num, "is a palindrome")
else:
    print(num, "is not a palindrome")
```

```
Enter a number: 1221
1221 is a palindrome
```

```
Enter a number: 1234
1234 is not a palindrome
```

Number reverse ചെയ്തത് original-ന് തുല്യമാണെങ്കിൽ palindrome. 1221 തിരിച്ചാലും 1221, പക്ഷേ 1234 തിരിച്ചാൽ 4321.

EXAMPLE 3: ARMSTRONG NUMBER, TWO RUNS

```
num = int(input("Enter a number: "))
n_digits = len(str(num))
total = 0
temp = num
while temp > 0:
    d = temp % 10
    total += d ** n_digits
    temp //= 10
if total == num:
    print(num, "is an Armstrong number")
else:
    print(num, "is not an Armstrong number")
```

```
Enter a number: 153
153 is an Armstrong number
```

```
Enter a number: 123
123 is not an Armstrong number
```

ഓരോ digit-നെയും digits-ന്റെ എണ്ണം എന്ന power-ലേക്ക് ഉയർത്തി കൂട്ടുന്നു. 153-ന് $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$, അതിനാൽ Armstrong number. 123-ന് $1 + 8 + 27 = 36 \neq 123$.

EXAMPLE 4: MULTIPLES OF 5 BETWEEN 108 AND 205 (CHECK YOUR PROGRESS 2, Q12)

IGNOU's answer:

```
for num in range(108, 205, 1):
    if num % 5 == 0:
        print(num, end=" ")
print()
```

```
110 115 120 125 130 135 140 145 150 155 160 165 170 175 180 185 190 195 200
```

Corrected, reading "between 108 and 205" as inclusive, since 205 is itself a multiple of 5:

```
for num in range(110, 206, 5):
    print(num, end=" ")
print()
```

```
110 115 120 125 130 135 140 145 150 155 160 165 170 175 180 185 190 195 200 205
```

IGNOU-ന്റെ `range(108, 205, 1)` 204-ൽ നിൽക്കുന്നതിനാൽ 205 വിട്ടുപോകുന്നു. ആദ്യത്തെ multiple 110 ആയതിനാൽ `range(110, 206, 5)` എഴുതിയാൽ if പോലും വേണ്ട, നേരിട്ട് 5 ഇടവിട്ട് numbers കിട്ടും.

Exam tip: the unit's answer to Q12 uses `range(108,205,1)`, which stops at 204 and so misses 205. Since 205 is itself a multiple of 5 and "between 108 and 205" is normally read as inclusive, the better answer is `range(110, 206, 5)` (or keep the if and use `range(108, 206)`).

Quick recap

- Control flow = the order in which statements execute: sequential, decision, loop.
- Blocks are marked by indentation; if, elif, else, while and for lines end with a colon.
- `if` runs its block only when the condition is True; 0, "", [] and None are False.
- `if…else` is a two-way decision; exactly one block runs. One-line form: `a if cond else b`.
- `if…elif…else` is multi-way; only the first True condition's block runs, so order matters.
- Nested if = an if inside another if or else block; inner test happens only if the outer leads there.
- `while` is an indefinite loop: repeats while the condition is True; the iteration variable must change or you get an infinite loop.
- `for` is a definite loop over a sequence (list, string, range, …).
- `range(start, stop, step)`: start defaults to 0, step to 1, stop is excluded, step ≠ 0.

- In nested loops the inner loop completes fully for each outer iteration (m × n runs); outer = rows, inner = columns in patterns.
- `break` exits the loop; `continue` skips to the next iteration; `pass` does nothing.
- A loop's `else` runs only if the loop ended without break.
- Comprehensions: `[expr for x in seq if cond]`, `{k: v for x in seq}`.
- Digit programs: `n % 10` gives the last digit, `n // 10` drops it.

Check yourself

Q. 1. In a Python program, a control structure: (a) defines program-specific data structures (b) directs the order of execution of the statements (c) dictates what happens before the program starts and after it ends (d) none of these

(b). A control structure directs the order in which the statements of a program are executed.

Q. 2. Which is False regarding loops in Python? (a) Loops perform tasks repeatedly (b) a while loop repeats statements until the condition becomes False (c) a while loop repeats statements until the condition becomes True (d) a for loop can iterate through the elements of a list

(c) is the false statement. A while loop keeps running while its condition is True and stops when it becomes False, so (b) is correct and (c) is wrong.

> **Exam tip:** the unit's answer key gives (b) here; that is a mistake. The false statement is (c).

Q. 3. We can write if/else in one line in Python. (a) True (b) False

(a) True. The conditional expression `result = "even" if num % 2 == 0 else "odd"` is an if/else on one line (see Topic 3, Example 2).

> **Exam tip:** the unit's answer key says "c", but this question has only options (a) and (b). The correct answer is (a) True.

Q. 4. What is the value of x after this code runs? (a) 0 (b) 4 (c) 2 (d) 3

```
x = 0
a = 5
b = 5
if a > 0:
    if b < 0:
        x = x + 5
elif a > 5:
    x = x + 4
else:
    x = x + 2
print(x)
```

```
0
```

(a) 0. a > 0 is True, so control enters the first block and the elif/else are skipped. Inside, b < 0 is False and the inner if has no else, so x is never changed.

Q. 5. `if -3` evaluates to: (a) True (b) False

```
if -3:
    print("True")
else:
    print("False")
print(bool(-3), bool(0))
```

```
True
True False
```

(a) True. Every non-zero number is truthy; only 0 (and 0.0) is False.

Q. 6. What is the output of this nested loop over numbers = [10, 20] and items = ["Chair", "Table"]?

```
numbers = [10, 20]
items = ["Chair", "Table"]
for x in numbers:
    for y in items:
        print(x, y)
```

```
10 Chair
10 Table
20 Chair
20 Table
```

(a). For each number the inner loop runs over both items before moving to the next number.

Q. 7. X = 0; while (X < 100): X += 2; then print the value. (a) 101 (b) 99 (c) infinite loop (d) 100

```
X = 0
while (X < 100):
    X += 2
print(X)
```

```
100
```

(d) 100. X takes 2, 4, ..., 98, 100; at 100 the condition X < 100 is False and the loop stops.

Note that the unit's code literally ends with `print(x)` (lowercase). Python is case-sensitive, so as printed it fails:

```
X = 0
while (X < 100):
    X += 2
print(x)
```

```
Traceback (most recent call last):
  File "prog.py", line 4, in <module>
    print(x)
          ^
NameError: name 'x' is not defined. Did you mean: 'X'?
```

> **Exam tip:** the intended answer is (d) 100, but only with `print(X)`. If asked to find the error, point out that x and X are different names.

Q. 8. What is the value of var after this code? (a) 20 (b) 21 (c) 10 (d) 30

```
var = 10
for i in range(10):
    for j in range(2, 10, 1):
        if var % 2 == 0:
            continue
            var += 1
    var += 1
else:
    var += 1
print(var)
```

```
21
```

(b) 21. The `var += 1` after `continue` can never run. If var is even, the inner loop just continues; if odd, the if is False and nothing happens. So only the `var += 1` after the inner loop changes var: 10 times, giving 20. The loop finishes without break, so the for's `else` adds 1 more: 21.

Q. 9. Output of `a = {i: i * i for i in range(6)}; print(a)`?

(d) `{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}`. range(6) stops at 5, so there is no 6: 36 (see Topic 11 for the run).

Q. 10. What is range() and why do we need it in a for loop? Write a for loop that prints the multiples of 5 between 108 and 205.

`range([start,] stop [, step])` is a built-in function that generates integers from start (default 0) up to but not including stop, in steps of step (default 1, never 0). A for loop needs a sequence to iterate over; range() supplies a sequence of numbers, so the loop runs an exact, known number of times without building a list by hand.

Program: `for num in range(110, 206, 5): print(num, end=" ")`, which prints 110 115 ... 200 205 (full run in Topic 12, Example 4). Loops in general let one block handle any amount of data, remove repeated code, and keep programs short and clear.

UNIT 4

Functions and File Handling in Python

ഈ unit–ൽ സ്വന്തമായി function എഴുതി call ചെയ്യുന്നതും, variable–ന്റെ scope, പലതരം arguments, return, function objects, lambda എന്നിവയും പഠിക്കുന്നു. രണ്ടാം ഭാഗത്ത് Python–ൽ file തുറക്കാനും വായിക്കാനും എഴുതാനും അടയ്ക്കാനും copy, delete ചെയ്യാനും പഠിക്കുന്നു.

Functions: definition and calling · Scope of a variable: local and global · The global keyword · Required and positional arguments · Default and keyword arguments · Arbitrary arguments · Returning from a function · Function objects · Nested functions · Lambda (anonymous) functions · map() and filter() · Files, file types and opening a file · Reading from a file · Writing, appending, tell() and seek() · Closing a file and the with statement · File information, copying and deleting (os, shutil)

Functions: definition and calling

Function: a named block of code that performs a specific task and runs only when it is called.

Function definition: the `def` statement that gives a function its name, parameter list and body. **Function call:** a statement that runs the function by writing its name with parentheses and arguments.

Function: ഒരു പ്രത്യേക ജോലി ചെയ്യുന്ന, call ചെയ്യുമ്പോൾ മാത്രം run ആകുന്ന, പേരുള്ള ഒരു code block.

Function definition: function–ന് പേര്, parameter list, body എന്നിവ നൽകുന്ന `def` statement. **Function call:** function–ന്റെ പേരും parentheses–ഉം arguments–ഉം എഴുതി അതിനെ run ചെയ്യിക്കുന്ന statement.

Functions make a program **modular** (a big problem is split into smaller sub-problems) and **reusable** (define once, call many times). Python has three types of functions: **built-in** functions such as `print()`, `max()`, `input()`; **user-defined** functions written with `def`; and **anonymous** (lambda) functions. A defined function does nothing until it is called.

മലയാളത്തിൽ

ഒരു വലിയ program–നെ ചെറിയ കഷണങ്ങളാക്കി ഓരോന്നിനും ഒരു function എഴുതുന്നതാണ് modularity. ഒരു കല്യാണ സദ്യ ഒരുക്കുമ്പോൾ സാമ്പാറിന് ഒരാൾ, പായസത്തിന് വേറൊരാൾ എന്ന പോലെ. `def` കൊണ്ട് function ഉണ്ടാക്കിയാൽ മാത്രം പോരാ, അതിന്റെ പേര് വിളിച്ച് (call ചെയ്ത്) കൊടുത്താലേ അത് പ്രവർത്തിക്കൂ. ഒരു തവണ define ചെയ്താൽ എത്ര തവണ വേണമെങ്കിലും call ചെയ്യാം.

Definition: def function_name(list_of_parameters):

  statement(s) # indented body

Call: function_name(list_of_arguments)

EXAMPLE 1

A function to display HELLO WORLD, called twice.

```
def hello():
    print("HELLO WORLD")

hello()
hello()
```

```
HELLO WORLD
HELLO WORLD
```

`def hello():` എന്ന വരി function define ചെയ്യുന്നു; അതിനു താഴെ indent ചെയ്ത `print` ആണ് body. താഴെ രണ്ടു തവണ `hello()` എന്ന് call ചെയ്തതുകൊണ്ട് HELLO WORLD രണ്ടു തവണ print ആകുന്നു.

EXAMPLE 2

A function that displays the factorial of a number (n! = 1 × 2 × ... × n, and 0! = 1).

```
def factorial(n):
    fact = 1
    for i in range(1, n + 1):
        fact = fact * i
    print("Factorial of", n, "is", fact)

factorial(5)
factorial(0)
```

```
Factorial of 5 is 120
Factorial of 0 is 1
```

ഇവിടെ n ആണ് parameter. `factorial(5)` call ചെയ്യുമ്പോൾ 5 എന്ന argument n-ലേക്ക് പോകുന്നു; loop 1 മുതൽ 5 വരെ ഗുണിച്ച് 120 തരുന്നു. `factorial(0)`-ൽ loop ഒരിക്കലും run ആകാത്തതുകൊണ്ട് `fact` 1 ആയിത്തന്നെ നിൽക്കുന്നു, അതാണ് 0! = 1.

Exam tip: A function must be defined before the line that calls it runs; calling it earlier gives `NameError`. Know the difference: parameters are the names in the definition, arguments are the values passed in the call.

Scope of a variable: local and global

Scope: the part of a program in which a variable can be accessed.

Local variable: a variable created inside a function; it is accessible only within that function. **Global variable:** a variable created outside all functions; it is accessible anywhere in the program.

Scope: ഒരു variable ഉപയോഗിക്കാൻ കഴിയുന്ന program-ന്റെ ഭാഗം.

Local variable: ഒരു function-നുള്ളിൽ ഉണ്ടാക്കുന്ന variable; ആ function-നുള്ളിൽ മാത്രം ഉപയോഗിക്കാം. **Global variable:** എല്ലാ function-നും പുറത്ത് ഉണ്ടാക്കുന്ന variable; program-ൽ എവിടെയും ഉപയോഗിക്കാം.

A function can read a global variable directly. A local variable is created when the function runs and is destroyed when the function finishes, so code outside the function cannot see it. If a function assigns to a name that also exists globally, Python creates a separate local variable with that name.

മലയാളത്തിൽ

വീട്ടിലെ അടുക്കളയിലെ ഉപ്പുപാത്രം local ആണ്, ആ വീട്ടുകാർക്ക് മാത്രം. പഞ്ചായത്ത് കിണർ global ആണ്, എല്ലാവർക്കും ഉപയോഗിക്കാം. അതുപോലെ function-ങ്ങളിലെ variable പുറത്ത് കാണില്ല; function തീരുമ്പോൾ അത് ഇല്ലാതാകും. പക്ഷേ പുറത്തുള്ള global variable function-ങ്ങളിൽ നിന്ന് വായിക്കാം.

EXAMPLE 1

IGNOU's interest program. `DEFAULT_RATE` is global; `interest`, `decimal_rate` and `total_amount` are local. (IGNOU reads principal and years with `input()`; here they are fixed at 10000 and 2 so the output is reproducible.)

```
DEFAULT_RATE = 5.0          # global variable: 5% per year

def calculate_simple_interest(principal, years, rate=DEFAULT_RATE):
    interest = (principal * rate * years) / 100      # local
    return interest

def calculate_compound_interest(principal, years, rate=DEFAULT_RATE,
                                compounds_per_year=1):
    decimal_rate = rate / 100                        # local
    total_amount = principal * (1 + decimal_rate / compounds_per_year) \
                   ** (compounds_per_year * years)
    interest = total_amount - principal              # local
    return interest

principal = 10000.0
years = 2.0
print("Simple Interest:", calculate_simple_interest(principal, years))
print("Compound Interest:", round(calculate_compound_interest(principal, years), 2))
print("Rate used:", DEFAULT_RATE)
```

```
Simple Interest: 1000.0
Compound Interest: 1025.0
Rate used: 5.0
```

`DEFAULT_RATE` എല്ലാ function-നും പുറത്ത് ഉള്ളതുകൊണ്ട് രണ്ടു function-ലും അവസാനത്തെ `print`-ലും ഉപയോഗിക്കാൻ പറ്റി. Simple interest = 10000 × 5 × 2 / 100 = 1000. Compound interest = $10000 \times 1.05^2 - 10000 = 1025$. `interest` രണ്ടു function-ലും വെവ്വേറെ local variable ആണ്; പുറത്ത് അത് ഇല്ല.

EXAMPLE 2

A local variable cannot be used outside its function.

```
def show():
    msg = "I am local"
    print(msg)

show()
try:
    print(msg)
except NameError as e:
    print("NameError:", e)
```

```
I am local
NameError: name 'msg' is not defined
```

Function-ങ്ങളിൽ `msg` ശരിയായി print ആകുന്നു. പക്ഷേ function തീർന്നശേഷം പുറത്ത് `msg` ചോദിച്ചപ്പോൾ Python-ന് അങ്ങനെയൊരു പേര് അറിയില്ല, അതുകൊണ്ട് `NameError` വരുന്നു. (`try/except` ഉപയോഗിച്ചത് error message വൃത്തിയായി കാണിക്കാൻ മാത്രമാണ്.)

The global keyword

global keyword: a declaration inside a function stating that a name refers to the global variable, so the function can modify it or create it without making a new local variable.

global keyword: ഒരു പേര് global variable-നെയാണ് സൂചിപ്പിക്കുന്നത് എന്ന് function-ങ്ങളിൽ പറയുന്ന declaration; ഇതുവഴി പുതിയ local variable ഉണ്ടാക്കാതെ global variable മാറ്റാനോ പുതുതായി ഉണ്ടാക്കാനോ function-ന് കഴിയും.

By default, any assignment inside a function creates a local variable. So to change a global variable from inside a function you must first write `global name`. The same keyword can also create a brand-new global variable from inside a function.

മലയാളത്തിൽ

Function-ങ്ങളിൽ `x = ...` എന്ന് എഴുതിയാൽ Python അത് പുതിയ local x ആയി കാണും, global x മാറില്ല. "ഇത് പഞ്ചായത്ത് കിണറിലെ വെള്ളം തന്നെ" എന്ന് ആദ്യം പറയുന്നതാണ് `global x`. അതിനുശേഷം ചെയ്യുന്ന മാറ്റം global variable-ൽ തന്നെ നടക്കും.

EXAMPLE 1

IGNOU's Fun_1/Fun_2 program (indentation restored).

```
G_Var = 45                      # global variable

def Fun_1():
    L_Var = 5                   # local variable
    global G_Var
    print("Global Variable:", G_Var)
    G_Var = G_Var + 10          # changes the global G_Var
    print("Local Variable inside Function 1:", L_Var)

def Fun_2():
    G_Var = 15                  # new LOCAL G_Var, hides the global one
    print("Local Variable inside Function 2:", G_Var)
    G_Var + 10                  # computes 25 but stores it nowhere

Fun_1()
Fun_2()
print("Global Variable:", G_Var)
```

```
Global Variable: 45
Local Variable inside Function 1: 5
Local Variable inside Function 2: 15
Global Variable: 55
```

`Fun_1`-ൽ `global G_Var` ഉള്ളതുകൊണ്ട് 45 എന്നത് 55 ആയി മാറുന്നു. `Fun_2`-ൽ `global` ഇല്ല, അതുകൊണ്ട് `G_Var = 15` ഒരു പുതിയ local variable ആണ്; global-നെ തൊടുന്നില്ല. അവസാനം print ചെയ്യുമ്പോൾ global `G_Var` 55 തന്നെ.

EXAMPLE 2

Modifying a global without `global` fails; with it, it works.

```
x = 10

def test_wrong():
    x = x + 5          # x is treated as local, but has no value yet
    print(x)

def test_right():
    global x
    x = x + 5
    print("Inside:", x)

try:
    test_wrong()
except UnboundLocalError as e:
    print("UnboundLocalError:", e)

test_right()
print("Outside:", x)
```

```
UnboundLocalError: cannot access local variable 'x' where it is not associated with
a value
Inside: 15
Outside: 15
```

`test_wrong`-ൽ x-ലേക്ക് assign ചെയ്യുന്നതുകൊണ്ട് Python x-നെ local ആയി കരുതുന്നു; പക്ഷേ വലതുവശത്ത് അതിന് ഇതുവരെ value ഇല്ലാത്തതിനാൽ `UnboundLocalError`. `test_right`-ൽ `global x` പറഞ്ഞതിനാൽ global x 15 ആയി മാറുന്നു, പുറത്തും 15 കാണാം.

EXAMPLE 3

Creating a new global variable inside a function (IGNOU's corrected Check Your Progress example).

```
def f():
    global r
    r = "IGNOU"

f()
print(r)
```

```
IGNOU
```

Program തുടങ്ങുമ്പോൾ r എന്നൊരു variable ഇല്ല. `f()` run ചെയ്യുമ്പോൾ `global r` കാരണം r global ആയി ഉണ്ടാകുന്നു, അതുകൊണ്ട് പുറത്ത് `print(r)` IGNOU എന്ന് കാണിക്കുന്നു. `global r` ഇല്ലെങ്കിൽ r local ആയിരിക്കും, `print(r)` `NameError` തരും.

Exam tip: In IGNOU's `Fun_2`, the line `G_Var +10` does nothing: it calculates a value but does not assign it. To change a variable you need `G_Var = G_Var + 10` or `G_Var += 10`. Also, `global` must come before the name is used in that function.

Required and positional arguments

Argument: a value passed to a function in a call; it is assigned to the matching parameter of the definition.

Required argument: an argument for a parameter that has no default value; omitting it raises `TypeError`.
Positional argument: an argument matched to a parameter by its position (order) in the call.

Argument: function call ചെയ്യുമ്പോൾ കൊടുക്കുന്ന value; അത് definition-ലെ അനുയോജ്യമായ parameter-ലേക്ക് പോകുന്നു.

Required argument: default value ഇല്ലാത്ത parameter-ന് കൊടുക്കേണ്ട argument; കൊടുത്തില്ലെങ്കിൽ `TypeError`. **Positional argument:** call-ലെ സ്ഥാനം (order) അനുസരിച്ച് parameter-മായി ചേരുന്ന argument.

Python gives five ways to pass arguments: required, positional, default, keyword and arbitrary. With required/positional arguments the first value goes to the first parameter, the second to the second, and so on. The number must match, and if the order is wrong the program runs but gives a wrong result.

മലയാളത്തിൽ

Bus-ൽ reserved seat പോലെ: seat 1-ൽ ഇരിക്കേണ്ടയാൾ seat 1-ൽ തന്നെ ഇരിക്കണം. ആദ്യത്തെ argument ആദ്യത്തെ parameter-ലേക്ക്, രണ്ടാമത്തേത് രണ്ടാമത്തേതിലേക്ക്. എണ്ണം കുറഞ്ഞാൽ error; order തെറ്റിയാൽ error വരില്ലെങ്കിലും ഉത്തരം തെറ്റും.

EXAMPLE 1

IGNOU's `Child()` function: name and sex are required, age is optional.

```
def Child(name, sex, age=18):
    print(f"Hello {name}, you are {sex}, you are {age} years old.")

Child("Rani", "Female")
Child("Ravi", "Male", 23)
try:
    Child()
except TypeError as e:
    print("TypeError:", e)
```

```
Hello Rani, you are Female, you are 18 years old.
Hello Ravi, you are Male, you are 23 years old.
TypeError: Child() missing 2 required positional arguments: 'name' and 'sex'
```

ആദ്യ call-ൽ age കൊടുക്കാത്തതുകൊണ്ട് default 18 എടുത്തു. രണ്ടാമത്തെ call-ൽ 23 കൊടുത്തതിനാൽ അത് ഉപയോഗിച്ചു. `Child()` എന്ന് ഒന്നും കൊടുക്കാതെ വിളിച്ചപ്പോൾ required ആയ name, sex രണ്ടും missing ആണെന്ന് `TypeError` പറയുന്നു.

EXAMPLE 2

Order of positional arguments matters.

```
def power(base, exp):
    return base ** exp

print(power(2, 3))
print(power(3, 2))
```

```
8
9
```

ഒരേ രണ്ടു സംഖ്യകൾ, പക്ഷേ order മാറിയപ്പോൾ ഉത്തരം മാറി: $2^3 = 8$, $3^2 = 9$. ഇതാണ് positional argument-ന്റെ സ്വഭാവം.

Exam tip: In IGNOU's code the comment on `Child( )` says "missing 1 required positional argument: 'name'", but both `name` and `sex` are missing, so the real error (also given in the unit's text) is "missing 2 required positional arguments: 'name' and 'sex'". In the modified program the comment "TypeError" on `Child("Ravi", "Male", 23)` is wrong: that call works, as shown above.

Default and keyword arguments

Default argument: a parameter given a value in the function definition; that value is used when the call does not pass an argument for it.

Keyword (named) argument: an argument passed as `parameter_name=value`, matched by name, so it can be given in any order.

Default argument: function definition-ൽ തന്നെ value കൊടുത്ത parameter; call-ൽ അതിന് argument കൊടുത്തില്ലെങ്കിൽ ആ value ഉപയോഗിക്കും.

Keyword (named) argument: `parameter_name=value` എന്ന രൂപത്തിൽ കൊടുക്കുന്ന argument; പേര് കൊണ്ട് match ചെയ്യുന്നതിനാൽ ഏത് order-ലും കൊടുക്കാം.

Any number of parameters can have defaults, but in the definition all default parameters must come after the non-default ones. With keyword arguments position no longer matters, only the name does. In a call, positional arguments must come before keyword arguments.

മലയാളത്തിൽ

ചായക്കടയിൽ "ഒരു ചായ" എന്ന് പറഞ്ഞാൽ സാധാരണ പഞ്ചസാര ഇട്ട ചായ കിട്ടും, അതാണ് default. "പഞ്ചസാര കുറച്ച്" എന്ന് പറഞ്ഞാൽ അത് മാറും. Keyword argument-ൽ പേര് പറഞ്ഞാണ് കൊടുക്കുന്നത്, അതുകൊണ്ട് order എങ്ങനെയായാലും ശരിയായ parameter-ലേക്ക് തന്നെ value പോകും.

```
def f(a, b, c=10):   # valid: default after non-default
def f(a=1, b):       # SyntaxError
f(1, c=5, b=2)       # valid: positional first, then keywords
```

EXAMPLE 1

Default argument.

```
def greet(name, msg="Good morning"):
    print(msg + ",", name)

greet("Anu")
greet("Anu", "Good night")
```

```
Good morning, Anu
Good night, Anu
```

ആദ്യ call-ൽ `msg` കൊടുത്തില്ല, അതുകൊണ്ട് default ആയ "Good morning" വന്നു. രണ്ടാമത്തെ call-ൽ "Good night" കൊടുത്തതിനാൽ default-നു പകരം അത് ഉപയോഗിച്ചു.

EXAMPLE 2

Keyword arguments in any order, and mixing with positional ones.

```
def student(name, course, marks):
    print(name, "|", course, "|", marks)

student(marks=78, name="Rahul", course="MCS-082")
student("Meera", marks=91, course="MCS-081")
```

```
Rahul | MCS-082 | 78
Meera | MCS-081 | 91
```

ആദ്യ call-ൽ order പൂർണമായും മാറ്റിയിട്ടും പേര് പറഞ്ഞതുകൊണ്ട് ഓരോ value-ഉം ശരിയായ സ്ഥലത്ത് എത്തി. രണ്ടാമത്തേതിൽ "Meera" positional ആയി ആദ്യം, ബാക്കി keyword ആയി.

EXAMPLE 3

A default parameter before a non-default one is a syntax error (checked with `compile()` so the program keeps running).

```
try:
    compile("def f(a=1, b): pass", "demo", "exec")
except SyntaxError as e:
    print("SyntaxError:", e.msg)
```

```
SyntaxError: parameter without a default follows parameter with a default
```

Default ഉള്ള a-ക്ക് ശേഷം default ഇല്ലാത്ത b വന്നതിനാൽ Python ആ definition തന്നെ സ്വീകരിക്കുന്നില്ല. Default parameters എപ്പോഴും അവസാനം വരണം.

Arbitrary arguments

Arbitrary arguments: a parameter written with * (e.g. `*args`) that collects any number of positional arguments into a tuple.

Arbitrary arguments: * ചേർത്ത് എഴുതുന്ന parameter (ഉദാ. `*args`); എത്ര positional arguments കൊടുത്താലും അവയെല്ലാം ഒരു tuple ആയി ശേഖരിക്കുന്നു.

Use `*args` when you do not know, while writing the function, how many values will be passed. Inside the function the parameter is an ordinary tuple, so you can loop over it. A related form, `**kwargs`, collects any number of keyword arguments into a dictionary.

മലയാളത്തിൽ

കല്യാണത്തിന് എത്ര പേർ വരുമെന്ന് മുൻകൂട്ടി അറിയില്ലെങ്കിൽ ഒരു വലിയ ഹാൾ ബുക്ക് ചെയ്യും, അതുപോലെയാണ് *args. എത്ര values കൊടുത്താലും എല്ലാം ഒരു tuple-ൽ കയറും, ഒന്നും കൊടുത്തില്ലെങ്കിൽ empty tuple. **kwargs ആണെങ്കിൽ പേരും value-ഉം ചേർത്ത് ഒരു dictionary ആയി കിട്ടും.

EXAMPLE 1

Sum of any number of values.

```
def total(*nums):
    print("nums =", nums)
    return sum(nums)

print(total(1, 2))
print(total(5, 10, 15, 20))
print(total())
```

```
nums = (1, 2)
3
nums = (5, 10, 15, 20)
50
nums = ()
0
```

ഓരോ call-ലും കൊടുത്ത values nums എന്ന tuple ആയി മാറുന്നത് കാണാം. രണ്ട്, നാല്, പൂജ്യം values: മൂന്നും ഒരേ function കൈകാര്യം ചെയ്യുന്നു. Empty tuple-ന്റെ sum 0 ആണ്.

EXAMPLE 2

Any number of keyword arguments with **.

```
def profile(**info):
    print(info)
    for key, value in info.items():
        print(key, "->", value)

profile(name="Anoop", city="Kochi", course="MSc AI & ML")
```

```
{'name': 'Anoop', 'city': 'Kochi', 'course': 'MSc AI & ML'}
name -> Anoop
city -> Kochi
course -> MSc AI & ML
```

`**info` എല്ലാ keyword arguments-ഉം ഒരു dictionary ആക്കുന്നു. പിന്നെ `items()` ഉപയോഗിച്ച് ഓരോ key-value ജോഡിയും print ചെയ്യുന്നു.

Returning from a function

return statement: ends the execution of a function and sends a value back to the caller; a function without a `return` value returns `None`.

return statement: function-ന്റെ പ്രവർത്തനം അവസാനിപ്പിച്ച് ഒരു value വിളിച്ച സ്ഥലത്തേക്ക് തിരികെ അയയ്ക്കുന്നു; `return` value ഇല്ലാത്ത function `None` തിരികെ നൽകുന്നു.

A returned value can be stored, printed, used in expressions or passed to another function. `return` sends back exactly one object, but that object can be a tuple or list, so several values can be returned together. When `return` runs, the function stops immediately and the statements after it are skipped.

മലയാളത്തിൽ

`print` ഉത്തരം screen-ൽ കാണിക്കുക മാത്രം ചെയ്യുന്നു; `return` ഉത്തരം വിളിച്ചയാളുടെ കയ്യിൽ തിരികെ ഏൽപ്പിക്കുന്നു, അത് പിന്നെയും ഉപയോഗിക്കാം. കടയിൽ പോയി സാധനം വാങ്ങി വീട്ടിൽ കൊണ്ടുവന്നു തരുന്ന പോലെ. `return` ഇല്ലെങ്കിൽ കിട്ടുന്നത് None. Tuple ആക്കി ഒന്നിലധികം values ഒരുമിച്ച് return ചെയ്യാം.

```
def function_name(arg1, arg2, ...):
  ...
  return value
```

EXAMPLE 1

No `return` gives `None`; a returned average can be stored and reused.

```
def nothing():
    print("No return here")

result = nothing()
print(result)

def example(a, b, c):
    return (a + b + c) / 3

avg = example(10, 20, 30)
print("Average:", avg)
print("Double of average:", avg * 2)
```

```
No return here
None
Average: 20.0
Double of average: 40.0
```

`nothing()` ഒരു വരി print ചെയ്യുന്നു, പക്ഷേ ഒന്നും return ചെയ്യാത്തതുകൊണ്ട് `result`-ൽ `None`. `example()` average return ചെയ്യുന്നതുകൊണ്ട് അത് `avg`-ൽ സൂക്ഷിച്ച് പിന്നീട് 2 കൊണ്ട് ഗുണിക്കാനും കഴിഞ്ഞു.

EXAMPLE 2

Returning two values (sum and average) with one `return`.

```
def sum_avg(nums):
    s = sum(nums)
    avg = s / len(nums)
    return s, avg

total, average = sum_avg([4, 8, 12])
print(total, average)
print(sum_avg([4, 8, 12]))
```

```
24 8.0
(24, 8.0)
```

`return s, avg` യഥാർത്ഥത്തിൽ `(s, avg)` എന്ന ഒരൊറ്റ tuple ആണ് തിരികെ നൽകുന്നത്. അതിനെ `total, average` എന്ന രണ്ട് variables-ലേക്ക് unpack ചെയ്തു. നേരിട്ട് print ചെയ്യുമ്പോൾ tuple ആയി തന്നെ കാണാം.

EXAMPLE 3

Returning from the middle of a loop, and a function with several `return` statements.

```
def loop_demo():
    for i in range(1, 10):
        if i == 5:
            return i
        print("i =", i)

print("Returned:", loop_demo())

def sign(n):
    if n > 0:
        return "positive"
    elif n < 0:
        return "negative"
    return "zero"

print(sign(7), sign(-3), sign(0))
```

```
i = 1
i = 2
i = 3
i = 4
Returned: 5
positive negative zero
```

Loop 1 മുതൽ 9 വരെ ആണെങ്കിലും i 5 ആകുമ്പോൾ `return` run ആകുന്നു; function അവിടെ നിൽക്കുന്നു, ബാക്കി iterations നടക്കുന്നില്ല. അതുകൊണ്ട് 4 വരികൾ മാത്രം print ആകുന്നു. `sign()`-ൽ മൂന്ന് `return` ഉണ്ട്, പക്ഷേ ഓരോ call-ലും അവയിൽ ഒന്ന് മാത്രം run ആകും.

Exam tip: The unit says "a function can have only one return statement". That is not correct: a function may contain several `return` statements (as in `sign()` above); only one of them executes in a given call, and each returns exactly one object (which may be a tuple holding many values).

Function objects

First-class object: an object that can be treated like any other data: assigned to a variable, stored in a data structure, passed as an argument and returned from a function. Python functions are first-class objects.

First-class object: മറ്റേതൊരു data-യെയും പോലെ കൈകാര്യം ചെയ്യാവുന്ന object: variable-ലേക്ക് assign ചെയ്യാം, data structure-ൽ സൂക്ഷിക്കാം, argument ആയി കൊടുക്കാം, function-ൽ നിന്ന് return ചെയ്യാം. Python functions first-class objects ആണ്.

In Python everything, including a function, is an object. So a function can be (1) assigned to a variable, (2) stored as an element of a list, tuple or dictionary, (3) passed as an argument to another function, and (4) nested inside another function. Assigning a function to a variable just creates a second name for the same function; deleting one name does not delete the function while another name still refers to it.

> **മലയാളത്തിൽ**
>
> ഒരാൾക്ക് "രാജു" എന്നും വീട്ടിൽ "കുട്ടൻ" എന്നും രണ്ട് പേരുണ്ടെന്ന് കരുതുക. "രാജു" എന്ന പേര് മാറ്റിയാലും "കുട്ടൻ" എന്ന് വിളിച്ചാൽ അതേ ആൾ വരും. Function-നെ variable-ലേക്ക് assign ചെയ്യുന്നതും അതുപോലെയാണ്. Function object ആയതിനാൽ list-ലോ dictionary-യിലോ ഇടാം, മറ്റൊരു function-ന് argument ആയും കൊടുക്കാം.

EXAMPLE 1

Function assigned to a variable, then the original name deleted.

```
def check_positive(n):
    return n > 0

test = check_positive          # no parentheses: we copy the reference
print(test(35))
print(test is check_positive)

del check_positive
print(test(-4))
try:
    check_positive(35)
except NameError as e:
    print("NameError:", e)
```

```
True
True
False
NameError: name 'check_positive' is not defined
```

`test = check_positive` എന്നതിൽ () ഇല്ല, അതുകൊണ്ട് function call ചെയ്യുന്നില്ല, അതിന്റെ reference മാത്രം `test`-ലേക്ക് പോകുന്നു. is True തരുന്നത് രണ്ടും ഒരേ object ആണെന്ന് കാണിക്കുന്നു. `del` ചെയ്ത ശേഷം പഴയ പേര് കൊണ്ട് വിളിച്ചാൽ `NameError`, പക്ഷേ `test` വഴി ഇപ്പോഴും പ്രവർത്തിക്കുന്നു.

EXAMPLE 2

Built-in functions stored as dictionary values and applied to the same list.

```
ops = {"max": max, "min": min, "sum": sum, "len": len}
data = [4, 9, 1, 7]
for name, fn in ops.items():
    print(name, "->", fn(data))
```

```
max -> 9
min -> 1
sum -> 21
len -> 4
```

Dictionary-യിലെ ഓരോ value-ഉം ഒരു function ആണ്. Loop-ൽ `fn(data)` എന്ന് വിളിക്കുമ്പോൾ ഓരോ തവണയും വേറെ function ഒരേ list-ൽ run ആകുന്നു.

EXAMPLE 3

A function passed as an argument: add is passed to `display`.

```
def add(a, b):
    return a + b

def mul(a, b):
    return a * b

def display(func, x, y):
    print(func.__name__, "gives", func(x, y))

display(add, 3, 4)
display(mul, 3, 4)
```

```
add gives 7
mul gives 12
```

`display`-ന് ആദ്യ argument ആയി ഒരു function തന്നെ കൊടുക്കുന്നു. അതിനുള്ളിൽ `func(x, y)` എന്ന് വിളിക്കുമ്പോൾ ആദ്യം add, പിന്നെ `mul` run ആകുന്നു. ഇങ്ങനെ function-നെ argument ആയി എടുക്കുന്ന function-നെ higher-order function എന്ന് പറയും.

Nested functions

Nested (inner) function: a function defined inside another function; it can use the enclosing function's variables and is hidden from the global scope (encapsulation).

Nested (inner) function: മറ്റൊരു function-നുള്ളിൽ define ചെയ്യുന്ന function; പുറത്തെ function-ന്റെ variables ഉപയോഗിക്കാം, global scope-ൽ നിന്ന് മറഞ്ഞിരിക്കും (encapsulation).

The inner function's name exists only in the local namespace of the outer function, so calling it from outside gives `NameError`. Use an inner function for a small helper that is useful only inside one function, or to avoid repeating a few lines several times inside it without exposing that code to the rest of the program.

മലയാളത്തിൽ

ഒരു ഹോട്ടലിന്റെ അടുക്കളയിലെ സഹായി പോലെയാണ് inner function: ഉള്ളിൽ പണി ചെയ്യും, പക്ഷേ customer-ക്ക് നേരിട്ട് വിളിക്കാൻ പറ്റില്ല. പുറത്തെ function-ന്റെ variables (ഇവിടെ a, b) inner function-ന് നേരിട്ട് ഉപയോഗിക്കാം.

EXAMPLE 1

An inner function using the outer function's parameters.

```
def outer(a, b):
    def inner():
        return a + b          # uses outer's variables
    return inner() * 2

print(outer(3, 4))
try:
    inner()
except NameError as e:
    print("NameError:", e)
```

```
14
NameError: name 'inner' is not defined
```

`outer(3, 4)`-നുള്ളിൽ `inner()` 3 + 4 = 7 തരുന്നു, അതിനെ 2 കൊണ്ട് ഗുണിച്ച് 14. പുറത്ത് നിന്ന് `inner()` വിളിച്ചപ്പോൾ ആ പേര് global scope-ൽ ഇല്ലാത്തതുകൊണ്ട് `NameError`.

EXAMPLE 2

The outer function may return the inner function object; it can then be used outside (a closure).

```
def make_multiplier(n):
    def times(x):
        return x * n
    return times

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(21), triple(21))
```

```
42 63
```

`make_multiplier(2)` ഉള്ളിലെ `times` function-നെ തന്നെ return ചെയ്യുന്നു, അത് `n = 2` ഓർത്തുവയ്ക്കുന്നു. അതിനാൽ `double(21)` 42, `triple(21)` 63.

Exam tip: The unit says inner functions "can never be directly called outside the outer function". Their name cannot be used outside, but if the outer function returns the inner function object (Example 2), the caller can call it through the returned reference.

Lambda (anonymous) functions

Lambda (anonymous) function: a small unnamed function created with the `lambda` keyword; it can take any number of arguments but has a single expression, whose value is returned automatically.

Lambda (anonymous) function: `lambda` keyword കൊണ്ട് ഉണ്ടാക്കുന്ന പേരില്ലാത്ത ചെറിയ function; എത്ര arguments വേണമെങ്കിലും എടുക്കാം, പക്ഷേ ഒരു expression മാത്രം, അതിന്റെ value തനിയെ return ആകുന്നു.

A lambda is not declared with `def` and has no `return` keyword; the result of its one expression is returned. It suits short, one-time operations and is mostly passed to higher-order functions such as `map()`, `filter()` and `sorted()`. It can be assigned to a variable and then called like any function.

മലയാളത്തിൽ

ഒറ്റ വരിയിൽ തീരുന്ന ചെറിയ ജോലിക്ക് മുഴുവൻ def എഴുതേണ്ട കാര്യമില്ല, അപ്പോഴാണ് lambda. ഒരു ദിവസത്തെ ജോലിക്ക് വിളിക്കുന്ന പണിക്കാരനെ പോലെ: പേര് രജിസ്റ്ററിൽ വേണ്ട, പണി കഴിഞ്ഞാൽ പോകും. Colon-ന് ശേഷമുള്ള expression-ന്റെ ഉത്തരം തനിയെ return ആകുന്നു.

variable = lambda arguments : expression

cube = lambda x : x ** 3   is the same as   def cube(x): return x ** 3

EXAMPLE 1

IGNOU's two lambdas: cube of a number and sum of two numbers.

```
cube = lambda x: x ** 3
add = lambda a, b: a + b

print(cube(3))
print(add(10, 5))
print((lambda x: x * 2)(8))      # called immediately, never named
```

```
27
15
16
```

cube-ഉം add-ഉം lambda-കളെ variable-ലേക്ക് assign ചെയ്തതാണ്; അതുകൊണ്ട് സാധാരണ function പോലെ വിളിക്കാം: 3^3 = 27, 10 + 5 = 15. മൂന്നാമത്തെ വരിയിൽ പേരില്ലാതെ തന്നെ lambda ഉണ്ടാക്കി ഉടനെ 8 കൊടുത്ത് call ചെയ്തു, 16 കിട്ടി.

EXAMPLE 2

A lambda as the key for sorting students by marks.

```
students = [("Anu", 72), ("Biju", 88), ("Cini", 65)]
print(sorted(students, key=lambda s: s[1]))
```

```
[('Cini', 65), ('Anu', 72), ('Biju', 88)]
```

`lambda s: s[1]` ഓരോ tuple-ൽ നിന്നും marks (index 1) എടുത്ത് കൊടുക്കുന്നു. `sorted()` ആ marks നോക്കി കുറവിൽ നിന്ന് കൂടുതലിലേക്ക് ക്രമീകരിക്കുന്നു.

map() and filter()

map(function, iterable): applies the function to every element of the iterable and returns the results as a map object (an iterator).

filter(function, iterable): returns a filter object (an iterator) containing only those elements for which the function returns True.

map(function, iterable): iterable-ലെ ഓരോ element-ലും function പ്രയോഗിച്ച് ഫലങ്ങൾ ഒരു map object (iterator) ആയി തരുന്നു.

filter(function, iterable): function True തരുന്ന elements മാത്രം അടങ്ങിയ ഒരു filter object (iterator) തരുന്നു.

Both take a function (often a lambda) as the first argument and a list or other iterable as the second. In Python 3 they return lazy iterator objects, so wrap them in `list()` to see or reuse the values. `map()` can also take two or more lists; the function then receives one element from each.

മലയാളത്തിൽ

`map` ഒരു ഫാക്ടറിയിലെ conveyor belt പോലെ: ഓരോ സാധനത്തിലും ഒരേ പണി ചെയ്ത് എല്ലാം പുറത്തേക്ക് വിടുന്നു. `filter` അരിപ്പ പോലെ: നിബന്ധന ശരിയാകുന്നവ മാത്രം കടത്തിവിടുന്നു. Python 3-ൽ ഇവ നേരിട്ട് list തരില്ല, അതുകൊണ്ട് `list()` കൊണ്ട് പൊതിയണം.

map(function, list) → list(map(...)) to get a list
filter(function, list) → list(filter(...)) to get a list

EXAMPLE 1

map() with a user-defined function and with a lambda.

```
nums = [1, 2, 3, 4]

def square(x):
    return x * x

print(type(map(square, nums)))
print(list(map(square, nums)))
print(list(map(lambda x: x * 10, nums)))
```

```
<class 'map'>
[1, 4, 9, 16]
[10, 20, 30, 40]
```

ആദ്യ വരി കാണിക്കുന്നത് map() ഒരു list അല്ല, map object ആണ് തരുന്നത് എന്നാണ്. list() ചേർത്തപ്പോൾ ഓരോ സംഖ്യയുടെയും square കിട്ടി. മൂന്നാമത്തെ വരിയിൽ പ്രത്യേകം function എഴുതാതെ lambda ഉപയോഗിച്ച് ഓരോന്നിനെയും 10 കൊണ്ട് ഗുണിച്ചു.

EXAMPLE 2

filter() with a lambda to keep only even numbers.

```
nums = [5, 12, 17, 18, 24, 32]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)
```

```
[12, 18, 24, 32]
```

`x % 2 == 0` ശരിയാകുന്ന (ഇരട്ട) സംഖ്യകൾ മാത്രം `filter` കടത്തിവിടുന്നു; 5, 17 എന്നിവ ഒഴിവായി.

> **Exam tip:** The unit says map() and filter() "return a list". That was true in Python 2. In Python 3 they return map/filter iterator objects; printing them directly shows something like `<map object at 0x...>`. Write `list(map(...))` in exam answers.

Files, file types and opening a file

File: a named location on disk (non-volatile storage) used to store related information permanently.

Text file: a file of characters organised in lines ended by an end-of-line character, read as `str`. **Binary file:** a file of raw bytes (e.g. .png, .pdf, .mp3, .exe), read as a `bytes` object.

File: ബന്ധപ്പെട്ട വിവരങ്ങൾ സ്ഥിരമായി സൂക്ഷിക്കാൻ disk-ൽ (non-volatile storage) ഉള്ള പേരുള്ള ഒരു സ്ഥലം.

Text file: end-of-line character കൊണ്ട് വേർതിരിച്ച വരികളായുള്ള characters അടങ്ങിയ file; `str` ആയി വായിക്കുന്നു. **Binary file:** raw bytes അടങ്ങിയ file (ഉദാ. .png, .pdf, .mp3, .exe); `bytes` object ആയി വായിക്കുന്നു.

Main memory (RAM) is volatile, so data that must survive after the program ends is kept in files. Every file operation has three steps: **open** the file, **operate** on it (read, write, append), and **close** it. `open()` returns a file object; its default mode is `'r'` (read, text).

മലയാളത്തിൽ

RAM-ലെ data current പോയാൽ പോകും, ബ്ലാക്ക്ബോർഡിലെ എഴുത്ത് പോലെ. File നോട്ട്ബുക്ക് പോലെയാണ്, അടച്ചുവച്ചാലും എഴുതിയത് നിലനിൽക്കും. Text file-ൽ വായിക്കാവുന്ന അക്ഷരങ്ങൾ; image, pdf പോലുള്ളവ binary, അവയെ Python bytes ആയി അതേപടി വിടുന്നു. `open()`-ന് കൊടുക്കുന്ന mode ആണ് file-ൽ എന്ത് ചെയ്യാമെന്ന് തീരുമാനിക്കുന്നത്.

```
file = open("file_name.ext", "mode")
Steps: open → read / write / append → close
```

Mode	Meaning	If file does not exist	If file exists
'r'	Read (default)	FileNotFoundError	Cursor at start
'w'	Write	Creates it	Erases old contents
'a'	Append (write at end)	Creates it	Keeps contents, writes at end
'x'	Exclusive create	Creates it	FileExistsError
'r+'	Read and write	FileNotFoundError	Keeps contents, cursor at start
'w+'	Write and read	Creates it	Erases old contents
'a+'	Append and read	Creates it	Keeps contents, writes at end
'rb', 'wb', 'ab'	Same as 'r', 'w', 'a' but binary (bytes)	Same as the text mode	

EXAMPLE 1

Opening `Test.txt` in different modes. All files in these examples are created in the program's current folder.

```
with open("Test.txt", "w") as f:        # create / overwrite
    f.write("Line 1\n")

with open("Test.txt", "a") as f:        # add at the end
    f.write("Line 2\n")

with open("Test.txt", "r") as f:        # read
    print(f.read(), end="")

try:
    open("Test.txt", "x")               # file already exists
except FileExistsError as e:
    print("FileExistsError:", e)

try:
    open("Missing.txt")                 # default mode 'r'
except FileNotFoundError as e:
    print("FileNotFoundError:", e)
```

```
Line 1
Line 2
FileExistsError: [Errno 17] File exists: 'Test.txt'
FileNotFoundError: [Errno 2] No such file or directory: 'Missing.txt'
```

Contents of `Test.txt` now:

```
Line 1
Line 2
```

`'w'` file ഉണ്ടാക്കി Line 1 എഴുതി, `'a'` പഴയത് കളയാതെ Line 2 അവസാനം ചേർത്തു, `'r'` വായിച്ചു. File ഇപ്പോൾ ഉള്ളതിനാൽ `'x'` `FileExistsError` തന്നു. ഇല്ലാത്ത file `'r'`-ൽ തുറക്കാൻ ശ്രമിച്ചപ്പോൾ `FileNotFoundError`.

EXAMPLE 2

`'r+'` keeps the contents, `'w+'` erases them; binary mode gives `bytes`.

```
with open("demo.txt", "w") as f:
    f.write("abcdef")

with open("demo.txt", "r+") as f:
    f.write("XY")            # overwrites from the start, rest stays
with open("demo.txt") as f:
    print(f.read())

with open("demo.txt", "w+") as f:
    f.write("new")           # old text is erased first
    f.seek(0)
    print(f.read())

with open("Test.txt", "rb") as f:
    data = f.read()
print(data, type(data))
```

```
XYcdef
new
b'Line 1\nLine 2\n' <class 'bytes'>
```

Contents of `demo.txt` now:

```
new
```

`'r+'`-ൽ തുടക്കത്തിൽ നിന്ന് "XY" എഴുതിയപ്പോൾ ab മാറി XY ആയി, ബാക്കി cdef അങ്ങനെ തന്നെ. `'w+'` ആദ്യം file ശൂന്യമാക്കുന്നു, അതുകൊണ്ട് "new" മാത്രം. `'rb'`-ൽ വായിച്ചപ്പോൾ `b'...'` എന്ന bytes object കിട്ടി, \n ഉൾപ്പെടെ raw ആയി.

Exam tip: The unit describes `'w+'` as "same as 'r+' but creates a new file". The key difference to write is that `'w+'` erases an existing file while `'r+'` keeps it and fails if it does not exist.

Reading from a file

read(n): returns the whole file (or the next n characters) as one string. **readline():** returns the next single line. **readlines():** returns a list of all remaining lines.

read(n): file മുഴുവനും (അല്ലെങ്കിൽ അടുത്ത n characters) ഒരു string ആയി തരുന്നു. **readline():** അടുത്ത ഒരു വരി മാത്രം തരുന്നു. **readlines():** ബാക്കിയുള്ള എല്ലാ വരികളും ഒരു list ആയി തരുന്നു.

The file must exist to be read. Every read starts from the current cursor position and moves the cursor forward, so reads continue from where the last one stopped. Each line returned by `readline()`/`readlines()` keeps its `\n`. You can also loop over the file object directly, one line at a time.

മലയാളത്തിൽ

പുസ്തകം വായിക്കുമ്പോൾ വിരൽ വച്ച് വായിക്കുന്ന പോലെ ഒരു cursor ഉണ്ട്. `read(7)` 7 അക്ഷരം വായിച്ചാൽ വിരൽ അവിടെ നിൽക്കും; അടുത്ത `readline()` അവിടെ നിന്ന് വരിയുടെ അവസാനം വരെ മാത്രം തരും. ഓരോ വരിയുടെയും അവസാനത്തെ `\n` കൂടെ വരും എന്ന് ഓർക്കുക.

EXAMPLE 1

Create `poem.txt`, then read it with `read(n)`, `readline()` and `readlines()`.

```
with open("poem.txt", "w") as f:
    f.write("Twinkle twinkle little star\n"
            "How I wonder what you are\n"
            "Up above the world so high\n")

f = open("poem.txt", "r")
print(repr(f.read(7)))        # first 7 characters
print(repr(f.readline()))     # rest of line 1
print(repr(f.readline()))     # line 2
print(f.readlines())          # remaining lines as a list
f.close()
```

```
'Twinkle'
' twinkle little star\n'
'How I wonder what you are\n'
['Up above the world so high\n']
```

Contents of `poem.txt` now:

```
Twinkle twinkle little star
How I wonder what you are
Up above the world so high
```

`repr()` ഉപയോഗിച്ചത് quotes-ഉം \n-ഉം വ്യക്തമായി കാണാനാണ്. `read(7)` "Twinkle" തരുന്നു; തുടർന്നുള്ള `readline()` ഒന്നാം വരിയുടെ ബാക്കി മാത്രം. അടുത്ത `readline()` രണ്ടാം വരി, `readlines()` ബാക്കിയുള്ള ഒരു വരി list ആയി.

EXAMPLE 2

Reading line by line with a `for` loop and numbering the lines.

```
with open("poem.txt") as f:
    for number, line in enumerate(f, start=1):
        print(number, line.rstrip("\n"))
```

```
1 Twinkle twinkle little star
2 How I wonder what you are
3 Up above the world so high
```

File object-ൽ നേരിട്ട് loop ചെയ്താൽ ഓരോ iteration-ലും ഒരു വരി കിട്ടും. `rstrip("\n")` അവസാനത്തെ newline കളയുന്നു, അല്ലെങ്കിൽ `print`-ന്റെ newline കൂടി ചേർന്ന് ഇടയ്ക്ക് ശൂന്യ വരി വരും.

Writing, appending, tell() and seek()

write(string): writes the string to the file and returns the number of characters written.

tell(): returns the current position of the file cursor. **seek(pos):** moves the file cursor to position pos.

write(string): string file-ലേക്ക് എഴുതുകയും എഴുതിയ characters-ന്റെ എണ്ണം തിരികെ തരികയും ചെയ്യുന്നു.

tell(): file cursor ഇപ്പോൾ എവിടെയാണെന്ന് (position) തരുന്നു. **seek(pos):** file cursor-നെ pos എന്ന position-ലേക്ക് മാറ്റുന്നു.

`write()` does not add a newline by itself, so put `\n` where a line should end. `writelines(list)` writes every string in a list. Mode `'w'` starts from an empty file; mode `'a'` adds to the end. `seek(0)` takes the cursor back to the start so the file can be read again.

മലയാളത്തിൽ

`write()` തനിയെ അടുത്ത വരിയിലേക്ക് പോകില്ല, `\n` നമ്മൾ തന്നെ ചേർക്കണം. `tell()` വിരൽ ഇപ്പോൾ എത്രാമത്തെ സ്ഥാനത്താണെന്ന് പറയുന്നു, `seek(0)` വിരൽ വീണ്ടും തുടക്കത്തിലേക്ക് കൊണ്ടുവരുന്നു, cassette rewind ചെയ്യുന്ന പോലെ.

EXAMPLE 1

Write, write several lines, then append.

```
with open("notes.txt", "w") as f:
    n = f.write("Hello\n")
    print("characters written:", n)
    f.writelines(["Python\n", "Files\n"])

with open("notes.txt", "a") as f:
    f.write("Appended line\n")

with open("notes.txt") as f:
    print(f.read(), end="")
```

```
characters written: 6
Hello
Python
Files
Appended line
```

Contents of `notes.txt` now:

```
Hello
Python
Files
Appended line
```

"Hello\n"–ൽ 6 characters ഉള്ളതുകൊണ്ട് `write()` 6 return ചെയ്തു. `writelines` list–ലെ രണ്ട് വരികളും എഴുതി. പിന്നീട് `'a'` mode–ൽ തുറന്നതിനാൽ പഴയ മൂന്ന് വരികളും നിലനിന്നു, നാലാമത്തെ വരി അവസാനം ചേർന്നു.

EXAMPLE 2

Watching the cursor with `tell()` and moving it with `seek()`.

```
with open("notes.txt") as f:
    print(f.tell())
    print(f.read(5))
    print(f.tell())
    f.seek(0)
    print(f.readline(), end="")
    print(f.tell())
```

```
0
Hello
5
Hello
6
```

തുടക്കത്തിൽ cursor 0-ൽ. 5 characters ("Hello") വായിച്ചപ്പോൾ 5-ലേക്ക് നീങ്ങി. `seek(0)` തുടക്കത്തിലേക്ക് തിരിച്ചുകൊണ്ടുവന്നു; പിന്നെ ഒരു വരി ("Hello\n", 6 characters) വായിച്ചപ്പോൾ cursor 6-ൽ.

Closing a file and the with statement

close(): ends the work on a file, saves pending data and frees the resources held by the file object.

with statement: opens a file for a block of code and closes it automatically when the block ends, even if an error occurs.

close(): file-ലെ ജോലി അവസാനിപ്പിച്ച്, ബാക്കിയുള്ള data save ചെയ്ത്, file object പിടിച്ചുവച്ച resources വിട്ടുകൊടുക്കുന്നു.

with statement: ഒരു code block-നു വേണ്ടി file തുറക്കുകയും, block തീരുമ്പോൾ (error വന്നാലും) അത് തനിയെ close ചെയ്യുകയും ചെയ്യുന്നു.

If an exception happens between `open()` and `close()`, the `close()` line may never run. The `with` statement solves this: no explicit `close()` is needed, and it is the recommended way to handle files in modern Python. The attribute `file.closed` tells whether a file is closed.

മലയാളത്തിൽ

വീട്ടിൽ നിന്ന് ഇറങ്ങുമ്പോൾ വാതിൽ പൂട്ടാൻ മറന്നാൽ പ്രശ്നമാണ്; ഇടയ്ക്ക് എന്തെങ്കിലും തിരക്ക് (error) വന്നാൽ മറക്കാനും സാധ്യതയുണ്ട്. with ഒരു automatic lock പോലെയാണ്: block-ൽ നിന്ന് പുറത്തിറങ്ങുമ്പോൾ എന്തു സംഭവിച്ചാലും file തനിയെ close ആകും.

```
file.close()
with open("file_name", "mode") as f:
    statements using f    # f is closed automatically here
```

EXAMPLE 1

Manual `close()` versus `with`.

```
f = open("notes.txt")
print("before close:", f.closed)
f.close()
print("after close:", f.closed)

with open("notes.txt") as f:
    print("inside with:", f.closed)
print("after with:", f.closed)
```

```
before close: False
after close: True
inside with: False
after with: True
```

ആദ്യ ഭാഗത്ത് നമ്മൾ തന്നെ `close()` വിളിച്ചപ്പോൾ `closed` True ആയി. `with` ഉപയോഗിച്ചപ്പോൾ block-നുള്ളിൽ file തുറന്നിരിക്കുന്നു, block കഴിഞ്ഞ ഉടനെ `close()` എഴുതാതെ തന്നെ അടഞ്ഞു.

EXAMPLE 2

Even when an error occurs inside the block, `with` still closes the file.

```
try:
    with open("notes.txt") as f:
        first = f.readline()
        number = int(first)          # "Hello" is not a number
except ValueError as e:
    print("ValueError:", e)
print("file closed?", f.closed)
```

```
ValueError: invalid literal for int() with base 10: 'Hello\n'
file closed? True
```

"Hello\n" integer ആക്കാൻ ശ്രമിച്ചപ്പോൾ `ValueError` വന്നു, program block-ൽ നിന്ന് പാതിവഴിയിൽ പുറത്തുചാടി. എന്നിട്ടും `f.closed` True: `with` file സുരക്ഷിതമായി അടച്ചു.

Exam tip: For a Windows path inside a string, either double each backslash (`"C:\\data\\My.txt"`, as the unit says), use a raw string (`r"C:\data\My.txt"`) or use forward slashes (`"C:/data/My.txt"`). A single backslash can form escape codes like `\n` or `\t`.

File information, copying and deleting (os, shutil)

Module: a file of Python code (functions, variables) that is brought into a program with the `import` keyword; `os` and `shutil` are standard-library modules for file and folder operations.

Module: `import` keyword കൊണ്ട് program-ലേക്ക് കൊണ്ടുവരുന്ന Python code (functions, variables) അടങ്ങിയ file; file, folder operations-നുള്ള standard-library modules ആണ് `os`, `shutil`.

These modules come with Python, so nothing needs installing. `os.path` functions answer questions about a file (does it exist, is it a file or a folder, what is its size), `shutil.copy()` copies a file, and `os.remove()`/`os.rmdir()` delete a file or an empty folder. `os.listdir()` lists the entries in a folder.

മലയാളത്തിൽ

File-ന്റെ കാര്യങ്ങൾ അന്വേഷിക്കാനും copy ചെയ്യാനും delete ചെയ്യാനും Python-ന്റെ കൂടെ തന്നെ വരുന്ന tool box-കളാണ് `os`, `shutil`. `import` ചെയ്താൽ മതി. Folder delete ചെയ്യാൻ `os.rmdir()`, പക്ഷേ folder ശൂന്യമായിരിക്കണം; ഉള്ളിൽ file ഉണ്ടെങ്കിൽ error വരും.

Function	What it does
`os.path.exists(p)`	True if a file or folder named p exists
`os.path.isfile(p)` / `os.path.isdir(p)`	True if p is a file / a folder
`os.path.getsize(p)`	Size of the file in bytes
`os.stat(p)`	Status of the file (size `st_size`, times, permissions)
`shutil.copy(src, dst)`	Copies file src to dst
`os.remove(p)`	Deletes file p
`os.rmdir(d)`	Deletes empty folder d
`os.listdir(d)`	List of names in folder d

EXAMPLE 1

IGNOU's `My.txt` program: create and write, read back, check size and status, append, recheck size.

```
import os

with open("My.txt", "w") as f:
    f.write("IGNOU MCS-082\nPython file handling\n")

with open("My.txt") as f:
    print(f.read(), end="")

print("Size:", os.path.getsize("My.txt"), "bytes")
print("Exists:", os.path.exists("My.txt"))
print("Is file:", os.path.isfile("My.txt"), "| Is dir:", os.path.isdir("My.txt"))
print("os.stat size:", os.stat("My.txt").st_size)

with open("My.txt", "a") as f:
    f.write("Appended text\n")
print("New size:", os.path.getsize("My.txt"), "bytes")
```

```
IGNOU MCS-082
Python file handling
Size: 35 bytes
Exists: True
Is file: True | Is dir: False
os.stat size: 35
New size: 49 bytes
```

Contents of `My.txt` now:

```
IGNOU MCS-082
Python file handling
Appended text
```

ആദ്യം രണ്ട് വരികൾ എഴുതി: "IGNOU MCS-082\n" (14) + "Python file handling\n" (21) = 35 bytes. `os.stat()`-ന്റെ `st_size`-ഉം അതേ 35 തരുന്നു. "Appended text\n" (14) ചേർത്തപ്പോൾ size 49 ആയി. English അക്ഷരങ്ങൾക്ക് ഓരോന്നിനും 1 byte, \n-നും 1 byte.

EXAMPLE 2

Copying `My.txt` to `Your.txt` with `shutil.copy()`.

```
import shutil

shutil.copy("My.txt", "Your.txt")
with open("Your.txt") as f:
    print(f.read(), end="")
```

```
IGNOU MCS-082
Python file handling
Appended text
```

Contents of `Your.txt` now:

```
IGNOU MCS-082
Python file handling
Appended text
```

`shutil.copy` My.txt–ന്റെ ഉള്ളടക്കം അപ്പാടെ Your.txt–ലേക്ക് പകർത്തി; append ചെയ്ത മൂന്നാം വരി ഉൾപ്പെടെ. Your.txt ഇല്ലായിരുന്നെങ്കിലും അത് ഉണ്ടാക്കപ്പെടുന്നു.

EXAMPLE 3

Deleting a file and a folder; `os.rmdir()` fails on a non-empty folder.

```
import os, shutil

os.mkdir("trash")
shutil.copy("My.txt", "trash/Their.txt")
print(os.listdir("trash"))

try:
    os.rmdir("trash")                # folder is not empty
except OSError as e:
    print("OSError:", e)

os.remove("trash/Their.txt")
print(os.listdir("trash"))
os.rmdir("trash")
print("trash exists?", os.path.exists("trash"))
```

```
['Their.txt']
OSError: [Errno 39] Directory not empty: 'trash'
[]
trash exists? False
```

`trash` എന്ന folder ഉണ്ടാക്കി അതിൽ Their.txt copy ചെയ്തു. File ഉള്ളപ്പോൾ `os.rmdir` "Directory not empty" error തന്നു. `os.remove` കൊണ്ട് file കളഞ്ഞ ശേഷം folder ശൂന്യമായി (`[]`), അപ്പോൾ `os.rmdir` വിജയിച്ചു.

Exam tip: The unit says you should move to the file's directory before `os.remove()`. That is not required: you can pass a relative or absolute path such as `os.remove("trash/Their.txt")`. Also, `os.path.getctime()` gives the creation time only on Windows; on Linux it is the time of the last metadata change.

Quick recap

- A function is a named block of code for one task; define it with `def name(params):` and run it with `name(args)`. Types: built-in, user-defined, anonymous (lambda).
- Functions give modularity (small sub-problems) and reusability (define once, call many times).
- Scope: local variables live inside their function only; global variables are created outside all functions and can be read anywhere.
- Assigning inside a function creates a local variable; use `global x` to modify or create a global `x`.

- Argument types: required, positional (by order), default (`p=value`, must come after non-defaults), keyword (`name=value`, any order), arbitrary (`*args` tuple, `**kwargs` dict).
- `return` ends the function and sends one object back; no return value means `None`; return a tuple to send several values.
- Functions are first-class objects: assign to variables, store in lists/dicts, pass as arguments, nest and return them.
- A nested function sees the outer function's variables and is hidden from global scope (encapsulation).
- `lambda args: expression` makes a one-expression anonymous function, often used with `map()`, `filter()`, `sorted()`.
- In Python 3 `map()` and `filter()` return iterators; use `list()` to get a list.
- Files store data permanently. Text files are read as `str`, binary files as `bytes`. Steps: open, operate, close.
- Modes: `'r'` (default), `'w'` (erase/create), `'a'` (append), `'x'` (create only), `'r+'`, `'w+'`, `'a+'`, and `'b'` for binary.
- Read with `read()`, `read(n)`, `readline()`, `readlines()`; write with `write()`; `tell()` and `seek()` handle the cursor.
- `with open(...) as f:` closes the file automatically, even on errors. `os`/`shutil` give size, existence checks, copy, remove, rmdir, listdir.

Check yourself

Q. 1. What are the benefits of using functions? Differentiate between function definition and function call.

Benefits: (i) break a large problem into small, manageable units (modularity); (ii) reuse code instead of repeating it; (iii) hide the details of an algorithm behind a clear name, making code self-documenting; (iv) make code easier to understand, test and debug.

Definition (`def name(params): body`) describes what the function does: its name, parameters and body. It runs nothing by itself. A **call** (`name(args)`) contains only the name and arguments (no body) and actually executes the function. A function is defined once but may be called many times.

Q. 2. What is the scope of a variable? Why does this program give NameError, and how do you fix it? `def f(): r = "IGNOU"` then `f(); print(r)`

Scope is the part of the program in which a variable can be accessed. Here `r` is assigned inside `f()`, so it is a local variable that disappears when `f()` ends; `print(r)` outside finds no such name. Fix it by declaring it global:

```
def f():
    global r
    r = "IGNOU"

f()
print(r)
```

```
IGNOU
```

Q. 3. Write a function named pattern to display 1 / 12 / 123 / 1234 / 12345.

```
def pattern(n):
    for i in range(1, n + 1):
        for j in range(1, i + 1):
            print(j, end="")
        print()

pattern(5)
```

```
1
12
123
1234
12345
```

Row i prints the numbers 1 to i without spaces (`end=""`), then `print()` moves to the next line.

Q. 4. Write a function multiply_all(*numbers) which multiplies all the arguments and returns the value.

```
def multiply_all(*numbers):
    product = 1
    for n in numbers:
        product = product * n
    return product

print(multiply_all(2, 3, 4))
print(multiply_all(5))
print(multiply_all())
```

```
24
5
1
```

`*numbers` collects any number of arguments into a tuple. Starting the product at 1 means no arguments gives 1 (the empty product).

Q. 5. Write a function which takes a list of numbers and returns a list of unique elements from it.

```
def unique(nums):
    result = []
    for x in nums:
        if x not in result:
            result.append(x)
    return result

print(unique([3, 1, 3, 2, 1, 5]))
```

```
[3, 1, 2, 5]
```

Each number is added only the first time it is seen, so the original order is kept. `list(set(nums))` also removes duplicates but does not guarantee the order.

Q. 6. Using lambda with map/filter: (a) cube of numbers in a list; (b) add two lists element-wise; (c) display vowels from a list of characters.

```
nums = [1, 2, 3, 4]
print(list(map(lambda x: x ** 3, nums)))

a = [1, 2, 3]
b = [10, 20, 30]
print(list(map(lambda x, y: x + y, a, b)))

chars = ["p", "y", "t", "h", "o", "n", "A", "i"]
print(list(filter(lambda c: c.lower() in "aeiou", chars)))
```

```
[1, 8, 27, 64]
[11, 22, 33]
['o', 'A', 'i']
```

(a) `map` applies the cube lambda to each element. (b) Given two lists, `map` passes one element from each to the two-argument lambda. (c) `filter` keeps only characters that are vowels; `lower()` makes it work for capitals too.

Q. 7. What are the various ways of passing arguments to a function? Give one line of code for each.

Required: `def f(a, b)` must get both, `f(1, 2)`. Positional: matched by order, `power(2, 3)` is 2^3. Default: `def greet(name, msg="Hi")`, `greet("Anu")`. Keyword: `student(marks=78, name="Rahul", course="MCS-082")`. Arbitrary: `def total(*nums)`, `total(1, 2, 3)`.

Note: the unit's answer key lists only four (required, default, keyword, arbitrary), leaving out positional, which its own Section 4.4 lists as a fifth type. Write all five.

Q. 8. Write a program to display the frequency of each word in a given file.

```
with open("words.txt", "w") as f:
    f.write("the cat and the dog\nthe dog ran\n")

freq = {}
with open("words.txt") as f:
    for word in f.read().split():
        freq[word] = freq.get(word, 0) + 1

for word, count in freq.items():
    print(word, count)
```

```
the 3
cat 1
and 1
dog 2
ran 1
```

Contents of `words.txt` now:

```
the cat and the dog
the dog ran
```

`split()` breaks the text into words at spaces and newlines. `freq.get(word, 0)` gives the current count (0 if the word is new) and 1 is added.

Q. 9. Write a program to display the first n lines from a given file, where n is given by the user.

```
n = 2          # in the exam: n = int(input("Enter n: "))
with open("poem.txt") as f:
    for i in range(n):
        line = f.readline()
        if line == "":           # file has fewer than n lines
            break
        print(line, end="")
```

```
Twinkle twinkle little star
How I wonder what you are
```

Each `readline()` gives the next line; an empty string means end of file, so the loop stops safely even if n is larger than the number of lines. (`poem.txt` is the file created in Topic 13.)

Q. 10. Write a program to display the size of a file in bytes.

```
import os

name = "My.txt"          # or name = input("File name: ")
if os.path.isfile(name):
    print(name, "is", os.path.getsize(name), "bytes")
else:
    print(name, "does not exist")
```

```
My.txt is 49 bytes
```

`os.path.getsize()` returns the size in bytes (same as `os.stat(name).st_size`). Checking `os.path.isfile()` first avoids an error for a missing file. `My.txt` is the 49-byte file from Topic 16.

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.